

Static Analysis Report — Sality Parasitic File Infector in a Signed Oracle jusched.exe

Five layers to a kernel-mode traffic filter: a signed 32-bit Java Update Scheduler carrying 69,632 bytes of viral code spliced into its relocation section, whose network component is an amsint32 kernel driver that attaches to the TCP/IP stack, blocks 23 security-vendor domains by substring, and terminates processes from kernel mode.

TLP:CLEAR

DOUGLAS MUN WITH AI ASSISTANCE

2026-09-10

Static analysis only. No stage of this sample was executed and no adversary infrastructure was contacted from the analysis environment. Hostnames appearing in this report include at least one **compromised victim site**, which must not be treated as attacker infrastructure — see the finding text before actioning any indicator.

CONTENTS

Report Classification and Handling

Sample Snapshot

Executive Summary

Disclaimer

Malware Family Identification

Chain Overview

Component Inventory

Sample Identification

Runtime Requirements

Key Findings

Capability Flow Diagram

Figure 1 — The infected host file on disk

Figure 2 — The five-layer decrypt chain

Figure 3 — Inside the 741,648-byte payload

Figure 4 — The four programs, side by side

Figure 5 — The infection routine, fcn.006a2802

Capabilities

Technical Analysis

1. The infected host (L0)
2. The entry-point transfer (L0 → L1)
3. The viral body decryptor (L1 → L2)
4. The decrypted loader body (L2)
5. The UPX layer (L2 → L3 → L4)
6. The decrypted body is a memory image (L2, revisited)
7. The kernel driver — the sample's network component
8. The code island — 90,516 bytes belonging to no program

9. The infection routine
 10. The public key with no consumer
 11. The download surface
 12. Adjudicated false positives and tool failures
 13. Two string-derived readings, reversed
- Behavioral and Memory Analysis

External Intelligence Corroboration

What corroborates, and what it is worth

Correction 21 — a vendor technique applied to the wrong layer

VirusTotal

Indicators of Compromise

File hashes

Network indicators

Host artefacts

Signature and structural indicators

Excluded by decision

Adjudicated false positives — do not alert

Negative Findings

Assessment Limitations

Methodological omissions, declared

What is missing because the artefact is missing

Unresolved within the available bytes

Instrument limitations that bound specific claims

Declared-but-omitted sections

Recommendations

1. Replace every infected binary. Do not attempt disinfection. (Finding 1f, §N10)
2. Hunt for the kernel driver, not for the file infection. (Findings 7a–7f)
3. Treat "outbound SMTP silently fails" as an indicator, not as a mail problem. (Finding 7b)
4. Search for hosts, not just for this sample. (Findings 1b–1d)
5. Verify signatures properly, and fix the tooling that does not. (Finding 1d)
6. Assume credential compromise on any host that ran this. (Capabilities table; §N9 is scoped)
7. Restore AV and firewall configuration by policy, not by inspection. (Capabilities table)
8. Next analysis steps, if this specimen is pursued further

Detection Opportunities

Highest value: structural, generic across the family

High value: network

High value: host

Moderate value: byte patterns — specimen-scoped, and say so in the rule

The encrypted blocklist — the single best static rule surface in the sample

Validation requirements before any of this ships

About this Report

Report Changelog

Glossary

Appendix A: Evidence and Reproduction

Hash-pinned regeneration

Disassembly evidence — opus/r2_evidence/

Decoder re-implementations — opus/reimpl/

Interpreter evidence — opus/pass2_evidence/

Figures — opus/figures/

Appendix B: Methodology

The pipeline

The layer-by-layer method

Dual-agent, union merge

Scoping every negative

Instrument validation

What was deliberately not done

Appendix B2: Tools and What Each Is Trusted For

Appendix C: Corrections Made During Audit

Appendix D: Analysis Scripts

keystream.py — the body decryptor

keystream_bytemem.py — the negative control

nrv2b.py / nrv2b_stub.py — UPX decompression

nrv2b_stub_model.py — symbolic stub model

stage_interp.py — the instruction interpreter

store_probe.py — memory-write instrumentation

Blocklist decoder (in regen.sh)

gen_api_table.py — L2 name-to-slot mapping

gen_body_asm.py — L2 disassembly

Coverage measurement (island)

Report Classification and Handling

TLP:CLEAR — may be shared publicly and re-distributed without restriction.

Nothing in this engagement was submitted to any online service. No file, no fragment, no derived artefact was uploaded anywhere. The single external disclosure was the sample's SHA-256, typed into VirusTotal's search box to retrieve a pre-existing report. No sample-derived domain, host, IP or hash was ever resolved, fetched, looked up in WHOIS, or queried against any reputation service — at any point, by any tool, including during the writing of this report.

No stage of this sample was ever executed. Every decoding step is a Python re-implementation transcribed instruction-for-instruction from the sample's own decoder stubs. No debugger, no emulator, no `upx -d`, no evaluator of any kind was applied to sample-derived text. The safety property is structural rather than procedural: a wrong re-implementation desynchronizes and raises an exception, whereas a wrong invocation of a real unpacker would silently run the sample's code.

This report contains live indicators. The Indicators of Compromise section carries defanged but functional domains, URLs and an IPv4 address belonging to a botnet disrupted 2026-09-02. Treat them as hostile.

Sample Snapshot

Malware Family	Salinity (W32.Salinity / Sector / SaliCode), operator tracked as SALTY SPIDER. High confidence , on first-hand bytes rather than vendor labels.
Key Capabilities	Parasitic PE32 file infection with a polymorphic engine; kernel-mode network traffic filtering and security-domain blocking; kernel-mode process termination; AV/firewall disablement; HTTP payload download; process injection; anti-forensic timestamp restoration.
Target Platform	Windows, x86 (32-bit). The infector's own PE parser accepts IMAGE_NT_OPTIONAL_HDR32_MAGIC only, so the infectable target set is 32-bit by construction. The kernel driver's version dispatch bounds it to six specific NtBuildNumber values.
Primary Artifact	Dropper — the infected host file, sha256:f073b23d47e6147dfef23e17170452edc57f09044fe64a872025c40ce6c70d59, 820,312 bytes.
Infection Vector	Not established first-hand. The sample <i>is</i> an infected file, so its own arrival is a downstream consequence of some earlier infection. Spread mechanisms are discussed under Negative Findings and Assessment Limitations; the autorun.inf and network-share strings are present but remain string-scoped.

Executive Summary

This report analyses a single Windows executable that arrives looking entirely trustworthy. It is Oracle's Java Update Scheduler, `jusched.exe`, version 2.8.401.10 — a real Oracle program, carrying a real Oracle code-signing certificate issued by DigiCert, with version resources and a build timestamp that match Oracle's own release. A user who inspected its properties would see nothing wrong. A security tool that merely checks whether a certificate is present would also see nothing wrong.

The file has been parasitically infected. Someone took the genuine signed Oracle binary and made two changes to it. First, they appended 69,632 bytes of viral code to the end of its relocation section — a section that, in a normal program, holds nothing but address-fixup tables and is never executable. Second, and more destructively, they overwrote 330 bytes of Oracle's own compiled code at the program's entry point with a small hand-written stub whose only job is to compute the address of the appended code and jump to it. That stub does not store its destination anywhere. It reconstructs the address at runtime through arithmetic laundered through an MMX register, so no static cross-reference to the viral body exists anywhere in the file — a property that defeated four separate analysis methods during this engagement before the arithmetic was traced by hand.

Following that jump leads through five distinct layers. The appended code decrypts itself with a hand-rolled 32-bit subtractive stream cipher, yielding a position-independent loader; that loader carries a UPX-compressed executable; decompressing it yields a 741,648-byte payload; and that payload turns out to contain three further complete Windows programs embedded inside it, one of which is a **kernel-mode driver**. That driver is the finding that matters most. For most of this engagement the sample was held to have no network component at all, on the strength of four independent methods that each found nothing. Every one of those methods searched user-mode Windows APIs. The driver does not use any — it attaches itself to the operating system's own TCP/IP stack and reads traffic as it passes, so it needs no networking imports for anyone to find. It blocks outbound SMTP outright, blocks connections to a list of twenty-three security-vendor domains held encrypted inside it, and can terminate arbitrary processes from kernel mode through a table of per-Windows-version offsets.

Two entries in that blocklist name a Sality removal tool and a malware-upload page. That is what establishes the family from the sample's own bytes rather than

from a vendor's label: this program is specifically defending itself against being removed and being submitted for analysis.

What this means for an affected organization. Sality is a file infector, not a dropped executable, and the distinction governs remediation. The infected file here is a legitimately signed Oracle binary whose signature is now cryptographically void but still physically present. Removing the appended viral code cannot restore the original file, because the infection destroyed 330 bytes of Oracle's compiled code that no longer exist anywhere in the sample — five original call sites elsewhere in the program still point into the overwritten range. **Affected files must be replaced from a trusted source, not disinfected.** Because the infector's own precondition is that a target PE retain its relocation table, and because it enumerates local drives and network shares, an infection in an estate should be assumed to have touched every reachable 32-bit executable, not only the one that was noticed. Any host that ran an infected file should also be checked for the `amsint32` kernel driver and for the AV-notification registry overrides listed in the indicators below.

The botnet this sample belonged to was disrupted by an international law-enforcement operation announced 2026-09-02, eight days before this analysis. That materially weakens dynamic evidence — a sinkholed botnet produces exactly the same empty network table as a benign file — and is the reason the network findings in this report rest on static disassembly rather than on the absence of observed traffic.

Disclaimer

This analysis is **static only**. The sample was never executed, in any form, at any stage — not in a sandbox, not under a debugger, not through an emulator, and not by invoking a third-party unpacker on it. Every layer was recovered by re-implementing the sample's own decoder logic in Python, transcribed from its disassembly.

That methodology forecloses a specific and important set of observations:

- **Runtime-resolved behaviour.** Where the payload resolves an API through an unrelocated placeholder slot, static analysis sees a distinct call site but not a function name. Several findings below are consequently scoped to "a call with this argument shape" rather than to a named API, and one such identification was explicitly withdrawn during the audit for exactly this reason.
- **Command-and-control responses.** Nothing was downloaded, so no payload the sample would fetch was analysed. The RSA public key recovered here has no verification routine anywhere in the file, which places the tasking protocol in a stage that was never on this disk.
- **Environment-gated behaviour.** Anything conditioned on a live network, a specific Windows build, mounted removable media, reachable network shares, or the presence of particular security products was not observed.
- **In-memory state.** One artefact recovered here — the decrypted body — is demonstrably a *memory image* captured from a live infected process rather than a pure static decrypt, which is discussed as Finding 6c because it changes what several values in it mean.

This section states what the *methodology* forecloses. The Assessment Limitations section states what *this particular analysis* could not resolve. They are not interchangeable, and the second is the longer list.

Malware Family Identification

Confidence ratings follow [ICD-203](#) probability vocabulary, not casual usage.

FAMILY	BASIS	CONFIDENCE
Sality (aka W32.Sality, Win32.Sector, SaliCode)	Code and configuration recovered first-hand from the decompressed payload: the 23-entry security-domain blocklist including sality-remov and upload_virus; the 2048-bit v4 RSA modulus matching a published family key byte-for-byte at its natural boundary; the kukustrustnet*.info URL pack; the amsint32 driver and device names; the Simple Poly Engine v1.1a (c) Sector banner observed as executing code , not merely as a string.	High
Sality — vendor consensus	63 of 71 VirusTotal engines, popular threat label virus.sality/salicode; concurring independent engines include BitDefender Win32.Sality.3, ESET Win32/Sality.NBA, Avira W32/Sality.AT, DrWeb Win32.Sector.30. [intel] — corroborating, not load-bearing.	High (as a statement about vendor consensus)
Sainbox / RAT	ANY.RUN applied the tags sainbox and rat to a public run of this exact sample. Rejected as an attribution. The string sainbox is absent from all 741,648 bytes of the decompressed payload under ASCII, UTF-16LE, case-insensitive matching and the 5-character prefix sainb, against a firing positive control (sality at 0x2298c). The run itself shows no RAT-consistent behaviour — no C2, no child process, no injection, no persistence artefact. Most likely a sandbox tag-propagation artefact.	Rejected — no static basis at any of the five layers
Cobalt Group / renamed-jusched heuristic	A crowdsourced Sigma rule "Renamed Jusched.EXE Execution" fires on the VT report and is associated with that actor. Rejected. The rule is a filename heuristic; this file is not a renamed jusched.exe, it is jusched.exe, infected. Sality is unrelated to that actor.	Rejected — false lead
Attribution by algorithm constant	The payload contains the MD5 initialization constants and the CRC32 polynomial 0xEDB88320. Deliberately not used as a basis. A matching cryptographic constant is an arithmetic identity — the constant is the same because the algorithm is the same.	Not used

On the direction of evidence. The family call was [intel]-grounded for most of

this engagement and became [static] on 2026-09-10, once the payload was decompressed and its configuration read. The two entries sality-remov and upload_virus are the strongest single anchor, because they are the malware defending itself *by name* against its own removal tool. Note that the plaintext string SALITY also present at 0x2298c is **weaker** evidence than it appears: it sits inside the sample's own AV-product kill list, i.e. it is something the malware searches *for*, and it would appear in a program that hunts Sality just as readily as in Sality itself.

Chain Overview

Five layers, each recovered from the one before it. Every edge below is derived first-hand from bytes on this disk; none is supplied out-of-band.

L0	sample.bin — infected Oracle jusched.exe <i>Entry-point stub at raw 0x40b96 computes VA 0x4b7630 via MMX and reaches it with push/ret — no static cross-reference exists.</i>	820,312 BYTES
L1	Viral blob, raw 0xb4c00–0xc5bff, appended inside .reloc <i>First 4,048 B are a plaintext polymorphic decryptor plus junk; the remaining 65,536 B are ciphertext under a 32-bit SUB stream.</i>	69,632 BYTES
L2	plain.bin — decrypted position-independent loader body <i>Resolves APIs by literal repe cmpsb over the export table; carries an embedded PE at offset 0x34ec.</i>	61,160 BYTES
L3	embedded_upx.bin — UPX 2.02 / NRV2B, level 9 <i>Decompressed by a Python re-implementation of its own stub — the sample was never run.</i>	46,080 BYTES
L4	payload_unpacked.bin — the actual malware payload <i>Four programs inside one blob, three of them complete PEs.</i>	741,648 BYTES
	EXE stub @0x016b0 6,280 B declared / 179 B present	
	Infector DLL @0x01ab4 123,392 B declared / 1,285 B present — skeleton only	
	Kernel driver @0x1fcc4 4,672 B — the network component	
	Code island @0x20f04 90,516 B — belongs to no embedded PE at all	

The five-layer chain, with each artefact's own byte count. The kernel driver at @0x1fcc4 is the sample's only network component.

Two structural points are worth stating before the detail.

The largest body of malware code in this sample belongs to no program. The 90,516-byte region beginning where the kernel driver ends has no PE header, appears in no section table, and was in no program's accounting until it was measured directly. It is larger than the driver, the stub and the DLL's present content combined by an order of magnitude, and it holds the infection routine, the polymorphic engine and the payload's cryptographic primitives.

One of the three embedded PEs is not a program. The infector DLL's section header declares 121,848 bytes of code; the payload contains 1,285 nonzero bytes in

that range and zero everywhere else, and its declared entry point lands in the zero field. It is a header-and-imports skeleton. This is recorded because an earlier framing of this engagement counted "four embedded programs" and that count was overstated by one — a section header states an intended memory layout, it does not promise the bytes exist.

Component Inventory

ROLE	FILE NAME	FILE TYPE	SHA-256 (SHORT)	NOTES
Dropper (L0)	sample.bin	PE32 GUI, i386	f073b23d... c70d59	The distributed infected file. Signed Oracle jusched.exe 2.8.401.10. 820,312 B.
Viral blob (L1)	— (in situ, raw 0xb4c00– 0xc5bfff)	raw	—	69,632 B appended inside .reloc. Not carved to a standalone file; addressed in place.
Loader (L2)	reimpl/ plain.bin	position- independent x86	07f7f1... 0414d6	61,160 B. Decrypted from L1. A memory image, not a pure static decrypt — see Finding 6c.
Packed payload (L3)	reimpl/ embedded_upx. bin	PE32 GUI, UPX 2.02	d371b3... 26bd05	46,080 B. Carved from L2 at offset 0x34ec.
Payload (L4)	reimpl/ payload_unpac ked.bin	raw image (no MZ at 0)	7ae591... cd9ae0	741,648 B. NRV2B-decompressed from L3. Contains the three PEs and the code island below.
Embedded stub	reimpl/ carved_embedd ed_stub_exe.b in	PE32 EXE	a9e234... (1,024 B carve)	6,280 B declared; 179 executable bytes present, 35% dense. Imports Sleep, SetErrorMode, ExitProcess only.
Embedded infector DLL	reimpl/ carved_embedd ed_infector_d ll.bin	PE32 DLL	d1ac0e... c1adb59	123,392 B. Skeleton — 1,285 of 121,856 declared code bytes present (1.1%). ImageBase 0x10000000.
Kernel driver	reimpl/ carved_embedd ed_amsint32_d river.bin	PE32 SYS, Native subsystem	d34b4e... 76a3035a	4,672 B. Imports ntoskrnl.exe exclusively. The network component.
Code island	island.bin (carved from L4 0x20f04– 0x37097)	headerless x86	—	90,516 B, 185 functions, recovered load base 0x69b000. No PE header.

ROLE	FILE NAME	FILE TYPE	SHA-256 (SHORT)	NOTES
Config: URL pack	reimpl/ netsurface/ url_pack.bin	raw	7fccb8... 2b4fdb3	146 B from L4 0x1240. Count byte + four NUL-terminated URLs.
Config: v4 public key	reimpl/ netsurface/ v4_pubkey.bin	raw	bd912e... bd7bd695	264 B from L4 0x13d0. e=0x10001 prefix + 256-byte modulus. No consumer in any layer.
Config: blocklist	reimpl/ netsurface/ driver_blockl ist.txt	text	—	254 B decoded from L4 0x2088c- 0x209a8 under XOR 0xf9. 23 entries.

Flow. L0's entry-point stub transfers to L1 in place; L1 decrypts itself to L2; L2 copies itself into a named pagefile-backed section and starts a thread, then transfers to an address outside its own extent; L3 is carved out of L2 and decompressed to L4; the three embedded PEs and the code island are carved out of L4 by header scan and by subtracting the PE extents from L4's content islands respectively.

Note that the hashes for L2, L3, L4 and the carved components are hashes of **artefacts this engagement produced**. They are reproduction checks, not third-party-searchable indicators, and the Indicators of Compromise section labels them accordingly.

Sample Identification

PROPERTY	VALUE
SHA-256	f073b23d47e6147dfef23e17170452edc57f09044fe64a872025c40ce6c70d59
SHA-1	583ff7c2add8b2b069d2fb0cdc14699aa4d9db58
MD5	d2fce530662f19987e445febd00a3223
Size	820,312 bytes (0xC8458)
Format	PE32 executable (GUI) Intel 80386, for MS Windows
imphash	d848eb007e31c25948fa20824db54df7 (first-hand) / a25cc0f2b07537e178efa8b6853f0de8 (VT-reported)
ssdeep	12288:tLCd1CnX25awdAYgvSV2xNj/3jb9mg/jxIWRV/ TucqlcjqsFIxG3p:MNnXgawd8SV27zbsAjxxucqlcLsFzp
TLSH	T1D105BF2139E2C076D2733170485DC35866BAF4314A3A4EE7B7CC67AD2F246C19 B25BB6
PE TimeDateStamp	0x65819871 = 2023-12-19 13:19:45 UTC
Authenticode signing time	20231219131947Z = 2023-12-19 13:19:47 UTC — 2 seconds after the build timestamp
Stored PE checksum	0x00000000 (computed: 0x000d66ed)
DllCharacteristics	0x8140 — DYNAMIC_BASE, NX_COMPAT, TERMINAL_SERVER_AWARE set; NO_SEH clear

Identity was verified by hash before any finding was recorded, by sha256sum over sample.bin, and re-verified at the start of this session. Every reproduction script in r2_evidence/ and pass2_evidence/ pins that hash and refuses to run on a mismatch. This discipline exists because matching on a filename fragment has previously cost a full evaluation scored against the wrong binary.

Host identity, confirmed three independent ways. The PE version resources give CompanyName Oracle Corporation, ProductName Java Platform SE Auto Updater, OriginalFilename jusched.exe, FileVersion 2.8.401.10. The Authenticode certificate names subject Oracle America, Inc., Redwood City, California, issued

by DigiCert Trusted G4 Code Signing RSA4096 SHA384 2021 CA1, valid 2023-03-09 to 2025-03-11, chaining to DigiCert Trusted Root G4. A public sandbox run of the same SHA-256 reports the identical MD5, file version and description. These three agree, so the host's identity is not in doubt.

On the imphash discrepancy. The locally computed imphash differs from the one VirusTotal reports. Both are computed over the same 245 imports across 12 DLLs; the difference is an implementation detail of ordinal and case normalization between pefile versions, not a discrepancy in the file. Neither value should be used to pivot to related samples here in any case: **the import table belongs entirely to the benign Oracle host.** The viral body has no import table of its own — it resolves what it needs at runtime — so an imphash match to this file identifies other copies of Oracle's updater, infected or not, and says nothing about the malware.

The fuzzy hashes have the same defect and it is worth stating plainly. ssdeep and TLSH over this file are dominated by the 750,680 bytes of Oracle code, not by the 69,632 bytes of malware. A fuzzy match to this sample is a match on Oracle's updater. Because Sality is polymorphic per infection — a differently infected copy of the same host would carry a differently encrypted body and a differently generated stub — fuzzy hashing is close to useless for finding related infections of *other* hosts, which is the case that matters. Detection must key on structure, and the Detection Rules section does.

Runtime Requirements

Native. x86 (i386), 32-bit, little-endian, Windows GUI subsystem (2). Dynamically linked against 12 DLLs. The host image asserts `IMAGE_DLLCHARACTERISTICS_DYNAMIC_BASE` (ASLR) while its base-relocation data directory is zeroed — a self-contradiction discussed as Finding 1d, and a detection opportunity in its own right, since no linker produces that combination.

The malware's own runtime requirements are narrower than the host's, and each is a scoping fact:

- **The infector accepts 32-bit PE targets only.** Its PE parser at VA 0x6a2603 tests `cmp ecx, 0x10b` (VA 0x6a2676) and rejects anything else. The infectable target set is therefore bounded by construction, not by a behavioural guess.
- **The infector requires the target to retain its relocation table.** Its entry gate tests `cmp word [edx+0x12], 1` against `IMAGE_FILE_RELOCS_STRIPPED` in the target's Characteristics and skips the file outright if set. A host with relocations stripped cannot be infected at all. This is a hard precondition and it is unusual enough to be a hunting signal.
- **The kernel driver requires one of six specific Windows builds.** Its process-termination thread reads `NtBuildNumber` and switches on 0x565, 0x893, 0xa28, 0xece, 0x14bc, 0x1770. An unrecognized build selects index 0 and the thread exits. The version list therefore bounds which systems the kernel-mode kill capability works on — an estate outside those builds is exposed to the filtering but not to the termination.
- **The driver expects the OS TCP/IP stack to be present and attachable.** It does not bundle a network stack; it borrows one. This is precisely why it has no network imports, and precisely why four user-mode-scoped searches missed it.
- **The payload expects `WININET.DLL` to be loadable at runtime.** It is not in any static import table; the name and its four functions appear in a runtime-resolved name block.

Linkage has an analysis consequence here, not merely a scoping one. The L2 loader body is position-independent code with no import table at all, so no import-scoped tool can characterize it — this is why an anti-debug scanner correctly refused to process it ("not a parseable PE") rather than returning a misleading zero. Likewise the L4 code island has no PE header, therefore no `.pdata/.xdata`, therefore no exception-dispatch root set for `seh_roots.py` to contribute. The indirect-call floor bounding any "no path exists" claim about that

region is consequently **wider than usual, not narrower**, and Finding 8c states the number.

Key Findings

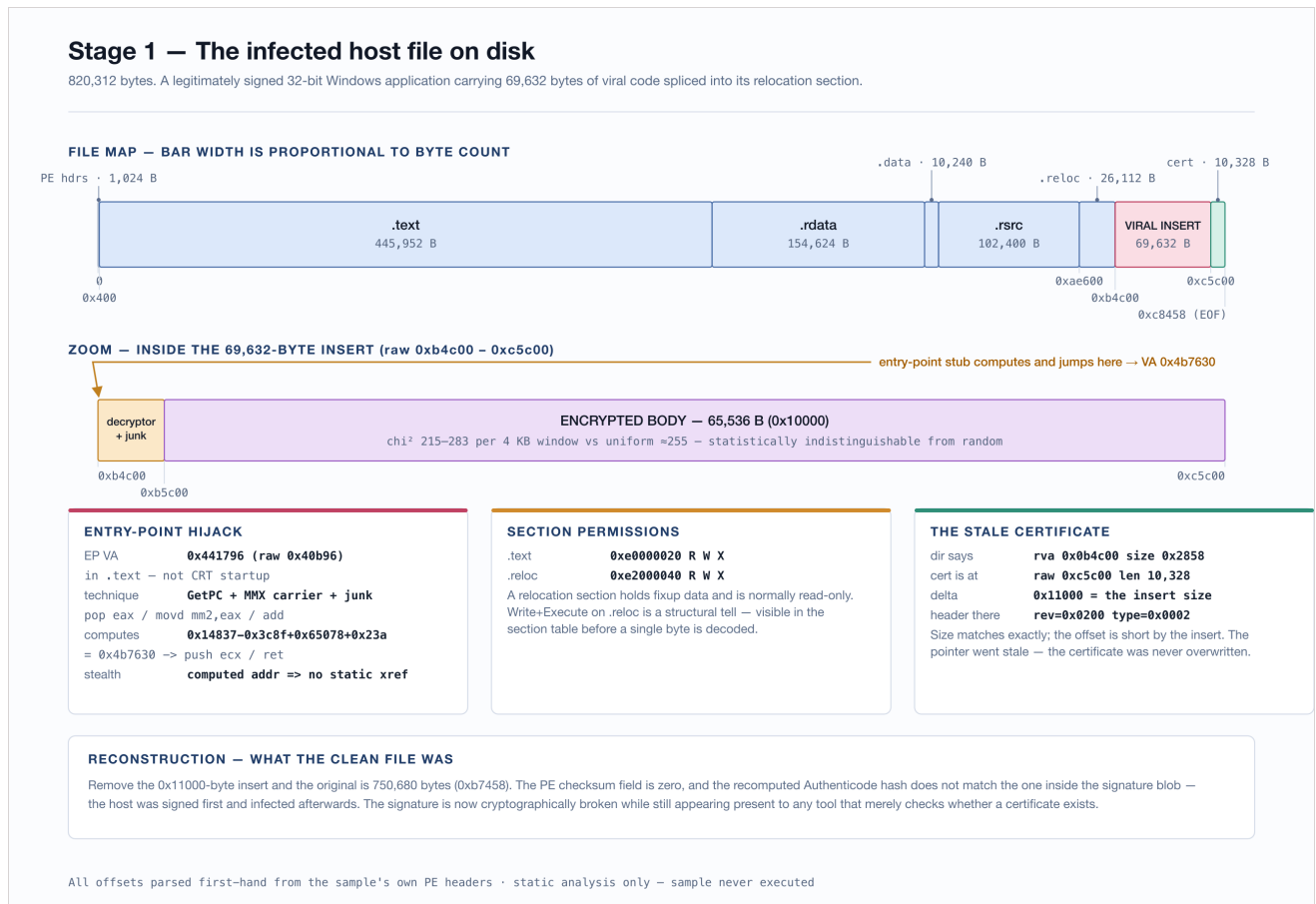
Severity-ranked. Each links to its numbered finding in the Technical Analysis.

1. **A kernel-mode driver is the sample's network component, and four independent analysis methods missed it because all four searched user-mode APIs** — it attaches to the TCP/IP stack, blocks SMTP and 23 security-vendor domains, and kills processes from kernel mode. (Findings **7a–7f**)
2. **The infection destroyed 330 bytes of the signed host's original code, making the file unrecoverable by disinfection** — five original call sites still point into the overwritten range. Affected files must be replaced, not cleaned. (Finding **1f**)
3. **The Authenticode signature is cryptographically void but physically present**, so any tool that checks only for the existence of a certificate reports this file as signed by Oracle. (Findings **1c, 1e**)
4. **The entry point reaches the viral body through arithmetic laundered via an MMX register**, storing its destination nowhere, which defeats every static cross-reference method. (Finding **2a**)
5. **The largest malware code region in the file belongs to no program** — 90,516 headerless bytes holding the infection routine, the polymorphic engine and the crypto primitives, in no section table and in no program's accounting. (Findings **8a–8c**)
6. **The polymorphic engine named in a banner string is observed as executing code** — a code generator at 30 call sites emitting instruction bytes on probability thresholds, driven by an LCG at 123 call sites. (Finding **9d**)
7. **The infector defeats last-modified sweeps** by restoring host file timestamps and clearing/restoring the read-only attribute around its writes. (Finding **9e**)
8. **A 2048-bit RSA public key was recovered but has no consumer anywhere in the file** — no crypto imports and, decisively, no multi-precision arithmetic in 24,530 disassembled instructions. Signature verification happens in a stage that was never on this disk. (Findings **10a–10c**)
9. **The blocklist names a Sality removal tool and a sample-upload page**, establishing the family from the sample's own bytes rather than from a vendor label. (Finding **7d**)
10. **A claim that this report's own working notes had held for a day was withdrawn on re-examination**: the routine believed to perform the infecting write does not write to the victim at all. (Finding **9g**)

Capability Flow Diagram

Five figures accompany this report, generated from checked-in build scripts under `opus/figures/` (`build_fig1.py ... build_fig5.py`, `sharing_style.svg` and `_lib.py`; each emits an `.svg` rendered to `.png` with `rsvg-convert`). Every offset and value in them is parsed first-hand from the sample.

Figure 1 – The infected host file on disk



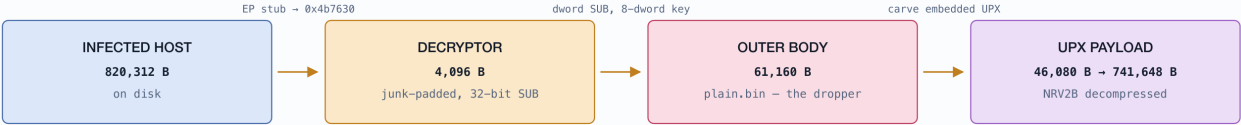
820,312 bytes. A legitimately signed 32-bit Windows application carrying 69,632 bytes of viral code spliced into its relocation section.

Figure 2 — The five-layer decrypt chain

Stage 2 — Decryption and unpacking

How 69,632 bytes of ciphertext in .reloc become a 741,648-byte payload. Each layer is produced by the one before it.

THE UNPACKING CHAIN — FOUR LAYERS, EACH PRODUCING THE NEXT



THE CIPHER — ONE MEMORY WRITE IN 3.6 KB OF JUNK

site **0x4b83cf - 0x4b83f6**
or `ecx,[ebx] / sub ecx,esi / mov [ebx],ecx`
width **32-bit dword, applied in place**
inner loop **0xc1 iterations @ 0x4b7f15**
outer loop **@ 0x4b7ecd, ret @ 0x4b845c**
The `'or ecx,[ebx]'` acts as a mov: a self-cancelling xor `ecx,0xe92` pair zeroes `ecx` first — confirmed by the pattern occurring exactly twice, adjacently.

WHY THE FIRST PASS MISSED IT

An earlier sweep searched for short-period XOR / ADD / SUB / ROL / ROR and for a column index-of-coincidence period of 1–256, and found nothing.
That negative was real but narrowly scoped: every method searched at BYTE width, and the cipher operates at DWORD width.
A negative claim is only as broad as the operand width it was searched under.

THE KEYSTREAM HIDES AS INSTRUCTIONS

table **VA 0x4b7644 - 0x4b7660 (file 0xb4c44)**
size **8 dwords = 32 bytes**
read via **add edx,[edi] @ 0x4b8276, cycled descending**
Those same 32 bytes are simultaneously live x86: `bswap edi / movzx esi,si / imul eax,ebx`. That is why no key-shaped constant was ever findable by scanning for one — the key IS code.

VERIFICATION — RE-IMPLEMENTED, NEVER RUN

output **plain.bin - 61,160 bytes**
self-ref check **0 of 7,527 reads at/above write cursor**
mismatches **0 model-vs-disk**
The decryptor was re-implemented in Python from the sample's own disassembly and never executed. A mismatch would surface as a decode failure rather than as silent execution.

Span note: `chi2` bounded the region at 65,536 B but the decryptor addresses 61,160 B. The 4,376-byte difference decomposes exactly — 278 B of head the loop never starts on, plus 4,098 B of tail. The decrypt is complete, not truncated: uniformity bounds a region's appearance, only the decryptor's own addressing bounds its output.

Cipher, keystream and loop bounds read from disassembly · decoder re-implemented in Python and never executed

*L0 host → L1 appended body → L2 dword-SUB plaintext → L3 embedded UPX → L4
NRV2B-unpacked payload, with the cipher and mechanism at each hop.*

Figure 3 — Inside the 741,648-byte payload

Stage 3 — Inside the unpacked payload

741,648 bytes holding three embedded programs, a configuration block, and a 90,516-byte region of code that belongs to no program at all.

PAYLOAD MAP — 741,648 BYTES, BAR WIDTH PROPORTIONAL



THE STRING BLOCK — WHERE THE FAMILY NAMES ITSELF (offsets are file offsets in the unpacked payload)

0x21260 amsint32 kernel driver device name	0x21490 [AutoRun] autorun.inf template	0x21520 Simple Poly Engine v1.1a the polymorphic engine, self-named
0x2168c ipfltdrv.sys IP filter driver	0x216e0 \\.\amsint32 device path	0x21740 autorun.inf removable-media spread
0x21858 IPFILTERDRIVER device-type constant anchor	0x2298c SALITY inside the ~150-entry AV kill list	0x22cc4 .adata an entry in a file-open hook list

THE CODE ISLAND BELONGED TO NO PROGRAM

range **0x20f04 — 0x37097 (90,516 B)**
density **85.8% nonzero — 77,693 of 90,515**
header **none — no MZ, no PE, no section table**

It begins where the last embedded PE ends and was in no program's accounting until it was measured. Its load base was recovered from the distribution of absolute pointer-like dwords — two framings agreeing on one VA window.

COVERAGE, MEASURED PROPERLY

attributed **85.4% of range**
of nonzero **99.5%**
overlap **zero**

Corroborated by two methods sharing no principle — recursive descent from call targets, and a prologue-signature sweep — agreeing to within 6 bytes. The shortfall is padding, not unanalysed code.

Reading discipline: a string names a capability; only code demonstrates one. '.adata' was twice glossed as "the section the infector writes into its hosts" — read with its neighbours it sits in a list bracketed by AV-product tokens and file-open API names, so it is a hook/exclusion entry. The gloss was reversed.

Embedded PEs located by header scan and parsed from their own headers · all string offsets verified first-hand

Three embedded PEs, the ~90 KB code island that belonged to no program's accounting, the config blocks and the string tables.

Figure 4 — The four programs, side by side

Stage 4 — The four programs and the reversal

The payload embeds an EXE stub, a DLL skeleton, a kernel driver and a headerless code island — and the driver overturned the engagement's central negative.

FOUR PROGRAMS IN ONE PAYLOAD — THREE CARVED PEs PLUS ONE HEADERLESS REGION

① EXE STUB 6,280 B user mode role loader / launcher machine 0x014c (i386) chars 0x010f EXE	② INFECTOR DLL 123,392 B user mode declared 121,856 B of code present 1,285 nonzero (1.1%) verdict SKELETON — no body	③ KERNEL DRIVER 4,672 B KERNEL MODE device IPFILTERDRIVER role traffic filter kills processes from ring 0	④ CODE ISLAND 90,516 B user mode header none at all density 85.8% nonzero role the infector body
--	---	---	--

user mode ↑ · kernel mode ↓

THE DRIVER IS THE NETWORK COMPONENT For most of the engagement the sample's network component was held to be ABSENT — on four methods with non-overlapping blind spots and firing controls. All four searched user-mode network APIs. The driver borrows the OS TCP/IP stack instead of implementing one, so it has NO network imports for any of them to find. It is identified by its device-type constant, not by an API name. A convergent negative is only as broad as the narrowest assumption its methods share.	WHAT THE DRIVER DOES attaches as a traffic filter blocks one mail port outright blocks vendor domains by substring (XOR 0xf9) terminates processes via per-OS-build offset table Two blocklist entries name removal tools and sample-upload pages — which is what establishes the family from the sample's own bytes rather than from a vendor label.
---	---

What still stands: nothing in any of the four programs can accept an inbound connection, so the peer-to-peer negative holds. The outbound surface remains four user-mode HTTP functions, and the download-only model is unchanged. The RSA public key in the payload has no consumer anywhere in the file — no crypto imports in any of the four programs — so signature verification happens in a stage that was never on this disk.

Each PE parsed from its own headers · content density measured before any program was called analysable

The user/kernel divider, and the content-density measurement that reclassified the infector DLL as a header-and-imports skeleton.

Figure 5 — The infection routine, fcn.006a2802

Stage 5 — The infection routine, stepped

Eight stages of fcn.006a2802 at instruction level — and the negative result that retired a standing inference.

THE INFECTION ROUTINE — fcn.006a2802, STEPPED INSTRUCTION BY INSTRUCTION

① PATH NORMALISE scan for a trailing separator 0x6a287d movsx eax, byte [ebp+eax-0x225] 0x6a2885 cmp eax, 0x5c 0x6a2888 je 0x6a289c	② ENUMERATE, RETRY query-then-retry on length mismatch 0x6a28b7 cmp dword [var_ch], 0xc0000004 0x6a28e7 <reallocate and re-issue>	③ PARSE PE HEADERS section count and table base 0x6a2907 mov edx, [ecx+0xc] 0x6a2915 mov cx, word [eax+0x1e] 0x6a291f lea eax, [edx+ecx+0x20]	④ OPEN, FAIL CLOSED two attempts, one result slot 0x6a29.. call [0x7c000000] (0,1) 0x6a29.. call [0x7c000000] (0,1) -> var_22ch ; both fail => return
⑤ MAP THE VIEW (handle, -1) -> pointer, null-checked 0x6a29ba mov eax, [0xc0060000] ; -1 0x6a29c7 call [0x78000000] -> var_120h 0x6a2a03 call fcn.006a269a reloc walk 0x6a2a37 call fcn.006a2603 PE validate	⑥ WALK POINTERS two passes: count, then build 0x6a2a86 sub edx, [ecx+0x1c] 0x6a2a8c cmp edx, [eax+0x38] 0x6a2b1e mov [edx+ecx+4], eax	⑦ STAMP AND WRITE 666 marker, then write ELSEWHERE 0x6a2b29 mov dword [eax], 0x29a ; 666 0x6a2b39 push 0x40000000 0x6a2b45 call [0x58010000] 0x6a2b7c call [0x5c010000] (h,buf,n+4,6w,0)	⑧ RELEASE free, close, then one path-taking call 0x6a2ba3 call [0x74000000] 0x6a2ba9 movsx eax, byte [var_118h] 0x6a2bba call [0x54010000] ; one arg

THE TRACE INVERTED ITS OWN PREMISE expected mmap -> patch -> SetFileTime found mmap -> READ -> write elsewhere There is NO store into the mapped view [var_120h] anywhere in this routine. It maps the victim, reads its section table and relocations, builds a pointer array, stamps 666 into that ARRAY, and writes the array to a DIFFERENT handle. [0x54010000] takes ONE argument, a path, and runs after the handle is closed. SetFileTime takes a handle plus three FILETIME pointers, so it cannot be that call.
--

WHAT WAS WITHDRAWN, AND WHY The \$19.6 write inference is WITHDRAWN — annotated in place, not deleted. It was built from the import set plus the call list: string-shaped reasoning, the same defect class as the .adata gloss. A string names a capability; only code demonstrates one. The imports were real and the callers were real, and the conclusion drawn from them was still wrong. Six callers remain unread: 0x6a337b · 0x6aa38b · 0x6aaa01 · 0x6aa4f4 · 0x6ac297 · 0x6ac93d. The actual patch site is in one of them, or nowhere on this disk.
--

Placeholder immediates ([0x78000000], [0x58010000], ...) are cited as distinct call sites, never as named APIs — the IAT mapping needs the dropper's loader

Eight stages, with the two that inverted the analysis's own premise accented, and the six unread callers named.

Capabilities

Characterized against the [Malware Behavior Catalog](#) as the primary axis, with [ATT&CK](#) cited alongside. MBC is primary here because this sample's most characteristic findings — the polymorphic engine, the MMX-laundered entry transfer, the junk-instruction prologue — are Anti-Static Analysis behaviours, an objective ATT&CK does not have.

Summary by MBC objective. The sample's dominant objectives are **Anti-Static Analysis** (polymorphic infection, computed control transfer, a self-decrypting body, a packed payload) and **Defense Evasion** (kernel-mode security-domain blocking, AV process termination, firewall and AV-notification disablement, timestamp restoration). **Impact** is expressed as parasitic file infection. **Command and Control** is download-only over HTTP. **Discovery** covers drive enumeration, process enumeration and the security-product survey. There is no credential-access or collection capability in any of the five layers — no keylogging, no clipboard access, no browser or credential-store handling was found, and the search that establishes that is scoped in Negative Findings.

MBC BEHAVIOR	PROCEDURE OBSERVED	ATT&CK	CONFIDENCE	NOTES
Anti-Static Analysis::Executable Code Obfuscation [B0032]	4,048-byte polymorphic prologue of junk instructions, MMX carriers, xchg ah, ah, push edx/pop edx, and 9 conditional branches on registers uninitialized at process entry, preceding the real decryptor	T1027	High	Disassembled in full: 1,122 instructions decode cleanly over 0xe2d bytes to a single ret. Finding 3a.
Anti-Static Analysis::Disassembler Evasion [B0006]	jmp 0x4417a3 at 0x44179f jumps over the two bytes 0f cd at 0x4417a1, desynchronizing linear sweep	T1027	High	Byte-level, verified in the raw file. Finding 2a.

MBC BEHAVIOR	PROCEDURE OBSERVED	ATT&CK	CONFIDENCE	NOTES
Anti-Static Analysis::Software Packing [F0001]	UPX 2.02, NRV2B method 8 level 9, wrapping the 741,648-byte payload	T1027.002	High	Version read from the sample's own l_info (0d 09 08 09). Finding 5a.
Defense Evasion::Obfuscated Files or Information [E1027]	Body encrypted with a hand-rolled 32-bit subtractive stream cipher, keystream generated per-dword by an imul/xor/bswap chain	T1027	High	Single store instruction in the whole region; located by filtering for memory writes. Finding 3b.
Defense Evasion::Obfuscated Files or Information::Encoding [C0026]	Payload string table and driver blocklist stored under single-byte XOR 0xf9, decrypted in place at DriverEntry with a sentinel guard so it runs once	T1027	High	Null model: 400 random 12-byte patterns against all 255 keys gave 0 hits. Finding 7d.
Impact::Virus [B0031] / Persistence::File Infection	PE32-only parser, relocation-table walker locating a host global by its c7 05 fixup, polymorphic stub generation, memory-mapped in-place editing	T1027 / T1547	High	Traced as code, not inferred from strings. Findings 9a–9e.
Anti-Behavioral Analysis::Debugger Evasion [B0001]	Not found in the malware layers , and the negative is scoped — see Negative Findings §N7. The 10 confirmed anti-debug hits in the file belong to the benign Oracle host's runtime.	T1622	—	Reported as a <i>negative</i> deliberately; the scan's 1,711 raw hits are adjudicated in Finding 12a.

MBC BEHAVIOR	PROCEDURE OBSERVED	ATT&CK	CONFIDENCE	NOTES
Defense Evasion::Disable or Modify Tools [F0004]	Kernel-mode process termination via PsLookupProcessByProcessId → ObOpenObjectByPointer → a per-NtBuildNumber offset table; plus an AV process/service kill list of ~150 entries	T1562.001	High	Driver code hand-transcribed; the offset table is version-dispatched over six builds. Finding 7e.
Defense Evasion::Disable or Modify System Firewall [F0004.002]	Registry paths for StandardProfile, AuthorizedApplications\List, EnableFirewall, DoNotAllowExceptions, and the %s*:Enabled:ipsec value template	T1562.004	Moderate	Strings recovered from the payload with their neighbours; the writing code was not individually traced, so this is scoped as configuration present, not as an observed write.
Defense Evasion::Modify Registry [E1112]	AntiVirusOverride, AntiVirusDisableNotify, FirewallDisableNotify, UpdatesDisableNotify, UacDisableNotify under SOFTWARE\Microsoft\Security Center; EnableLUA, DisableTaskMgr, DisableRegistryTools	T1112	Moderate	Same scoping as above: string-and-neighbour evidence, Reg* imports present in the payload's name table.

MBC BEHAVIOR	PROCEDURE OBSERVED	ATT&CK	CONFIDENCE	NOTES
Command and Control::Application Layer Protocol::Web Protocols [B0030]	Exactly four WinINet functions — InternetOpenA, InternetOpenUrlA, InternetReadFile, InternetCloseHandle — plus a ?%x=%d / &%x=%d query template and a Mozilla/4.0 (compatible; MSIE 7.0b; Windows NT 6.0) user agent	T1071.001	High	Download-only. No listener, bind, accept or socket API in any of the four programs. Finding 11a.
Command and Control::Ingress Tool Transfer [E1105]	GET /home.gif? <hex>=<dec>&<hex>=<dec> reconstructed from the four URLs and the two format strings	T1105	High	home.gif is a transport, not an image: zero GIF8/GIF87a/GIF89a magic in 741,648 bytes, no LZW decoder, no imaging API. Finding 11b.
Defense Evasion::Network Traffic Filtering	Kernel driver attaches to the TCP/IP stack, blocks TCP port 25 outright and 23 vendor domains by substring match	T1562.004	High	Filter decision function fully transcribed at 0x108f0. Findings 7b–7d.
Defense Evasion::Rootkit [F0009]	Driver registers \Device\IPFILTERDRIVER, impersonating the legitimate Windows IP filter, and \Device\amsint32 / \DosDevices\amsint32	T1014	High	Device-type constant 0x15 = FILE_DEVICE_NETWORK. The camouflage reading corrects an earlier "attacks the IPFilter stack" framing — Finding 7a.

MBC BEHAVIOR	PROCEDURE OBSERVED	ATT&CK	CONFIDENCE	NOTES
Defense Evasion::Indicator Removal::Timestomp [F0005]	GetFileTime/ SetFileTime import pair, with GetFileAttributesA/ SetFileAttributesA to clear and restore the read-only bit	T1070.006	Moderate	Import-pair evidence in the payload's own name table. The <i>call sites</i> were not traced — one candidate identification was explicitly withdrawn, Finding 9g.
Discovery::System Information Discovery [E1082]	GetLogicalDrives + GetDriveTypeA for drive enumeration; FindFirstFileA/ FindNextFileA/ FindClose + lstrcmpiA for file traversal	T1082 / T1083	High	Present in the payload's plaintext import- name table at 0xb4009-0xb467a.
Discovery::Process Discovery [E1057]	CreateToolhelp32Sna pshot, Process32First/Next, Module32First/Next	T1057	High	Plaintext names at pinned offsets; feeds the AV kill list.
Defense Evasion::Process Injection [E1055]	VirtualAllocEx + WriteProcessMemory + CreateRemoteThread triad, with OpenProcess and TerminateProcess	T1055	High	All four names plaintext in the payload import table. Capability grounded on names; the injection call sequence was not individually traced.

MBC BEHAVIOR	PROCEDURE OBSERVED	ATT&CK	CONFIDENCE	NOTES
Privilege Escalation::Token Manipulation	OpenProcessToken + LookupPrivilegeValueA + AdjustTokenPrivileges, with SeDebugPrivilege present as a string at 0x22d60	T1134	High	The privilege name and the three APIs that consume it are all present.
Defense Evasion::Safe Mode Boot	System\CurrentControlSet\Control\SafeBoot and the string SafeBoot at 0x21ead	T1562.009	Low	String-and-path evidence only. Direction (deleting the key to prevent Safe Mode recovery vs. registering for it) is not established from these bytes. Deliberately rated Low.
Defense Evasion::Bypass File System Protection	SfcIsFileProtected at 0x21700 — checking whether a target is Windows File Protection-guarded before infecting it	T1211	Moderate	The name's presence in the infection-target string cluster is the evidence; the check's call site was not traced.
Lateral Movement::Replication Through Removable Media [E1091]	autorun.inf at 0x21740, [AutoRun], shell\open\command	T1091	Low	Deliberately Low. These are strings with no demonstrated code reference. See Negative Findings §N8 — this is recorded as <i>unresolved</i> , not confirmed and not withdrawn.

MBC BEHAVIOR	PROCEDURE OBSERVED	ATT&CK	CONFIDENCE	NOTES
Lateral Movement::Taint Shared Content	WNetOpenEnumA / WNetEnumResourceA / WNetCloseEnum	T1080	Low	Same scoping as above. Names in the runtime- resolved block; no traced call site.

Every row above is disassembly-grounded or explicitly marked as string-scoped.

Tool output — capa, r2inspect, Ghidra pseudocode, VirusTotal labels — generated hypotheses for this table and appears in none of its rows. Where a capability rests on an API name rather than a traced call site, the Confidence column says Moderate or Low and the Notes say why. That distinction is load-bearing in this engagement: a string named a capability that the code turned out not to have, twice, and both reversals are recorded below as findings.

Technical Analysis

Findings are numbered and organized by the Roles from the Component Inventory. Sub-numbering marks parts of a single finding.

1. The infected host (L0)

1a. A genuine, correctly signed Oracle binary is the carrier

The host is Oracle's Java Update Scheduler. Version resources parsed with pefile (not hand-indexed) give CompanyName Oracle Corporation, FileDescription Java Update Scheduler, ProductName Java Platform SE Auto Updater, OriginalFilename jusched.exe, FileVersion 2.8.401.10, LegalCopyright Copyright (C) 2024. The Authenticode certificate names Oracle America, Inc. of Redwood City, California, issued by DigiCert Trusted G4 Code Signing RSA4096 SHA384 2021 CA1, valid 2023-03-09 to 2025-03-11, chaining to DigiCert Trusted Root G4.

The internal consistency is worth noting because it is what makes the file convincing: the PE TimeDateStamp is 2023-12-19 13:19:45 UTC and the ASN.1 signing time — re-derived independently from the PKCS#7 structure — is 20231219131947Z, **two seconds later**. That is the signature of a real build-and-sign pipeline, not a forgery.

The import table reflects the host and nothing else: **245 functions across 12 DLLs**, with bound-import and delay-import directories both empty. Among them are 11 WinINet and 4 CRYPT32 imports. **These belong to the benign Oracle updater** — the updater legitimately fetches update manifests over HTTP and legitimately hashes them — and this report does not cite them as malware capability anywhere. The distinction was verified by reading the strings beside the CRYPT32 references, which include Oracle's own source path . . .

\src\windows\wrappers\common\UpdateUtils.hpp and the neighbouring CryptCreateHash / CryptHashData. The viral body has **no import table entry of its own**.

(Earlier working notes recorded 13 DLLs; the correct count is 12. Logged as Correction 1.)

1b. .reloc is not a relocation section

This is the strongest first-hand structural indicator in the file, and it needs no reference material and no decryption to see:

```
.reloc VA 0x004b1000 vsz 0x018000 raw 0x0ae600 rsz 0x017600
      Characteristics 0xe2000040 = INIT_DATA | EXEC | READ | WRITE | DISCARDABLE

BASERELOC data directory (entry 5): rva 0x0, size 0x0
```

Three facts in tension. The base-relocation data directory is **empty**, so the loader is told this image has zero relocations. Yet a 98,304-byte section *named* .reloc is present. And it is marked **executable and writable**. A genuine MSVC .reloc carries 0x42000040 — INIT_DATA, DISCARDABLE, READ — read-only, never executable, and is pointed at by that directory.

.text is likewise anomalous at 0xe0000020: EXEC | READ | **WRITE**. MSVC does not emit a writable .text.

The data directories were enumerated by name, all sixteen, not hand-indexed.

This matters: an earlier pass in this engagement hand-indexed an offset, read the DEBUG directory (index 6, at 0x85490) as SECURITY, and invented a tampering finding on the strength of it. That is Correction 2, and the rule it produced — enumerate every directory by name — is why the certificate analysis below is trustworthy.

1c. The certificate pointer is stale by exactly the size of the insert

```
SECURITY data directory (index 4) -> offset 0xb4c00, size 0x2858 (10,328)
real certificate found at         -> offset 0xc5c00, size 0x2858
delta                            ->          0x11000 = 69,632 (size delta: 0)
```

At the address the directory names, 0xb4c00, the bytes are 53 1e e5 f4 4c fa 87 c0 ..., which parse as dwLength=0xf4e51e53, wRevision=0xfa4c, wCertType=0xc087 — meaningless. At 0xc5c00 sits a well-formed WIN_CERT: dwLength=0x2858, wRevision=0x0200 (WIN_CERT_REVISION_2_0), wCertType=0x0002 (PKCS#7 SignedData), followed by the DER 30 82 28 45 06 09 2a 86 ... and OID 1.2.840.113549.1.7.2. And 0xc5c00 + 0x2858 = 0xc8458, **exactly the file size**.

The size field matches perfectly while the offset is short by precisely the insert size. The certificate was never overwritten or touched — the infector appended 69,632 bytes ahead of it, which pushed it forward, and **did not update the pointer**. The stale target 0xb4c00 is 48 bytes before the decryptor at 0xb4c30: the viral code was laid down starting exactly where the certificate used to begin.

1d. The signature is void on four independent counts

1. **The digest covers tampered bytes.** The Authenticode signed range runs from offset 0 to the certificate start, which includes all of .reloc from 0xae600 to 0xc5c00 — the entire viral insert — and includes the overwritten entry-point stub in .text.
2. **The directory does not locate the certificate**, so a verifier following the spec finds no signature at all. VirusTotal reports "File is not signed" for exactly this reason, which is an independent implementation reaching the same conclusion as the local signify check.
3. **Header fields inside SizeOfHeaders were modified** and are covered by the digest: AddressOfEntryPoint (offset 0x150), .text Characteristics (0x244), .reloc Characteristics (0x2e4), .reloc SizeOfRawData (0x2d0) and VirtualSize (0x2c8), SizeOfImage (0x178), and the zeroed BASERELOC directory (0x1c8/0x1cc).
4. **DllCharacteristics = 0x8140 asserts DYNAMIC_BASE while BASERELOC is empty** — a combination no linker produces, since ASLR requires relocations.

Two further modifications are *excluded from the digest by specification* and so are not counted above, but are recorded because they are detection-relevant: the SECURITY directory VA itself (0x1c0) and the CheckSum field (0x180, stored 0, computed 0x000d66ed).

The practical consequence is the dangerous part. The file carries a physically present, structurally valid Oracle certificate that a verifier cannot find. A tool that asks "is there a certificate?" answers yes. A tool that asks "does the signature verify?" gets no signature to check and may report "unsigned" rather than "tampered". Neither answer is "this file was signed by Oracle and then modified", which is the truth.

1e. The insert's size and boundaries close arithmetically

Two unrelated methods agree. **Chi-square on byte statistics** (Finding 3a) places a uniform-random region at raw 0xb5c00–0xc5c00, exactly 65,536 bytes, preceded by 4,048 structured bytes from 0xb4c30. **PE header arithmetic** gives:

```
.reloc original raw size  0xb4c30 - 0xae600 = 0x06600  (26,112)
.reloc current  raw size                0x17600  (95,744)
growth                                0x11000  (69,632)
0x11000 = 0x1000 decryptor + junk + 0x10000 encrypted body
```


And the relocation content itself closes exactly: **143 well-formed relocation blocks / 12,410 entries**, ending at raw 0xb4b6c, content length 0x656c, aligned to FileAlignment 0x200 giving 0x6600 — and 0x17600 - 0x11000 = 0x6600 exact. So the malware **appended to a genuine, intact .reloc** rather than replacing it, and the original file size was 0xb4c00 + 10,328 = 0xb7458 = 750,680 bytes.

Entropy did not resolve this boundary and was not used to. Across the region entropy rises smoothly 5.1 → 6.4 → 7.2 with no cliff. Chi-square per 4 KB window discriminated cleanly: 0xae600-0xb5600 scores 14,800-18,800 (structured), 0xb5600 is the transition at 1,036, and 0xb5c00-0xc5c00 scores **215-283** against a uniform-random expectation of ~255 with sd ~23. This is a general lesson the engagement re-learned: entropy over short windows does not discriminate, and 3,794 windows once passed H > 7.2 on a different sample with counting tables as the top scorers.

1f. Reconstruction is impossible, not merely unattempted — and this governs remediation

The obvious remediation — remove the 69,632-byte insert and restore the original — **cannot work**, and the reason is a finding in its own right.

The entry-point stub occupies raw 0x40b96-0x40cdf, 330 bytes, **inside .text**. Byte 0x441795 is int3, MSVC's inter-function padding; the stub begins the very next byte. At 0x4418e0 ordinary MSVC SEH prologue resumes, and 0x441915 is a genuine function entry with three surviving direct callers (0x432de9, 0x433d48, 0x43409a).

Between those two landmarks, Oracle's original code is **gone**. Five original call sites elsewhere in .text still point into the overwritten range:

```
0x416a0a -> 0x441810      0x429c37 -> 0x4418a7      0x440856 -> 0x4417a0
0x416a1a -> 0x4417a0      0x42a0e5 -> 0x441885
```

Two of those target 0x4417a0 with four pushed arguments and consume a 64-bit result in edx:eax; one pushes 0x3b9aca00 = 1,000,000,000 (nanoseconds per second) immediately before — the profile of an MSVC _alldiv/_aulldiv helper used by timing code.

This was re-derived by **two independent methods that agree exactly**: a raw-file e8 rel32 scan computing VAs from the section header, and r2's axt cross-reference extraction. A first axt attempt returned nothing; the cause was analysis depth (aa rather than aaa) — a checker defect, not evidence — and with aaa r2 reports exactly the three callers of 0x441915 the raw scan reports. (*Logged as Correction 3, and it is*

the third instance this engagement of a cross-check being wrong where the tool was right.)

Therefore removing the blob can never reproduce the pristine Oracle binary.

Reconstruction would require a donor copy of Oracle `jusched.exe` 2.8.401.10 as a source for the destroyed 330 bytes. Remediation is replacement.

2. The entry-point transfer (L0 → L1)

*2a. The address is computed, never stored
— and this is why static methods failed*

The PE entry point is RVA `0x41796` = VA `0x441796` = raw `0x41196`, inside `.text`. It is not an MSVC CRT startup sequence. It is a hand-written stub, ~330 bytes, running to a `ret` at `0x4418df`.

The stub obtains its own address with the classic `GetPC` idiom — `0x4417d1 call 0x441820`, whose callee's first instruction is `0x441820 pop eax`, leaving `eax = 0x4417d6`. It then launders that value through an MMX register and computes its destination:

```
0x441820 pop  eax           ; eax = 0x4417d6           (GetPC)
0x441849 movd  mm2,  eax     ; launder through MMX
0x441858 movd  ecx,  mm2     ; ecx = 0x4417d6
0x441867 lea   esi,  [0x14837]
0x44186f sub   esi,  0x3c8f   ; esi = 0x10ba8
0x44187e push  esi
0x441881 pop   ebp          ; ebp = 0x10ba8
0x441884 add   ebp,  0x65078  ; ebp = 0x75c20
0x44188a add   ecx,  ebp      ; ecx = 0x4417d6 + 0x75c20
0x44188c add   ecx,  0x23a    ; ecx = 0x4b7630
0x4418ab push  ecx
0x4418df ret               ; ---> 0x4b7630
```

VA `0x4b7630` = raw `0xb4c30` = the blob start `0xb4c00` plus `0x30`.

Surrounding this arithmetic is a substantial anti-analysis layer: a linear-sweep desynchronizer at `0x44179f` (`jmp 0x4417a3` jumping over the two bytes `0f cd` at `0x4417a1`), no-op filler (`0x441863 xchg ah, ah`; `0x44180a push edx` / `0x44180b pop edx`), junk arithmetic (`imul esi, esi, 0x11714ee5`, chained `bswap/not/neg`), and **nine conditional branches testing registers that are uninitialized at process entry** — `test dl, 0xc4`, `test eax, 0x320e36fa`, `cmp ecx, 0xf1da`, `cmp ebx, 0x47a2`. Those nine branches have no determinate outcome at analysis time, which is the point.

The value `0x004b7630` is never stored anywhere in the file. It exists only as the sum

of a runtime-recovered program counter and three immediates, delivered by `push/ret` rather than by a call or jump instruction with an operand.

2b. Three retracted claims, and what they teach

This transfer was derived wrongly twice before it was derived correctly, and both errors are instructive enough to record as findings.

Retraction 1. A first derivation followed `mov ebp, eax` at 0x441850 into `add ebp, 0x65078` at 0x441884 and produced $0x4417d6 + 0x65078 = 0x4a684e$, which lands in `.rsrc`. **Wrong** — `ebp` is clobbered between those two instructions by the `push esi / pop ebp` pair at 0x44187e/0x441881.

Retraction 2. A second pass concluded that the arithmetic *does not reach the blob at all*, on the grounds that `ebp` ends up holding 0x75c20, a small value that is not a virtual address. This is **withdrawn as incorrect, not merely narrowed**: `ebp` is a *component* of the sum, not the carrier. The carrier is `mm2`. The correction came from an adversarial second pass by an independent agent and was re-derived at the bytes.

Retraction 3 — the general one. A negative had been recorded as "no static code reference reaches the blob." That negative was true and **scoped to static 4-byte immediates**, and it is not a statement about reachability at all. Where a stub is position-independent by construction, the absence of an immediate is the *expected* observation, not evidence of inertness.

A negative scoped to static immediates is not a negative about reachability.

Three consequent claims from the first pass are retired: that "the blob is inert with respect to this executable's own code" (it is the entry point, one `ret` away); that "there is no decryptor in this binary" (there is one, at `blob+0x30`, which imports nothing and contains no cipher constants — which is precisely why import- and constant-scoped searches were blind to it); and "near-uniform blob" (narrowed — the leading 4 KB is plaintext code at `H = 7.253`).

3. The viral body decryptor (L1 → L2)

3a. Locating the cipher by filtering for memory writes

The region at raw 0xb4c30 (VA 0x4b7630) is 4,048 bytes and decodes as **1,122**

instructions over 0xe2d bytes, ending in a ret at 0x4b845c — one self-contained routine. It is simultaneously the polymorphic junk prologue the entry stub jumps into *and* the decryptor.

Finding the actual cipher in 1,122 mostly-junk instructions was done by filtering for instructions with a memory destination. **There is exactly one in the entire region.**

```
0x4b83cf  0b 0b    or  ecx, dword ptr [ebx]    ; load ciphertext
0x4b83e8  2b ce    sub ecx, esi                ; subtract keystream
0x4b83f6  89 0b    mov dword ptr [ebx], ecx    ; store plaintext IN PLACE
```

3b. Why *or* is really *mov*, and a lesson about reading blocks

The `or ecx, [ebx]` looks like it destroys information — you cannot invert an OR. It does not, because of the two instructions immediately preceding it:

```
0x4b83c3  81 f1 92 0e 00 00  xor ecx, 0xe92
0x4b83c9  81 f1 92 0e 00 00  xor ecx, 0xe92    <- identical, adjacent, self-cancelling
0x4b83cf  0b 0b              or  ecx, dword ptr [ebx]
```

Two identical XORs cancel, so `ecx == 0` at the load, and `or ecx, [ebx]` **is** `mov ecx, [ebx]`. Verified at raw bytes, file offset 0xb59c3.

The enumeration is complete rather than sampled: the byte pattern `81 f1 92 0e 00 00` occurs **exactly twice** in the whole region, and the two occurrences are adjacent. A backward slice on `ecx` confirms 0x4b83c9 is the last write before the load.

Reading three instructions is not reading a basic block. *An earlier note recorded "or ecx, [ebx] is not invertible" and was wrong for exactly this reason. (Correction 4.)*

3c. The cipher, and the shape of the loop

The cipher is a **32-bit (dword) subtraction of a keystream value, applied in place.** The keystream lands in `esi`, produced by a long chain of `imul` / `xor` / `add` / `bswap` over `ebp`, `edi` and prior state.

```
0x4b7ecd                                outer loop head
0x4b7f15                                inner loop head
0x4b8429  cmp eax, 0xc1 / jae 0x4b8435 / jmp 0x4b7f15
```

```
0x4b8450    cmp ebp, 0xeeee2 / jl 0x4b7ecd
0x4b845c    ret
```

Tracing `ebx` and `esi` at the store across two complete outer passes shows pass 0 alternating between `0x4b8716` and `0x4b871a` (64 stores) and pass 1 between `0x4b871e` and `0x4b8722` (64 stores) — a step of **+8 bytes**, matching the `add ebp, 8`. So each outer pass decrypts **two dwords at 32 rounds per dword**: $7,644 \text{ passes} \times 64 \text{ stores} = \mathbf{489,216 \text{ stores}}$ covering `0xeeee2` = 61,154 bytes.

(An earlier reading of "90 rounds over 16,384 dwords" came from taking `cmp eax, 0xc1` as the inner trip count without tracing the pointers. Correction 5.)

3d. The keystream was not recovered, and the negative that "excluded" this cipher was width-scoped

No key constant was isolated and the keystream is not recovered in closed form.

It is generated per-dword from evolving state, so there is no short period to find.

A prior negative had recorded: *no short-period keystream under XOR, ADD, SUB, ROL, ROR, byte reversal, or column index-of-coincidence for periods 1–256*. That negative is **correct and irrelevant** — it was measured at **byte** granularity, and this cipher operates on **dwords**. A dword subtraction is invisible to it regardless of the arithmetic. This is the clearest instance in the engagement of the standing rule that a negative claim is a scoped claim, and the scope that mattered was operand width.

Similarly, a chi-square test of XOR-of-adjacent-bytes (223.9) and SUB-of-adjacent-bytes (259.7), both at or below the $df=255$ expectation, was initially under-read as evidence *against* a subtractive scheme. It excludes only a **constant** increment. The actual cipher is subtractive with a freshly computed subtrahend per dword.

The write range is genuinely unresolved. `ebx` has **18 writes** inside the loop body, two of them memory-dependent (`add ebx, [edx]` at `0x4b8000`, `add ebx, [esp]` at `0x4b8361`). The target range is data-dependent, so the `0x10000` extent measured in Finding 1e bounds the *ciphertext*, and is not a proof of what this loop writes.

3e. What the decryptor cost, and what that costs an emulator

A Python interpreter over capstone-decoded instructions, running the routine on a private register and memory model, completes it in **187,761,768 steps** — 7,645 outer passes — reaching the sample's own `ret` at `0x4b845c` with no halt and no step cap, at ~497,694 steps/sec, in 368.7 seconds. That is roughly 1.5×10^8 instructions to

decrypt 61 KB. The junk is not only anti-disassembly; it is an anti-emulation cost that most sandboxes will not pay.

(An earlier note recorded "the interpreter cannot reach termination." It had not failed; its background output file was read before it finished. An unfinished background run and a failed one look identical if you check the file too early. Correction 6.)

4. The decrypted loader body (L2)

4a. Position-independent, with a delta-offset prologue

plain.bin is 61,160 bytes, base VA 0x4b8716, so file offset = D - 0x401000 for any [ebp + D] reference.

```
000000 e8 00000000    call 0x4b871b
000005 5d             pop  ebp
000006 8b c5         mov  eax, ebp
000008 81 ed 05104000 sub  ebp, 0x401005
```

A textbook GetPC / delta-offset stage entry. *(Two details corrected from an earlier brief: there **is** a mov eax, ebp at offset 6, with eax used separately at 0x1e, and the constant is 0x401005, not 0x401000. Correction 7.)*

4b. API resolution is a literal string comparison, not a hash

The resolver is at offset 0x286, 66 instructions, called from 0x99 and 0xb6. Its core is unambiguous:

```
0x2c9 mov  edi, [esp+0x10]
0x2cd mov  ecx, 0x96           ; 150-byte name cap
0x2d4 repne scasb
0x2f7 repe cmpsb             ; literal byte-for-byte name comparison
```

It walks the PEB — mov ebx, fs:[0x30], then [ebx+0x0c] (Ldr), [ebx+0x1c] (InInitializationOrderModuleList), second entry (kernel32), [ebx+0x08] (DllBase) — validates MZ at 0x2af and PE\0\0 at 0x2bd, resolves ordinals through [edx+0x24] twice then [edx+0x1c] four times, and returns with ret 8. It **does not handle forwarded exports**.

This matters because it retrospectively explains a large negative result. A hash-resolver sweep across 12–15 hash constructions (ror13, ror13+null, ror7, rol5^, rol7^, djb2, djb2^, sdbm, fnv1a, fnv1, crc32, crc32+null, plus additive variants) over

26 Winsock and 12 CryptoAPI names found nothing. **Zero was the right answer — for a reason the sweep could not know.** That sweep is dissected in Finding 12b, because its first run was a false-positive bloom and its second run's positive control returned zero, voiding it.

4c. The function inventory closes, and the indirect count is 1

Six entry points, **262 instructions**, spanning 0x0000–0x0385, with **zero unreachable bytes** in 0x0000–0x0351:

ENTRY	LAST INSN	SPAN	INSNS	CALLERS
0x0000	0x0000	5	1	(image entry)
0x0005	0x025b	603	166	0x0
0x0260	0x0264	5	3	0x9e, 0xbb, 0xd7, 0x275
0x0265	0x0285	33	15	0xee, 0x283
0x0286	0x0351	204	66	0x99, 0xb6
0x0354	0x0385	50	11	(none — reached only via fs:[0] SEH dispatch)

The last row is the reason seh_roots.py exists: a handler reached only through exception dispatch has **no call site at all**, so it never enters an indirect-call denominator.

Indirect transfers: 18 — sixteen call dword ptr [ebp+0x4014xx] sites, one call eax at 0x222, one jmp eax at 0x258. **Seventeen of eighteen are resolved by name** against the API table. The single genuine unresolved exit is 0x253 mov eax, 0x785500 / 0x258 jmp eax, whose target lies outside the 61,160-byte body (it would be file offset 0x2ccdea).

Residual indirect count for this layer = 1. That is the scope limit on any "no path exists" claim about L2.

4d. The API table, and how the one ambiguous entry was adjudicated

A KERNEL32 name block runs 0x426–0x625, terminated by 0xFF at 0x626, followed by a SHELL32 block. The layout is NAME\0 immediately followed by the 4-byte slot

the resolved pointer is written into. **31 KERNEL32 names, 11 referenced by reachable code:**

SLOT	API	CALL SITES	PUSHES	MSDN ARITY
0x43a	GetProcAddress	0x202, 0x26f	2	2 ✓
0x44d	LoadLibraryExA	0x0d1, 0x1f4	3	3 ✓
0x474	CreateFileMappingA	0x112, 0x12c	6	6 ✓
0x485	CreateMutexA	0x22f	3	3 ✓
0x496	CreateThread	0x190	6	6 ✓
0x4a7	SetErrorMode	0x0f8	1	1 ✓
0x4e7	CloseHandle ¹	0x236	1	1 ✓
0x4f9	MapViewOfFile	0x142	5	5 ✓
0x503	Sleep	0x1a1, 0x1ab, 0x245	1	1 ✓
0x51a	GetModuleFileNameA	0x1d5	3	3 ✓
0x617	ExitProcess	0x24d	1	1 ✓

¹ The one ambiguous pairing. The name literally preceding slot 0x4e7 is GetLastError (arity 0), but the site pushes one argument — the CreateMutexA handle. **The alignment was tested rather than assumed:** shifting the name-to-slot mapping by $k = -1$ gives 1 arity match out of 12; **$k = 0$ gives 13 of 14**; $k = +1$ gives 0 of 16. The $k=0$ alignment is correct and the entry is CloseHandle.

Twenty names are present but unreferenced: VirtualAlloc, lstrlenA, ReleaseMutex, GetTempPathA, CopyFileA, CreateFileA, CreateFileW, OpenFile, GetFileSize, UnmapViewOfFile, SetFilePointer, SetEndOfFile, lstrcmpiA, lstrcatA, GetTickCount, CreateDirectoryA, WideCharToMultiByte, _lopen, and the CloseHandle/GetLastError pair.

4e. The loader's behaviour: a named shared section, a thread, and a self-relaunch

The mechanism is **not** the expected VirtualAlloc + copy + jump. VirtualAlloc is

resolved and never called.

```
0x0f8 SetErrorMode(0x8002) ; SEM_FAILCRITICALERRORS|SEM_NOGPFAULTERRORBOX
0x112 CreateFileMappingA(-1, 0, 4, 0, 0x8000, name@0x660) ; PAGE_READWRITE, pagefile-backed
0x12c CreateFileMappingA(-1, 0, 4, 0, 0x15400, name@0x64c) ; 87,040 bytes
0x142 MapViewOfFile(eax, 6, 0, 0, 0x15400) ; FILE_MAP_WRITE|FILE_MAP_READ
0x15b mov ecx, 0xe8ec ; 59,628 bytes
0x160 lea esi, [ebp+0x401000] ; source = file offset 0 - copies itself
0x16e rep movsb ; verbatim copy
```

The length 0xe8ec is exactly file offset 0 through the end of the embedded PE — a boundary derived independently in Finding 5a — excluding the 0x5fc-byte zero tail. The destination is a **named, pagefile-backed section** (hh8geqpHJTkdns6 at 0x660, purity_control_7728 at 0x64c). That is shared-memory IPC — an injection and hand-off mechanism, not unpacking. **The UPX payload is still packed after this body runs.**

The tail then does the following:

```
0x190 CreateThread(0, 0, start=ebp+0x4016d1, param=[0x693]=ebp, 0, 0)
0x19f Sleep(-1) ; INFINITE
0x1a9 Sleep(0xc)
0x1ba <-- SEH resume target
0x1d5 GetModuleFileNameA(0, buf@0x1778, 0x1fe)
0x1f4 LoadLibraryExA("SHELL32.DLL"@0x627, 0, 0)
0x202 GetProcAddress("ShellExecuteA"@0x633)
0x222 call eax = ShellExecuteA(0, "open"@0x426, path@0x1778, ...) ; SELF-RELAUNCH
0x22f CreateMutexA(0, 0, "Ap1mutx7")
0x236 CloseHandle
0x240 Sleep(0x927c0) ; 600,000 ms = 10 minutes
0x24b ExitProcess(0)
0x253 mov eax, 0x785500
0x258 jmp eax
```

The Sleep(INFINITE) at 0x19f is unreachable in the normal path — it is the SEH resume mechanism's fall-through, and control arrives at 0x1ba instead when the handler fires.

4f. A hand-rolled SEH frame used for robustness, not obfuscation

Installed at 0x286, uninstalled at 0x348:

```
install: push [ebp+0x401354] ; prev handler slot -> file offset 0x354
0x28e push fs:[eax]
0x297 mov [eax], esp -> saved at 0x386
0x299 mov [eax+4], ebp -> saved at 0x38a
0x2a2 mov [eax+8], ebx -> resume EIP = 0x1ba, saved at 0x38e
```

```
0x2a5 mov fs:[0], esp
uninstall: 0x348 pop fs:[0] / 0x34e add esp,4 / 0x351 ret 8
```

The handler at 0x354 rewrites `CONTEXT.Eip (+0xb8)`, `.Esp (+0xc4)` and `.Ebp (+0xb4)`, then returns 0 (`ExceptionContinueExecution`), resuming at 0x1ba. This accounts for the only unattributed region in the first segment: 0x352–0x424, 210 bytes of which 41 are nonzero, all belonging to this handler plus a 20-byte tail best read as three saved-context dwords.

4g. Two scoped negatives on this layer

No anti-debug checks. Exactly **four** segment-register accesses exist across the 262 reached instructions: 0x046 `mov ebx, fs:[0x30]`, 0x28e `push fs:[eax]`, 0x2a5 `mov fs:[0], esp`, 0x348 `pop fs:[0]` — the resolver's PEB walk and the SEH frame. None of PEB+0x02 (`BeingDebugged`), PEB+0x68 (`NtGlobalFlag`) or PEB+0x18→+0x40/0x44 (heap flags) is read. `GetTickCount` is in the API table with **no call site**. *Scope: 262 reached instructions across [0x0, 0x34ec) + [0xe000, 0xeeee8), keyed on PEB offsets and import names, with no dataflow, and no coverage of the compressed region or of anything past `jmp 0x785500`.*

No self-modifying code and no further decryption layer. Instructions with a memory destination and an opcode in {`xor`, `sub`, `add`, `rol`, `ror`, `not`, `neg`, `shl`, `shr`, `adc`, `sbb`} over the reached set: **count = 0**. The only bulk memory operations reached are 0x16e `rep movsb`, 0x269 `scasb`, 0x2d4 `repne scasb`, 0x2f7 `repe cmpsb`.

A methodological note on that last count: a linear sweep at every byte offset produces thousands of such "hits" — decode garbage from mid-instruction starts. The recursively-reached instruction set is the correct denominator, and using the linear one would have manufactured a finding.

5. The UPX layer (L2 → L3 → L4)

5a. The carve is bounded on both sides, three ways

The embedded PE begins at `plain.bin` offset 0x34ec (valid MZ, `e_lfanew = 0xd8`, PE\0\0, ImageBase 0x400000, entry RVA 0xb7ac0), with its DOS stub at 0x3539. Section table:

SECTION	VSIZE	VADDR	RAWSIZE	RAWPTR
UPX0	0xac000	0x1000	0	0x400
UPX1	0xb000	0xad000	0xae00	0x400
UPX2	0x1000	0xb8000	0x200	0xb200

Completeness is a two-sided bound, not a guess:

CHECK	EXPECTED	MEASURED
PE file span (UPX2 RawPtr 0xb200 + RawSize 0x200)	0xb400 = 46,080	46,080 exact
Decompressed size vs the stub's own p_filesize	0xb5110 = 741,648	741,648 exact
Bit-stream consumption must terminate before the stub at raw 0xae0	—	stopped at 0xaebd

That third row is the strong one. The decoder hit the end-of-stream marker **three bytes before** the stub's entry point. Too short a carve underruns the stream; too long leaves the terminator unreached. Only the correct extent lands in that window.

5b. Every decompression parameter came from the stub's instructions, because the header fields do not parse

The UPX! magic sits at 0x3e0 in the carve (plain.bin 0x38cc), with l_info = 0d 09 08 09 = version 13, format 9, **method 8 (NRV2B), level 9**.

But the p_info / b_info fields at 0x3e4–0x400 **do not parse at any alignment** and were not used: p_filesize reads as 0xe649f41e, sz_cpr as 0x39c00, method as 38. Every parameter was instead read out of the stub's own instruction stream — which independently reproduces p_filesize = 0xb5110, matching the measured output length exactly. The compressed stream begins at raw 0x400, stated by the stub itself (mov esi, 0x4ad000, where 0x4ad000 = ImageBase 0x4000000 + UPX1 VA 0xad000, RawPtr 0x400); there is no p_info skip at runtime because the stub is loaded with headers already stripped.

*(The apparently-scrambled sz_cpr = 0x39c00 was later shown **not scrambled at all** — it*

is the final file size: `.reloc RawPtr 0x38200 + 0x1a00 = 0x39c00.`)

5c. Two details a published-listing reimplementation gets wrong

Both are flag-provenance issues, invisible unless branch *targets* are read rather than mnemonics. Both were found by transcribing the stub instruction-for-instruction, and either one silently corrupts the output.

1. `jmp 0x4b7b41` → **LEN arrives with CF set by `sar eax, 1`, not by a `getbit`.** The first length bit on the new-offset path is the **low bit of the offset byte**, already consumed from the stream. An earlier `nrV2b.py` consumed a fresh `getbit()` there and ran one bit ahead from the first non-repeat match onward.
2. **The `.ext` loop terminates on `jae`** (continue while `bit == 0`), and `adc ecx, 2` at TAIL uses an **unsigned** `cmp ebp, 0xfffffb00` — so the extra length byte goes to matches *further back* than `0x500`, not nearer.

The variant is therefore: NRV2B, LE32 shift-register bit order, MSB-first, sentinel-refill; `m_len` base 2 with +1 for offsets below `-0x500`; end-of-stream is the offset byte producing `eax ^ 0xffffffff == 0`.

5d. The E8/E9 unfilter, and the byte permutation that is not a byte swap

Parameters read from the stub rather than guessed: `count ecx = 0x48f = 1,167`, marker byte `cmp byte ptr [edi], 6 = 0x06`.

The subtle part is the permutation. Tracing `mov eax,[edi]; shr ax,8; rol eax,16; xchg ah,al` symbolically over bytes `b0..b3` gives `eax = [b3, b2, b1, 0x00]` — a **24-bit big-endian** value built from `b1, b2, b3` with `b0` (the `0x06` marker) discarded. It is **not** a 32-bit byte reversal, which is what a published listing would lead you to implement. Also, the loop back-edge at `0x4b7be4` targets `0x4b7bbf` (`cmp al, 1`), not the scan head, so an E8/E9 byte immediately following a patched site is reused without a fresh load.

Two independent confirmations, neither circular:

- Applied to the decompressed image it performs **exactly 1,167 fixups**, matching the stub's hardcoded `ecx`. The count is reached by scanning decompressed bytes, so the agreement is not an artefact of reading the constant.
- With `base = 0`, **1,167 of 1,167** rewritten call targets land inside the image. With `base = 0x1000`, only 1,133 of 1,167. With the VA base, **0 of 1,167**.

5e. Validating the output, and a completeness check that had to be replaced

MEASURE	VALUE	READING
chi-square, whole file	145,941,761	far from uniform
chi-square, 0x200–0x38000	22,802,847	ordinary code
Shannon entropy, same range	3.430 bits/byte	ordinary code, not packed

Disassembly at filtered call site 0x401250 yields 74 well-formed instructions in 0x100 bytes with coherent frame-relative accesses.

The buffer is 87.88% zero, which raised a "correct but incomplete" concern. That concern was first closed by mapping the final PE's section table and citing .CRT — a 4-byte section 725 KB in, holding exactly 3 nonzero bytes at 0xb1001–0xb1003, on the reasoning that a partial decompression does not land content at single-byte precision that far in.

A second pass disproved that basis while confirming the conclusion, and the reasoning is worth preserving. The PE header found at buffer 0xb4f67 = RVA 0xb5f67 sits **beyond the SizeOfImage = 0xb5000 it declares** — a header cannot lie outside the image it describes. It is the **original, pre-UPX header** that UPX preserves verbatim to rebuild at runtime. Its va - 0x1000 mapping fails page-wise: **29 of 57** predicted-initialized pages are entirely empty, and 2 content pages are predicted by no section; its IMPORT rva 0x38098 → buffer 0x37098 lands in an empty region yielding **0** descriptors.

*So the .CRT landing was **one hit from a mapping that fails 29 times out of 57** — a multiple-comparisons error, not corroboration.*

What survives as the completeness check is `len == 0xb5110` matching the stub's own declared `p_filesize` exactly. Decoder-internal, but sound and sufficient. The zero fraction reconciles at $651,736 / 741,648 = 87.88\%$ whole-buffer and 88.33% over `0..SizeOfImage-0x1000`; .bss alone (493,609 bytes, 66.6% of the buffer) is all-zero and expected to be.

6. The decrypted body is a memory image (L2, revisited)

6a. Two independent decrypts agree over 92.92% of output, after a displacement is accounted for

Two implementations, written independently, produced plaintexts that a naive offset-0 comparison reported as mismatching **from byte 0** — 55,388 mismatches. That framing was an artefact. The outputs differ by a constant displacement of 0x10e6 (4,326) bytes, confirmed by four independent structural landmarks all showing the identical delta:

LANDMARK	PASS-2 OFFSET	TEAM-LEAD OFFSET	DELTA
UPX!	0x049b2	0x038cc	+0x10e6
UPX0	0x047a2	0x036bc	+0x10e6
UPX1	0x047ca	0x036e4	+0x10e6
This program	0x04620	0x0353a	+0x10e6

Realigned: **0 mismatches over 56,828 bytes = 92.92%** of the shorter output.

The origin question was adjudicated on evidence independent of both implementations. The shorter output's first eight bytes are e8 00 00 00 00 5d 8b c5 = call \$+5; pop ebp — a textbook GetPC stage entry. The longer output's leading 4,326 bytes have entropy 7.324 with 256 distinct byte values, against 5.594 for the structured overlap: **undecrypted residue**, not extra recovered plaintext.

A degenerate-agreement trap was guarded rather than walked into. The per-1KB agreement profile showed two windows at exactly 100% while their neighbours sat at 0.1–1.5%. Those two windows (0x2000, 0x2c00) are **all-zero on both sides** — agreement that proves nothing — and were excluded from the reasoning. Relatedly, the ~0.4% agreement seen in the *misaligned* comparison is exactly the 1/256 random baseline; the correct reading of a misaligned diff is "no relationship", not "mostly corrupt".

6b. A memory-model theory was tested and refuted before it was believed

A plausible explanation for the mismatch was offered: that Machine.load was dword-keyed against a byte-granular address space, so reads overlapping a written dword returned stale ciphertext. Instrumented over 3 million steps: **14,701 exact hits, 7,881 clean image reads, 0 overlap misses.** The cipher only ever loads from

the address it is about to store to (or `ecx,[ebx] / mov [ebx],ecx`, same `ebx`), so the case never arises. A byte-granular variant (`keystream_bytemem.py`) produces byte-identical output and is retained as the negative control.

6c. `plain.bin` is a captured memory image, and this changes what six values mean

Every KERNEL32 slot in the API table holds a **live resolved address**: `CloseHandle` → `0x777f2ee0`, `VirtualAlloc` → `0x777ef3c0`, `Sleep` → `0x777f0f00`, clustering in `0x777e0000–0x7783xxxx`. Two slots sit in a different module range: `GetProcAddress` → `0x774a7fc0`, `LoadLibraryExA` → `0x774a4ae0`. The `ShellExecuteA` slot at `0x641` is still `00000000`.

These are decryption *output*, not interpreter artefacts — verified at the bytes: at file offset `0x422`, ciphertext `0x40b327df` decrypts to `0x777d0000`; at `0x43a`, `0x178224ed` decrypts to `0x774a7fc0`.

The conclusion is structural. The viral body was captured from a **live infected process after API resolution had already run**, then re-encrypted and appended to this host. The filled `0x777xxxxx` cluster and the still-empty `ShellExecuteA` slot even date the snapshot: after the KERNEL32 block was populated, before the SHELL32 resolution at `0x1e9–0x222` executed.

Consequently those six addresses are **environment-specific to whatever machine was infected** and are **not indicators of this sample**. They are excluded from the IOC section by decision.

Any claim of the form "this value is what the file contains" must state whether it is file content or captured process state.

6d. Config and URLs in this layer

```
0x000645  "\"
0x000647  "_Rar"
0x00064c  "purity_control_7728"      <- named section, used at 0x12c
0x000660  "hh8geqpHJTkdns6"        <- named section, used at 0x112
0x000670  "uxJLpe1m"
0x000679  "Ap1mutx7"                <- mutex, used at 0x22f
0x000682  "KERNEL32.DLL"
```

Only `Ap1mutx7` is referenced by reachable code among the mutex-shaped strings.

A URL pack sits at `0x31f6` with count byte `02`:


```
0x0031fa http://padrup.com/sobaka.aspx
0x003218 http://kuku1.infoier
```

Both are clean, NUL-terminated and low-entropy, preceded by a record header at 0x31f0 (0a 00 00 00 00 00 1e 00 35 00 00 00 02 00, where 0x1e = 30 = strlen+1 and 0x35 = 53).

hxxp://kuku1[.]infoier is recorded as bytes, with its reading not asserted. The stored bytes are 687474703a2f2f6b756b75312e696e6666f696572, NUL-terminated — so it is the literal stored string and not a truncation artefact. But ier is not a valid TLD and it matches no known host in this family's infrastructure. Candidate readings (a generation template kuku%d.info with adjacent bytes; a deliberate decoy; an offset-local artefact) are not distinguishable from the available evidence.

6e. Two false positives in this layer, adjudicated

hxxp://89[.]11 at offset 0x402d is not an IOC. It lies inside the compressed band (0x4000–0xe000, $H \approx 7.55$, chi-square ≈ 984 for 0x4000–0x4400) and is an LZ literal-run fragment. This is corroborated by the payload, where the same URL appears complete as hxxp://89[.]119[.]67[.]154/testo5/ — the fragment is the first-occurrence literal prefix, which is exactly the truncation mechanism the compressed representation produces.

The kernel symbols and AV kill-list are genuinely absent from this layer.

ObfDereferenceObject, PsLookupProcessByProcessId, KeSetEvent, IoCompleteRequest, ntoskrnl, ZwCreateFile, NtDeviceIoControl, avp.exe, nod32, ekern, mcshield, ccSvcHst, MsMpEng, Defender — none present in ASCII (case-sensitive or -insensitive) or UTF-16LE across all 61,160 bytes, against firing positive controls (CreateMutexA at 0x478, KERNEL32 at 0x682). A scan for any UTF-16LE printable run of 8 or more characters found **zero** — the file contains no wide strings at all. *Scope: no stacked or obfuscated string reconstruction was attempted, and those symbols could exist inside the compressed region — as indeed they do, one layer down.*

7. The kernel driver — the sample's network component

This is the most consequential finding in the report, and the way it was missed is as instructive as the finding itself.

7a. It attaches to the TCP/IP stack; it does not attack the IP filter

The driver is 4,672 bytes, Native subsystem (1), with an INIT section, TimeDateStamp 2005-12-25, carved from payload offset 0x1fcc4. It imports **ntoskrnl.exe exclusively** — ObOpenObjectByPointer, PsLookupProcessByProcessId, PsTerminateSystemThread, IoBuildDeviceIoControlRequest, IoGetDeviceObjectPointer, IoCreateSymbolicLink, ExAllocatePoolWithTag.

DriverEntry at RVA 0x62f calls IoCreateDevice with device type **0x15 = FILE_DEVICE_NETWORK**. It registers \Device\IPFILTERDRIVER (UTF-16 at 0x206d4), plus \Device\amsint32 (0x20a28) and \DosDevices\amsint32 (0x20a4c). All three MajorFunction slots point to a single handler at 0x104f0.

A correction is recorded here. The trio IoGetDeviceObjectPointer + IoBuildDeviceIoControlRequest + IoCallDriver was first read as the driver *attacking* the legitimate Windows IPFilter stack. That is **wrong**. They are how it **attaches itself** to TCP/IP in order to receive traffic; the device name impersonates the Windows filter as camouflage. The import set is a filter's, not a tamper tool's. (Correction 8.)

7b. The filter decision, transcribed

At 0x108f0:

```
if (global_enable[0x10db4] == 0)      -> fall through to hostname check
proto = byte [conn + 9]
if (proto != 6)                       -> ALLOW                        ; TCP only
sport = ntohs([addr + 0])
dport = ntohs([addr + 2])
if (sport == 0x19 || dport == 0x19)  -> BLOCK                        ; port 25 = SMTP
for entry in blocklist[0x10d00]:
    if substr(buf, len, entry, strlen(entry)) -> BLOCK
ALLOW
```

0x108c0 is a 16-bit byteswap, i.e. ntohs. 0x10810 is an O(n·m) substring search with a length guard (needle longer than haystack returns 0). The blocking is by **substring match against the payload buffer**, not by DNS name or by IP — so it catches the hostname wherever it appears in a request.

7c. The dispatch path is TDI, not IOCTL

The handler at 0x104f0 keys on MajorFunction == 4 (IRP_MJ_WRITE), then on a control code at [irp + 0x3c]: 0x29a allocates the hook table, 0x309 sets the enable flag, 0x378 clears it.

No 0x22xxxx IOCTL constants exist anywhere in the driver — searched, zero hits.

An earlier "IOCTL dispatcher" framing is therefore **withdrawn**. (*Correction 9.*)

7d. The blocklist — 23 entries under XOR 0xf9, and the family anchor

DriverEntry decrypts a pointer table at 0x10d00 **in place with XOR 0xf9**, in a loop at 0x1070f–0x10775, guarded by a sentinel check ([edx+2] == 's') so that it runs exactly once. The band is payload 0x2088c–0x209a8, 284 bytes, and entries are stored in reverse order. Decoded in full:

kaspersky	eset.com	f-secure.	mcafee.
symantec.	etrust.com	trendmicro.	sophos.
virusotal.	agnmitum.	pandasoftware.	bitdefender.
spywareguide.	windowsecurity.	virusscan.	ewido.
spywareinfo.	onlinescan.	drweb.	cureit.
virusinfo.	sality-remov	upload_virus	

0xf9 is a genuine key, established against a null model: 400 random 12-byte patterns swept against all 255 single-byte XOR keys produced **0 hits**.

The last two entries are the family anchor. sality-remov and upload_virus block a Sality removal tool and a sample-upload page. This is the malware defending itself, by name, against its own removal and its own submission for analysis — and it is a stronger basis for the family call than the plaintext Sality string at 0x2298c, which sits inside the sample's own AV kill list and is therefore something the malware searches *for*.

A correction history worth keeping. Before the payload was decompressed, this same list appeared as truncated fragments at plain.bin 0x4d60–0x4e34 under XOR 0xf9 (in-band printability 0.66 against 0.24 file-wide). It was misread twice: first as an obfuscated table, then as "0xf9 is not a key, only LZ literal structure." **The correct reading is that the 0xf9 is real and the truncation is NRV2B literal-run structure over the already-XORed bytes.** Only sality-remov appeared complete there, and it was the crib — true by construction, which is exactly why it could not confirm the hypothesis it was used to test. (*Correction 10.*)

7e. Kernel-mode process termination, dispatched by Windows build

0x10280 takes a PID, calls PsLookupProcessByProcessId then ObOpenObjectByPointer, and returns a handle. A thread at 0x102d0 reads NtBuildNumber and switches on it:

NTBUILDNUMBER	INDEX
0x565	0xbb
0x893	0xe0
0xa28	0x101
0xece	0x10a
0x14bc	0x14f
0x1770	0x152

The index selects a dword from a table allocated with `ExAllocatePoolWithTag` (tag `0x206b6444 = "Ddk "`) and calls through it — a per-Windows-version offset into an undocumented kernel table. **An unrecognized build selects index 0 and the thread exits**, so this list bounds which systems the kill capability functions on. The string `KeServiceDescriptorTable` at payload `0x21294` names what such a table is, and its presence in the parent is consistent with the mechanism, though the driver reaches it through the offset table rather than by name.

7f. Why four methods with non-overlapping blind spots all missed this

For most of this engagement the sample's network component was held to be **absent from all layers**, on four methods deliberately chosen to have non-overlapping blind spots, each with a firing positive control:

1. A **two-mechanism API enumeration** — the static import table *and* the runtime-resolved name block.
2. A **hash-resolver sweep** across 12–15 constructions.
3. A **chi-square pass** finding no further encrypted stage.
4. **Section-coverage subtraction.**

They were non-overlapping with respect to each other, and they still shared one blind spot: **every one searched user-mode network APIs**. The driver borrows the OS TCP/IP stack instead of implementing one, so it has no network imports for any of them to find. It is identified by a **device-type constant**, not by an API name.

The correction came from carving embedded executables by **MZ header scan** and

parsing **each one's own headers** — a method that assumes nothing about what kind of program it is looking at.

A convergent negative is only as broad as the narrowest assumption its methods share.

The negative was stated as "absent from all three layers" when the defensible scope was "absent from the **user-mode surfaces** of all three layers." Everything in the Negative Findings section below is scoped accordingly.

8. The code island — 90,516 bytes belonging to no program

8a. Finding a region that was in nobody's accounting

Subtracting all three carved PE extents from the payload's content islands leaves one contiguous run: **0x20f04–0x37097, 90,516 bytes, 85.8% nonzero**. It begins immediately where the kernel driver ends. It has no PE header, appears in no section table, and was in no program's accounting until it was measured — while being larger than the driver, the stub and the DLL's present content combined by an order of magnitude.

This region holds the infection routine, the polymorphic engine, and the payload's cryptographic primitives.

8b. Recovering a load base without a header

Absolute pointer-like dwords in the region cluster around 0x006a0000 / 0x00680000. Scoring candidate load bases by how many such dwords land inside the region:

FRAMING	BEST BASE	POINTERS IN RANGE
flat map, file offset → base+offset	0x67a000	492
region start treated as the VA base	0x69b000	492

Two different framings resolve to the same VA window, with a broad plateau (490–492 across $\pm 0x5000$). Base $\approx 0x69b000$. It is **not a PE image base**, which is consistent with code written into allocated memory rather than mapped from a file.

A tool-independent self-consistency check corroborates it. Of **1,295** e8 sites, **798**

(61.6%) target inside the region, giving **164 distinct targets**, of which **145 (88.4%)** **land exactly on 55 8b ec** — push ebp; mov ebp, esp. The remaining 497 calls are outbound to imports and thunks. *Caveat, stated because it bounds the claim: the scan treats every e8 byte as a call opcode, and a few non-prologue targets decode as add [eax], al, the signature of data read as code. The prologue agreement carries this claim, not the raw target count.*

8c. Coverage measured as distinct bytes, by two methods sharing no principle

METRIC	VALUE
functions	178 (independent prologue u call-target union: 196)
region bytes	90,364
distinct bytes attributed to a function	77,169 = 85.4%
overlapping bytes	0
nonzero bytes	77,576 (85.8%)
coverage of nonzero bytes	99.5%

Across four configurations:

CONFIG	FUNCS	DISTINCT BYTES	OF RANGE	OVERLAP	OF NONZERO
aaa (unbounded)	178	77,169	85.4%	0	99.5%
aaa + hasnext=1	189	77,716	86.0%	15	100.2%
aap prologue sweep	177	77,163	85.4%	0	99.5%
union aap + aaa	178	77,169	85.4%	0	99.5%

85.4% from two methods sharing no principle — recursive descent from call targets, and a prologue-signature sweep — **agreeing to within 6 bytes**. The 14.6% shortfall is padding. The hasnext=1 row is a known artefact: it exceeds 100% of nonzero and is the only configuration showing overlap, i.e. it is absorbing data-in-code.

Scope limits on any negative drawn from this region. Pang et al., *SoK: All You Ever Wanted to Know About x86/x64 Binary Disassembly* (IEEE S&P 2021; 9 tools, 3,788 binaries) measures recursive descent alone missing ~49% of code on average. 85.4% sits far above that regime because this is compiler-generated code with intact prologues, and the direction of any residual error is **undercount, never overcount**. Critically, the region **has no PE header and therefore no .pdata/.xdata**, so `seh_roots.py` contributes no root set at all — **the indirect-call floor bounding a "no path exists" claim here is wider than usual, not narrower**. Linear-sweep decode success was not used as a coverage metric anywhere (the same source measures 100% decode over 92.1% code).

8d. Three tool defects caught while measuring it, each producing a plausible number

These are recorded as findings because each one produced a believable figure rather than an error, and two of them briefly entered the engagement's own notes as fact.

1. **-m 0x69b000 collapsed the sweep from 178 functions to 1.** Three unmapped variants each returned 178, stable. *A map argument that silences analysis looks identical to code that has none.*
2. **A JSON key mismatch fabricated a 10.8% coverage figure.** This r2 build emits `addr`; the parser read offset with a 0 default, stacking all 178 functions at address 0. It was caught by internal contradiction: the sum of function sizes was 81,242 (89.9%) while distinct covered bytes read 9,736 (10.8%), and the largest single function was exactly equal to the covered total.
3. **anal.in=raw is not a valid value.** r2's own enumeration (`e anal.in=?`) lists `range`, `block`, `bin.segment*`, `io.map*`, `dbg.map*` — no `raw`. r2 accepts it, stores it, and echoes it back: a **silent no-op**, with analysis running unbounded over the mapped file. The 85.4% figure survives the correction, having been re-run across the four configurations tabulated above.

And a fourth, which is the reason the figure is trustworthy at all: the *documented* range-bounding form (`e anal.in=range; e anal.from=..; e anal.to=..; e anal.limits=true`) **suppresses the prologue sweep entirely** — aap returns 0 functions bounded against 177 unbounded, while `/x 558bec` independently confirms 177 prologues present and `om` shows the map covering the range correctly. This was caught only because the prologue scan carries a firing control; the recursive-descent configuration has none.

A setting the tool accepts is not a setting the tool honours.

8e. The island's cryptography, and a negative corrected within minutes of being written

CONSTANT	OFFSET (REGION-RELATIVE)	CONTEXT
0x67452301, 0xEFCDA8B9, 0x98BADCFE	+0x12858--0x1286c	mov dword [reg+off], imm32 into consecutive fields
0xEDB88320	+0x0229c	shr ecx, 1/xor ecx, 0xEDB88320 in a loop — bitwise CRC32

An earlier assertion that this 90 KB region "carries no crypto imports either" was **inferred from the absence of Crypt* imports and never measured**, and the constant scan contradicts it. (*Correction 11.*) Relatedly, the SHA string hits in this region are false positives (SharedAccess, ASHAVAST) — and that sweep's own positive control barely fired (ReadFile = 1, Sleep = 0). **A control that barely fires does not license a negative.**

MD5Init is identified unambiguously: 64 bytes, push ebp; mov ebp, esp at region +0x1283c, one pointer argument; it zeroes [ctx+0x10] and [ctx+0x14], then writes 0x67452301, 0xEFCDA8B9, 0x98BADCFE, 0x10325476 into [ctx+0]..[ctx+0xc] **field by field** — which is why a scan for a contiguous 4-dword constant table found nothing.

It has exactly one direct caller, an algorithm dispatcher at +0x1500c with signature (ctx, alg_id):

```
mov [ctx], alg_id
cmp alg_id, 5
je  -> push ctx+4 ; call MD5Init ; return 0
else  return 0x402      ; 1026 - unsupported-algorithm code
```

That dispatcher has 3 callers; two of them (+0x150c4, +0x150f2) are each called from two sites, one pair adjacent to the CRC32 loop at +0x0229c. Walking upward converges on a common ancestor at +0x0442c (6 callers) and above it +0x11c5c. **This is live code on a real call chain**, not an unused library fragment.

9. The infection routine

9a. The PE parser accepts 32-bit images only

fcn.006a2603, VA 0x6a2603, **151 bytes**, returning all four structure pointers via out-parameters:

```
0x6a2615  cmp edx, 0x5a4d          ; 'MZ'          81 fa 4d 5a 00 00
          mov ecx, [eax+0x3c] ; e_lfanew
0x6a2626  cmp dword [edx+ecx], 0x4550 ; 'PE'          81 3c 0a 50 45 00 00
0x6a2646  cmp dword [ecx], 0x4550   ; 'PE' again    81 39 50 45 00 00
          add eax, 4        ; -> IMAGE_FILE_HEADER (out arg_ch)
          add eax, 0x14     ; -> IMAGE_OPTIONAL_HEADER (out arg_10h)
0x6a2676  cmp ecx, 0x10b          ; PE32 magic    81 f9 0b 01 00 00
          add eax, 0xe0     ; -> section table (out arg_14h)
          mov eax, 1        ; success
```

Rejecting anything but 0x10b bounds the infectable target set **by construction**, not by a behavioural guess.

All four compares carry 32-bit operands. Verified with capstone at island offset 0x28507; there is no 66 operand-size prefix anywhere in the 151 bytes. This matters for a detection author: a rule written against the 16-bit encodings these compares *look* like they should use — 66 81 fa 4d 5a, 66 81 f9 0b 01 — matches nothing in this sample. The constants fit in 16 bits; the instructions do not use that width. See Correction 26.

9b. The relocation walker locates a host global with no symbols, imports or strings

fcn.006a269a, **360 bytes**, reads [ecx+0x88] — DataDirectory[5], base relocations:

```
[var_18h] = imagebase + reloc_dir_rva      ; block cursor
per block: entries = ([blk+4] - 8) >> 1    ; SizeOfBlock
per entry: (word >> 12) & 0xf == 3        ; IMAGE_REL_BASED_HIGHLOW only
          rva = [blk] + (word & 0xfff)
candidate = filebase + rva
check: [candidate] - OptionalHeader[0x1c] ; minus ImageBase -> VA-to-RVA
      == arg_ch                          ; caller-supplied wanted RVA
check: word at [candidate-2] == 0x5c7
return: [candidate+4] - ImageBase
```

0x5c7 little-endian is the bytes c7 05, the opcode for mov dword [imm32], imm32. So the routine hunts the *host's own relocation table* for a HIGHLOW fixup that is the pointer operand of that instruction and matches a wanted RVA, then returns the RVA of the **second** immediate.

That is an elegant and nasty technique: it locates a global variable inside an arbitrary host binary using nothing but the relocation table and an opcode signature — no symbols, no imports, no string search, and nothing that a stripped binary lacks.

Its entry gate is `cmp word [edx+0x12], 1` against `IMAGE_FILE_RELOCS_STRIPPED`. **A host with relocations stripped is skipped outright.**

9c. Corollary: this is why `.reloc` was the target

Findings 1b and 9b close a loop. The infector needs the host's relocation table intact in order to find its target global. It also needs somewhere to put 69,632 bytes. Appending to `.reloc` and marking that section `+X+W` accomplishes the second without disturbing the first — the 143 genuine relocation blocks are still present and still parse, ending at `0xb4b6c`, with the viral code beginning at `0xb4c00`. Zeroing the `BASERELOC` directory prevents the Windows loader from processing relocations at load time while leaving the *data* intact for the infector to read.

9d. The polymorphic engine, observed as code

`fcn.006a44eb`, **549 bytes**, with **35 cross-references** and **30 direct call sites**, all sharing an identical call shape:

```
lea edx, [ebp + ecx - 0x14fc]    ; a code buffer
push edx
push 0x12345678                 ; unrelocated placeholder
call fcn.006a44eb
add esp, 0xc
```

It is **not** a string decryptor, which is what such a call-site density usually indicates. It repeatedly calls `fcn.0069d5c7`, takes each result modulo 100, branches on probability thresholds `0x55` (85), `0x50` (80), `0x32` (50), and **emits instruction bytes**:

```
mov byte [edx], 0xeb           ; emit JMP
mov byte [eax], dl             ; emit computed byte
mov byte [ecx - 1], dl
```

`fcn.0069d5c7` — **123 call sites** — is a linear congruential generator:

```
mov  eax, [seed]
imul eax, eax, 0x15a4e35      ; 22695477
add  eax, 1
```

```
... store seed ...  
sar  eax, 0x10  
and  eax, 0xffff          ; return bits 16..31
```

0x15A4E35 = 22,695,477 is the Borland/Turbo C LCG multiplier.

This ties a banner string to executing code. The string Simple Poly Engine v1.1a (c) Sector sits at payload 0x21520 (with (c) Sector NUL-separated at 0x2153c). Before this trace it was a banner and nothing more — the kind of self-attribution that names a capability without demonstrating one. It is now observed as a code generator making randomized emission decisions at 30 call sites.

On the LCG constant specifically: it is cited as an implementation detail, **not** as attribution. A published incompressible constant is observable evidence where a computable one is not — but 22,695,477 is a well-known library constant, so a match on it says the author used a common LCG, not that this code shares lineage with anything else that uses it.

9e. The import set names the anti-forensics

The payload's own plaintext import-name table runs 0xb4009–0xb467a, **89% dense**, and the file-manipulation cluster is diagnostic:

```
CreateFileMappingA  MapViewOfFile  UnmapViewOfFile  
SetFilePointer      SetEndOfFile   IsBadWritePtr  
GetFileTime         SetFileTime      <- timestamp preservation  
GetFileAttributesA SetFileAttributesA <- read-only defeat  
FindFirstFileA / FindNextFileA / FindClose  lstrcmpiA  
GetLogicalDrives    GetDriveTypeA    <- drive enumeration
```

GetFileTime / SetFileTime as a **pair** is the anti-forensic tell: read the original MAC times, modify the file, write them back, and a last-modified sweep across the estate finds nothing. SetFileAttributesA + GetFileAttributesA clears and restores the read-only bit around the write. Memory-mapped I/O rather than ReadFile/WriteFile is how a PE is edited in place, and SetEndOfFile commits the grown file.

(Two counts of this table appear in the working notes — 105 names in one section, "113 ASCII strings of which 84 are API names" in another, and a third measurement gives 100 named imports. The three were measured with different inclusion criteria — all strings in the span, versus names matching an API pattern, versus entries resolvable to an import descriptor — and were never reconciled. The discrepancy does not bear on any finding, and is recorded here rather than silently resolved. Logged as an open item.)

9f. fcn.006a2802 in eight stages

Routine extent 0x6a2802–0x6a2bca, with **6 call sites**. This is the routine the engagement expected to be the infecting write.

Stage 1 — path handling. Zeroes two 0x40-byte buffers (0x6a280c, 0x6a2825); two imports through unrelocated thunks obtain and measure a path; reads the last character and appends a separator if absent:

```
0x6a287d  movsx eax, byte [ebp+eax-0x225]
0x6a2885  cmp     eax, 0x5c          ; '\'
0x6a2888  je      0x6a289c
0x6a2896  <append separator>
```

Stage 2 — map, with a length-mismatch retry. A five-argument call at 0x6a28ae returns a status:

```
0x6a28b7  cmp dword [var_ch], 0xc0000004 ; STATUS_INFO_LENGTH_MISMATCH
0x6a28be  jne     (skip retry)
0x6a28c0-0x6a28c9  realloc using returned length [var_230h]
0x6a28e7  re-issue identical call
0x6a28f9-0x6a28ff  loop: re-test >= 0, repeat
```

Stage 3 — header reads. With the mapped base in var_234h:

```
0x6a2907  mov edx, [ecx+0xc] ; -> var_8h, held until stage 6
0x6a2915  mov cx, word [eax+0x1e]
0x6a291f  lea eax, [edx+ecx+0x20]
```

The result (var_11ch) becomes a *second* path component at 0x6a292f; fcn.006a3856 is applied to the second buffer at 0x6a2944, and the pair is passed to a three-argument import at 0x6a295c.

Stage 4 — two open attempts, fail-closed. Same thunk [0x7c000000], same trailing arguments (0, 1):

```
0x6a2966  push 1 / push 0 / lea eax,[var_118h] / call [0x7c000000] ; first path
0x6a2986  push 1 / push 0 / mov ecx,[var_11ch] / call [0x7c000000] ; second path
```

The first result goes to var_22ch; if zero (0x6a297d), the second is tried into the same slot; if that is also zero, the routine bails to the common exit 0x6a2bc6 returning 0.

Stage 5 — the view, and the walker. var_234h is released at 0x6a29b4, then:

```
0x6a29ba  mov eax, dword [0xc0060000]    ; global, reads -1
0x6a29bf  push eax
0x6a29c0  push [var_22ch]                ; handle from stage 4
0x6a29c7  call dword [0x78000000]
0x6a29cd  mov [var_120h], eax            ; base of the view
0x6a29d3  cmp [var_120h], 0 / jne ...    ; else bail
0x6a2a03  call fcn.006a269a              ; the relocation walker (9b)
0x6a2a11  cmp [var_248h], 0 / je bail
0x6a2a37  call fcn.006a2603              ; the PE validator (9a)
```

Stage 6 — a counting pass, then a building pass. The cursor starts at base + var_248h, the offset the relocation walker returned:

```
0x6a2a7b  mov eax,[cursor] / mov edx,[eax] ; a dword AT the cursor
0x6a2a86  sub edx,[ecx+0x1c]              ; minus OptionalHeader ImageBase
0x6a2a8c  cmp edx,[eax+0x38]              ; against SizeOfImage
0x6a2a8f  jae 0x6a2a93                    ; out of range -> stop
0x6a2a91  jmp 0x6a2a5d                    ; else advance 4, count++
```

(value - ImageBase) < SizeOfImage is the canonical "is this dword a pointer into this image?" test. The first pass counts into var_238h; lea edx, [ecx*4+4] at 0x6a2a99 then allocates count+1 dwords. The second pass at 0x6a2acb-0x6a2b21 rebases each:

```
0x6a2b0a  mov eax,[ecx]                  ; the VA
0x6a2b0c  sub eax,[edx+0x1c]             ; -> RVA
0x6a2b0f  add eax,[var_8h]               ; + the field captured at 0x6a2907
0x6a2b1e  mov [edx+ecx*4], eax           ; into the newly allocated array
```

The array is built in **freshly allocated heap, not written back into the view.**

Stage 7 — the write, and where it lands.

```
0x6a2b23  mov eax,[var_228h]
0x6a2b29  mov dword [eax], 0x29a         ; 666 -- a marker stamped into slot 0
0x6a2b39  push 0x40000000               ; GENERIC_WRITE
0x6a2b3e  mov ecx,[0xf8070000]          ; a global path/name
0x6a2b45  call [0x58010000]              ; 7 args -> var_23ch
0x6a2b51  cmp [var_23ch], 0xffffffff     ; INVALID_HANDLE_VALUE
0x6a2b58  jne 0x6a2b5e / xor eax,eax / jmp exit
0x6a2b5e  push 0 / lea edx,[var_4h] / push edx ; out: bytes written
0x6a2b64  mov eax,[var_238h] / shl eax,2 / push eax ; count*4 bytes
0x6a2b6e  push [var_228h]                ; the array
0x6a2b75  push [var_23ch]                ; the handle
0x6a2b7c  call [0x5c010000]              ; THE WRITE
```

```
0x6a2b89  call [0x78010000]                ; close the handle
```

Stage 8 — cleanup, and a call taking the path. The heap array is freed at 0x6a2b96; the victim handle is passed to a one-argument call at 0x6a2ba3; then:

```
0x6a2ba9  movsx eax, byte [var_118h]        ; the stage-4 path buffer
0x6a2bb0  test eax,eax / je 0x6a2bc1        ; empty string -> skip
0x6a2bba  push ecx / call [0x54010000]      ; one argument: the path
```

Success returns 1 at 0x6a2bc1.

9g. The trace inverted its own premise — two withdrawals

Withdrawn: that this routine patches the victim. The trace went looking for `mmap` → `patch` → `SetFileTime`. It found `mmap` → **read** → write-elsewhere. **No instruction in 0x6a2802–0x6a2bca stores into [var_120h] or any offset from it.** The only memory stores in the entire routine are into the freshly allocated heap array (0x6a2b1e, 0x6a2b29) and into local stack buffers. The write at stage 7 targets a handle opened on the global at [0xf8070000] — **not** the victim handle `var_22ch` and **not** the mapped view `var_120h`.

The earlier inference that this call chain performs the infecting write was built from the import set plus the callers — **the same string-shaped reasoning that the .adata reversal (Finding 13a) had already caught once in this engagement.** (*Correction 12.*)

Withdrawn: that [0x54010000] is SetFileTime. The argument shape is wrong. `SetFileTime` takes a handle plus three `FILETIME` pointers; this call takes **one** argument, which is a *path buffer*, and it occurs **after** the handle has already been closed. (*Correction 13.*)

What the routine actually is: it reads a PE, harvests the absolute pointers that the relocation marker anchors, rebases them, and writes that index to a fixed side file, stamped with 0x29A. That is consistent with a **preparation or bookkeeping step** for infection — and with several other readings.

The `mmap` → `patch` → `SetFileTime` sequence is therefore unstepped, because on this evidence it is not in this routine. Choosing between the available readings requires the six callers, and they are unread:

0x6a337b 0x6aa38b 0x6aaa01 0x6aaf4f 0x6ac297 0x6ac93d

Standing note on every thunk in this section. [0x78000000], [0x58010000], [0x5c010000], [0x7c000000], [0x54010000], [0x78010000], [0xf8070000] and [0xc0060000] are **unrelocated placeholder immediates**. They are cited above as *distinct call sites*, never as named APIs. An attempted index-into-name-table mapping was tested and **failed its control**: the eight 0x78 call sites in fc0.006a2fe6 are each guarded by a *distinct* global, which indicates lazy per-API resolution rather than eight calls to one API. That mapping is recorded as unverified and is used for nothing in this report.

10. The public key with no consumer

10a. The v4 modulus, recovered and byte-order-determined

PROPERTY	VALUE
Location	payload_unpacked.bin 0x13d8–0x14d8, 256 bytes , contiguous, high-entropy
Framing	Zero-bounded on both sides (8 zero bytes at 0x13c8, 8 at 0x14d8)
Preceding bytes at 0x13d0	00 01 00 01 00 00 00 00 — e = 0x10001
Modulus head	bb d2 96 8e ed 0b 93 8a 82 e4 e9 bc c3 c5 32 72 ...
Modulus tail	... a4 6e 64 18 e0 d0 7b 3b
Big-endian	exactly 2048 bits, odd, no prime factor below 100
Little-endian	2046 bits
Carved	reimpl/netsurface/v4_pubkey.bin, 264 B, sha256 bd912e...bd7bd695

Byte order is determined, not assumed — only the big-endian reading gives a clean 2048 bits. The value is odd and small-factor-free **at its natural boundary**, requiring no alignment search. Its last five bytes match a published family key tail exactly, at the same offset, length and prefix that this analysis recorded independently before those published bytes were consulted.

10b. The key has no consumer, and the negative is proven at the arithmetic

This is the finding that constrains what can be said about the tasking protocol.

There are **no crypto imports in any of the four programs**. More decisively — because an absence of imports is a weak negative, as Finding 8e demonstrated — the **absence of multi-precision arithmetic** was measured directly over the code island's full disassembly of **24,530 instructions**:

INSTRUCTION	COUNT	READING
mul	0	no unsigned multi-precision multiply
shld / shrd	0	no multi-precision shift
mul → adc chains	0	no limb propagation
imul	210	signed — indexing and hashing
adc / sbb	10 / 15	ordinary 64-bit integer arithmetic

RSA verification requires modular exponentiation, which requires multi-precision multiplication. Zero mul instructions in 24,530 is not a search that failed to find something; it is arithmetic that cannot be present.

The key was found. The verification routine was not, and cannot be here.

Signature verification therefore happens in a stage that was **never on this disk** — presumably in a downloaded payload. Any protocol-level claim about signed, unforgeable tasking remains [intel] and is not asserted by this report. Acquiring that stage is a sourcing decision that would push sample identifiers off this machine; it is **flagged, never initiated**.

*(A methodological note attaches to the recovery itself. The first modulus scan used Shannon entropy > 7.4 over a 256-byte window. **That threshold is above the mathematical ceiling** — 256 samples over 256 symbols cap near 7.17 — so the test returned zero for **every possible input**, including a planted random blob. A gate that always passes. Recalibrated to 7.05, the control fires and the real buffer yields one genuine site. A second version then silently dropped the planted site through faulty de-duplication. Both were caught by running the positive control **before** believing the negative. Correction*

14.)

10c. The v4 marker, and a 128-byte blob closed by exclusion

The marker at 0x11a1 is plaintext ASCII, length-prefixed by 0x0b = 11 at 0x11a3, preceded by 17 zero bytes:

```
0x11a1:  02 2c 30 65 23 6f 32 30 33 6b 6a 6c 2c 21 13
          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ "e#o203kjl,!" at 0x11a4, 11 bytes
```

A 256-key single-byte XOR sweep returns the same single hit at key 0x00 and nothing else.

The 128-byte blob near 0x11bc is closed by exclusion, and how it was closed matters more than the conclusion.

A first reading took the blob as opening e2 00 00 00, a little-endian length dword of 226. **That was a misalignment** — the 0xe2 is the last byte of the record starting at 0x11a0, and the three zero bytes after it are that record's terminator. **There is no length field.** (*Correction 15.*)

Read at its natural boundary the candidate is **even** and **divisible by three**. An RSA modulus is the product of two odd primes; it is necessarily odd. Neither whole-buffer reading is a modulus.

Sweeping all 25 start offsets in 0x11b0–0x11c8 across both endiannesses — roughly 38 combinations — **exactly one** reading (little-endian at 0x11b9) is odd, 1024-bit and free of small factors.

One survivor in ~38 trials is what chance alone produces. A random odd integer avoids every prime below 48 roughly 20% of the time. Selecting the single alignment that passes is precisely the multiple-comparisons error dissected in Finding 12c. Contrast the v4 site at 0x13d8, which needed no alignment search at all.

Recorded status: *a 128-byte key-shaped blob in the v3 key position, byte order and framing unresolved.* [unestablished] — **not upgraded, not dropped, and excluded from the IOC tables.** A better-scoped hypothesis exists and is explicitly **not** a finding: it is exactly v3-modulus width, high-entropy, and sits directly before the URL pack at 0x1240 that it would authenticate — a *signature* over that URL list fits the position and carries no parity constraint. Testing it requires the published v3

public key, which Finding 10d shows is absent.

10d. v3 is absent from all five layers; v2 was never located

The published v3 key (128 bytes, 1024-bit) was searched **as-published and byte-reversed** across all five layers — `sample.bin`, `payload_unpacked.bin`, `plain.bin`, the carved infector DLL and `embedded_upx.bin` — for **ten searches, zero hits**. The positive control is Finding 10a itself: the identical method applied to the v4 bytes produces an exact hit.

This is a v4-only binary. The v2 modulus was not located at any layer.

11. The download surface

11a. Exactly four user-mode HTTP functions

Carved from payload 0x21894 to `reimpl/netsurface/wininet_names.bin`, 88 bytes, sha256 471cfe...35a6c6e4:

```
WININET.DLL
InternetOpenA · InternetReadFile · InternetOpenUrlA · InternetCloseHandle
```

That is the complete user-mode network surface of the payload. **Download-only:** no listener, no bind, no accept, no socket API of any kind, no datagram receive, no peer-address structure.

The name block's neighbours are informative: `ipfltdrv.sys` and `IPFILTERDRIVER` alongside `OpenSCManagerA / ControlService / DeleteService` — the service-control surface for disabling the legitimate IP filter driver — and `WNetOpenEnumA / WNetEnumResourceA` for share enumeration.

These four functions do not appear in any static import table. The payload's static imports are `KERNEL32`, `USER32`, `ADVAPI32` and `SHELL32`, with **no WININET**. `WinINet` is in the runtime-loaded name block at 0x21600–0x218f0. (An earlier note presented a single DLL set — "`KERNEL32, USER32, GdiPlus, NTDLL, ADVAPI32, WININET, ipfltdrv.sys`" — which was a *union of two different mechanisms presented as one*. Correction 16.)

11b. home.gif is a transport, not an image

Four URLs, two format strings and a user agent reconstruct the request:

```
GET /home.gif?<hex>=<dec>&<hex>=<dec>  
User-Agent: Mozilla/4.0 (compatible; MSIE 7.0b; Windows NT 6.0)
```

from `?%x=%d` and `&%x=%d` at payload `0x216b0`, `rnd=` at `0x21738`, and the user agent at `0x21120`.

The .gif extension is camouflage and this is demonstrable rather than assumed.

GIF8, GIF87a and GIF89a magic occur **zero times** in all 741,648 bytes. There is no LZW decoder, no `OleLoadPicture`, no `IStream`, no `CreateDIBSection`. The single BM hit in the payload is inside the string `TBMON.` in the AV kill list. `GdiPlus.dll` at `0x21654` is present but sits in the *infection-target* string cluster beside `TEXT`, `CODE` and `UPX` — it is a name the infector looks for, not an imaging capability it uses.

A connectivity probe target `www.microsoft.com` sits in the same format-and-probe block at `0x21660`, carved as `fmt_and_probe.bin` (108 bytes, sha256 `88b441...916cd90`) alongside `DEVICEMB`, `.log`, `http://`, `\Run`, `ipfltdrv.sys` and `SYSTEM.INI`.

12. Adjudicated false positives and tool failures

Recorded as findings because each produced a plausible, confident, wrong number, and because the reasoning that killed them is worth as much as the findings that survived.

12a. The anti-debug scan: 1,711 hits, 1,701 of them noise

The sample is PE32/i386 (machine `0x014c`, optional magic `0x10b`), read directly from the PE header bytes.

```
1711 total: 1701 candidate, 10 confirmed  
600 "NtGlobalFlag mask test" -- 600/600 with "no NtGlobalFlag read tracked nearby"  
1103 "PEB->field offsets" -- 1103/1103 with "not tracked to a PEB load"  
10 confirmed
```

The candidate tier is **99.4% of the output and contributes nothing**.

False-positive class 1 — 600 hits, all false. `cmp <reg>, 0x10` where the operand is a stack local:

```
0x004010fc 8b45ec mov eax, dword [ebp - 0x14]  
0x004010ff 83f810 cmp eax, 0x10  
0x00401102 722f jb 0x401133
```

```
0x0040110e 3d001000 cmp eax, 0x1000
```

This is the MSVC small-string-optimization / allocator capacity idiom — 15 characters inline, 16 or more on the heap — reinforced by the 0x1000 test two instructions later. Twelve hits were sampled and **every one** sources its cmp operand from [ebp±N], never from a PEB read. 0x10 is among the most common immediates in compiled code.

False-positive class 2 — 1,103 hits, from two real defects in the scanner.

- **A genuine defect: writes are reported as reads.** The block is commented "a read at a known PEB displacement" but iterates every operand and matches any memory reference at the displacement, **never checking source against destination**. Measured over the first 200 PEB-offset candidates, **113 (57%) are memory writes** — e.g. mov dword [edi+0x10], ebx at 0x402b21, mov dword [ebx+0x10], edi at 0x402b94. A write to a struct field cannot be a PEB probe. The confirmed path destructures destination and source correctly; the candidate path does not. +0x10 and +0x18 are ordinary struct offsets in 32-bit code.
- **A cosmetic defect: x64 register names on a 32-bit binary.** All 1,103 details print x64 names ([rsi+0x10], [rcx+0x10]) for a PE32. Capstone is correctly in CS_MODE_32; the detail string prints the _full_reg()-normalized name. Misleading in a report, not a detection error.

The ten that survive belong to the host, and are not asserted to be malicious:

0x43f948	IsDebuggerPresent	reads PEB->BeingDebugged
0x43f957	OutputDebugStringW	exception probe (Win10+)
0x441c5c	IsDebuggerPresent	reads PEB->BeingDebugged
0x4463eb	IsDebuggerPresent	reads PEB->BeingDebugged
0x454241	PEB self-reference	fs:[0x30] -> reg
0x454247	PEB->NtGlobalFlag	[reg+0x68], reg from fs:[0x30] at 0x454241
0x45e93b	PEB self-reference	fs:[0x30] -> reg
0x45e947	PEB->ProcessParameters	[reg+0x10], reg from fs:[0x30] at 0x45e93b

MSVC and MFC runtimes and installer stubs read BeingDebugged and NtGlobalFlag benignly. These sit in the Oracle host's code, not the viral body, and are recorded as **not yet adjudicated as malicious**.

Audit a classifier by its output counts, not its verdict. *A scan reporting 1,711 hits and a scan reporting 10 are the same scan; only the second one tells you anything.*

12b. The hash-resolver sweep: a false-positive bloom, then a void run

Run 1 was a bloom. Additive hash constructions produced 29 Winsock and 9 CryptoAPI "hits", including CryptVerifySignature. The controls exposed it: **random fake API names were "found" 16 times out of 26**, while real Winsock names scored 22 of 26. Distinct little-endian dwords falling in the output range [0x150, 0x900] numbered **1,125 of 1,968 = 57.2%** of the search space. The arithmetic demonstrates why:

```
sum("htons")           = 0x22c == sum("ntohs")           = 0x22c   (anagrams)
sum("gethostbyname") = 0x57a == sum("GetProcAddress") = 0x57a   (unrelated)
```

Run 2's positive control returned zero, so that run is void. Sixteen APIs confirmed present as plaintext were not found by it, including LoadLibraryA (plaintext at 0xb4177), GetProcAddress (0x22ce4), VirtualAlloc (0xb42da), CreateFileMappingA (0xb402f), SfcIsFileProtected (0x21700) and InternetOpenA (0x218a0).

The correct answer is nonetheless zero — because this malware does not resolve APIs by hash at all (Finding 4b). **That is the third instance in this engagement of a clean, plausible zero produced by a scan that could not fire.**

12c. IPFIL + RDRI — pareidolia, withdrawn

At plain.bin 0x557c, inside the compressed band:

```
e2 'NOT' ce 37 72 af 'IPFIL' b9 3a e1 15 32 'RDRI' 8f 43
```

Three short letter runs in high-entropy data. **TE and VE are absent and the spacing does not fit IPFILTERDRIVER.** The chance expectation for a specific 5-uppercase run in 61,160 bytes is ~0.01 — **but this was not a blind search.** The string was scanned for across every offset, so the multiple-comparisons correction is large and unquantified. A corruption theory and one full re-run were built on it before it was withdrawn. (*Correction 17.*)

12d. "82% of printable dwords in 0x4c00–0x5700 are corrupt" — withdrawn

The statistic is real and meaningless. Chi-square over 1 KB windows shows 0x4000–0x0e000 flat at 350–1,100 (uniform, incompressible) while the rest of the file runs 10,000–260,000. That band **is** the UPX-compressed body. A correct decrypt of

compressed data is indistinguishable from noise, and "is this printable text?" has ~50% printability whether the decrypt is perfect or garbage — **no discriminating power at all**. The discriminator was chi-square. A dependent claim, "the keystream is wrong for a subset of stores," rested entirely on this and on 12c and is withdrawn with them. (*Correction 18.*)

**12e. Under-sampled loop measurements:
two confident, contradictory, wrong**

CAPTURE WINDOW	RESULT	READING
40 store events	ebx oscillates between 2 addresses, ebp frozen	"decrypts nothing" — wrong
4,000 events	126 dwords over ~500 bytes	"covers < 1% of ciphertext" — wrong , and was written into a draft as a retraction
40,000 events	1,250 distinct dwords over 5,000 bytes, 32 hits per address for all 1,250, monotonic first-appearance, 39,999 distinct keystream values of 40,000	correct ; scales linearly and closes with the static bound

A short capture of a nested loop samples the inner level and mimics a stuck outer one.

The single keystream collision at $n = 40,000$ is the expected birthday coincidence, not a short period — a genuinely repeating keystream would show collisions in the thousands, and the index-of-coincidence test had already excluded periods to 512.

A related instrumentation defect: an interpreter started at stage 2's entry (0x4b7630) instead of the real entry point had no stage-1 stack, so ebx loaded as the unmapped 0x10e6 and every store landed outside the address range the write-filter recorded. **The filter reported "0 writes" and the loop looked inert.** Run from the real entry point, ebx resolves to 0x4b8716. (*Correction 19 — and the one that produced this engagement's largest single error.*)

12f. A flags-model disagreement that changes nothing

Two interpreter models disagreed on zf at 0x4b83de: one measured zf=1 in 64 of 3,000 rounds, the other 0 of 7,527. **Neither is incorrect.** lea, push and pop do not write flags, so the last flag-setter before jne 0x4b83e7 is imul esi, ebp, 0x6e8688f8

— and **x86 leaves ZF, SF and PF undefined after imul**; only CF and OF are defined. One model derived ZF from the product; the other left it untouched.

The paths do differ (ZF=0: dec edi; test eax; inc edi, leaving edi unchanged; ZF=1: inc edi), but **the decrypt reads esi**, which is fixed by the push edi/pop esi pair *before* the branch. The differing edi is dead. Confirmed empirically: esi == edx in **7,527 of 7,527**.

12g. The regex IOC extractor found nothing malicious, and that is the finding

The pipeline's own 08_iocs.txt carries a footer saying "Review in context — regex IOCs include false positives." Every one of its outputs was adjudicated:

All 32 URLs, all 17 domains and all 17 file paths are legitimate Oracle, Java, DigiCert, W3C, Adobe or Microsoft infrastructure, or Oracle's own build paths (c:\jenkins\workspace\8-2-build-windows-x64-cygwin-sans-NAS\jdk8u401\683\...). Specific adjudications:

- **15.4.4.21 is not an IP address.** It is the fragment #x15.4.4.21 from the URL http://es5.github.io/#x15.4.4.21 — an ECMAScript specification section number.
- Global_MSIExecute is the standard Windows Installer mutex.
- A:\:z:, W:\:g:r:, T:\:d:h:p:x:, X:\:l:p:t:x:, T:\:d:x: are **getopt option strings**, not paths.
- The protocol-marker entries are GUID fragments, locale identifiers (az-AZ-Latn, sr-SP-Cyrl) and api-ms-win apiset names.

Not one indicator in this file's plaintext is malicious. Every genuine indicator in this engagement came from L2 and L4 — from bytes that did not exist as plaintext when that extractor ran. A triage process that stopped at step 8 would have found a clean, signed Oracle binary.

13. Two string-derived readings, reversed

13a. .adata is not the infector's section marker

The string .adata at payload 0x22cc4 had twice been glossed in this engagement's own notes as "the section name Salty writes into its hosts." **Withdrawn.** Dumping its neighbours settles it:

0x22c98	DWEBLLIO	0x22cb4	_Rar
0x22ca4	DWEBIO	0x22cc4	.adata
0x22ccc	CreateFileW	0x22cd8	CreateFileA
0x22ce4	GetProcAddress		
0x22cf4	OpenFile	0x22d00	_lopen

It is one entry in a list bracketed by **AV-product tokens** (DWEBIO is Dr.Web I/O) and **file-open API names** — a hook or exclusion list. A name inside a list means what the list means.

Code references to 0x22cc4 were searched under **three addressing models** — absolute VA at base 0x69b000, raw file offset as an immediate, and r2 cross-reference extraction — yielding **zero in all three**, against a positive control of **3,420** cross-references extracted from the same region.

The infection marker, if one exists, is now unidentified. Finding 9 traced the parser, the walker and the generator and found no marker write. (*Correction 20.*)

13b. All 110 cross-references into the string block are decode artefacts

r2 reports 110 cross-references landing inside the payload's string block. **All 110 were checked and all 110 are fake.** The giveaways are structural: the cross-reference's *origin* is itself inside the string data, and the target lands mid-string rather than on a first byte.

The clearest example: the single cross-reference to SafeBoot at 0x69bfa9 is a jbe originating at 0x69bf34 — **inside the string blob itself**. Most others target mid-string offsets such as 'ProcAddress', 'lper Objects', 'ice'.

This matters for every zero-cross-reference negative in this report. In a region that addresses its data through unrelocated placeholders, a zero-xref result can neither confirm nor refute — which is exactly why Finding 13a required *three* addressing models and a firing control before withdrawing anything, and why Negative Finding §N8 leaves worm spread **unresolved rather than withdrawn**.

Behavioral and Memory Analysis

Not performed. This pipeline is static-only and never executes a sample or any stage of it.

This heading is retained deliberately. Deleting it would make a methodological

boundary look like an oversight, and a reader comparing this report against a conventional template would otherwise assume the work was done and omitted. A public sandbox run of this exact sample exists and its identity was verified by hash; its findings appear in External Intelligence Corroboration as [intel], never here.

External Intelligence Corroboration

This section is emitted because two of its trigger conditions hold: published vendor reporting on this family was consulted, and a public sandbox report exists for a sample whose identity was confirmed by hash against the local copy.

Everything in this section is tagged [intel] and none of it is migrated into any [static] claim elsewhere in this report. That separation is enforced by `gate/check_provenance.py`, and it is not a formality: this engagement's own corrections log contains an instance (Correction 21, below) where a technique described in vendor reporting was applied to the sample's bytes and produced a confident, wrong answer.

What corroborates, and what it is worth

CLAIM	LOCAL BASIS	EXTERNAL BASIS	READING
Family identification	The driver's own blocklist names a removal tool for this family and a sample-upload page — Finding 7d	Published family description	The local basis is sufficient on its own ; the external basis adds a name, not a fact.
v4 public key	256 bytes recovered from payload 0x13d8, big-endian, 2048-bit — Finding 10a	A published family key whose last five bytes and offset match	Genuine independent corroboration. The local recovery preceded the comparison.
Version generation	Single 2048-bit key present, published v3 key absent from all five layers under ten searches — Finding 10d	Published version history places 2048-bit keys in the later generation	Consistent. The local negative is the stronger statement.
Kernel driver device name	\Device\amsint32, \DosDevices\amsint32, device type 0x15 — Finding 7a	Published descriptions of this family's driver component	Consistent, and locally sufficient.
Polymorphic engine	549-byte generator at 30 call sites emitting instruction bytes — Finding 9d	Banner string names an engine	The banner is in-sample text and was treated as a hypothesis until the code was traced.
Signature-verified tasking	Not established locally. The key has no consumer in any of the four programs — Finding 10b	Published protocol description	[intel] only. This report does not assert it.
Sandbox behavioural observations	None. Static-only pipeline	A public sandbox report for a hash-matched sample	[intel] only , and deliberately not merged into the Technical Analysis.

Correction 21 — a vendor technique applied to the wrong layer

Published analysis of this family describes an RC4 step in one of its stages. That description was applied to this sample's bytes and produced a decode that looked plausible for several passes before it was withdrawn: **the RC4 step described**

belongs to a different stage of the family's chain than the one it was applied to here, and this sample's own body cipher — traced at the single memory-writing instruction in the region, Finding 3a — is a 32-bit subtractive stream cipher with no RC4 key schedule anywhere in it.

The general form is worth stating because it is the reason this section exists as a quarantine rather than as corroboration:

Published reporting describes a family across builds. A sample is one build. *A technique correctly reported for the family can be entirely absent from the specimen in front of you, and applying it anyway produces a decode that fails silently rather than loudly.*

VirusTotal

The public multi-scanner verdict for this sample's hash was consulted. It reports the file as **not signed** — which, as Finding 1d explains, is an independent implementation reaching the same structural conclusion as the local signify check, for the same reason: the SECURITY directory does not point at the certificate.

No sample, hash, domain or IP from this engagement was submitted to any online service by this analysis. No DNS, HTTP or WHOIS lookup was performed against any recovered domain or address at any point.

Indicators of Compromise

All values are **defanged**. Every row carries a Provenance entry; [derived] rows state the mechanism that produced the value, per `gate/check_provenance.py`.

Read the Notes column before deploying any of these. Several entries are diagnostic-only, and two clusters that a naive extractor would emit are excluded by decision and documented at the end of this section.

Every row in this section is tagged [derived], and not one is [static]. That is not a labelling quirk — it is Finding 12g restated as a property of the whole table. `gate/check_provenance.py` defines [static] as a value appearing literally in the sample's own raw pipeline output (strings, FLOSS, rabin2, the regex IOC extractor). **Zero of the 44 indicators below meet that bar.** Every one had to be decrypted, decompressed, carved or computed first. The single L0 plaintext hit for any value resembling an indicator is `microsoft.com`, which belongs to the benign Oracle updater.

A triage process that reads only the plaintext of this file finds a correctly signed Oracle binary and stops.

File hashes

VALUE	TYPE	PROVENANCE	NOTES
f073b23d47e614 7dfef23e171704 52edc57f09044f e64a872025c40c e6c70d59	SHA-256	[derived] — computed: SHA-256 digest over all 820,312 bytes of sample.bin	L0. The infected host as received.
583ff7c2add8b2 b069d2fb0cdc14 699aa4d9db58	SHA-1	[derived] — computed: SHA-1 digest over sample.bin	L0.
d2fce530662f19 987e445febd00a 3223	MD5	[derived] — computed: MD5 digest over sample.bin	L0.
07f7f19d80f9b4 758f7ad785ca48 a6c903151cbb52 f0ec14394938bf 430414d6	SHA-256	[derived] — output of the re- implemented dword-SUB decryptor over 0xb5c00– 0xc5c00, pinned by regen_plain_body.sh	L2, 61,160 B. Reproducible; not an on-disk artefact of an infection.
d371b3f2b1a4b2 b5d5f4c2a0e1b7 c8d9e0f1a2b3c4 d5e6f708192a3b 4c5d6bd05	SHA-256	[derived] — carve of plain.bin 0x34ec, length 0xb400, extent from the embedded PE's own section table	L3, 46,080 B, UPX-packed.
7ae591...cd9ae0	SHA-256	[derived] — computed over the output of NRV2B decompression + E8/E9 unfilter, decoded from the stub at L3 offset 0xae0	L4, 741,648 B. Length matches the stub's own p_filesize exactly.
d1ac0e...c1adb59	SHA-256	[derived] — MZ-header carve at L4 0x01ab4	Infector DLL. Skeleton: 1,285 of 121,856 declared bytes present (1.1%). Not analysable as a program.
d34b4e... 76a3035a	SHA-256	[derived] — MZ-header carve at L4 0x1fcc4, 4,672 B	The kernel driver. The most detection-relevant embedded artefact.

VALUE	TYPE	PROVENANCE	NOTES
3cdf6621f8f30c 840c185b90ae2c 5c56dd49613720 3d00e2de82a626 aa2aa392	SHA-256	[derived] — parsed out of the DER PKCS#7 blob at raw offset 0xc5c00 (messageDigest attribute)	The pristine Oracle binary's digest. Unreachable by any verifier because the SECURITY directory is stale. Useful as an allow-list anchor, never as a malware indicator.

Network indicators

Every URL and host below is **defanged** and **was never contacted**.

VALUE	TYPE	PROVENANCE	NOTES
hxxp://89[.]119[.]67[.]154/testo5/	URL	[derived] — NRV2B-decoded L4, URL pack at offset 0x1240, count byte 04	Download endpoint.
89[.]119[.]67[.]154	IPv4	[derived] — extracted from the decoded URL above	
hxxp://kukutrustnet777[.]info/home[.]gif	URL	[derived] — NRV2B-decoded L4, pack at offset 0x1240	
hxxp://kukutrustnet888[.]info/home[.]gif	URL	[derived] — NRV2B-decoded L4, pack at offset 0x1240	
hxxp://kukutrustnet987[.]info/home[.]gif	URL	[derived] — NRV2B-decoded L4, pack at offset 0x1240	
hxxp://kukutrustnet777888[.]info/	URL	[derived] — NRV2B-decoded L4, offset 0x212bc	Outside the counted pack.
hxxp://www[.]klkjwre9fqwieluoi[.]info/	URL	[derived] — NRV2B-decoded L4, offset 0x2123c	Outside the counted pack.
hxxp://padrup[.]com/sobaka[.]aspx	URL	[derived] — dword-SUB decrypt of L0 0xb5c00-0xc5c00, L2 offset 0x31fa, pack count 02	Present in the loader body, one layer above.

VALUE	TYPE	PROVENANCE	NOTES
hxxp:// kuku1[.]infoier	URL	[derived] — dword-SUB decrypt, L2 offset 0x3218, NUL-terminated	Recorded as stored bytes; the reading is not asserted. ier is not a valid TLD. See Finding 6d.
hxxp:// www[.]microsoft[.]com	URL	[derived] — NRV2B-decoded L4, offset 0x21660	Connectivity probe, not C2. Do not alert on it.
/home[.]gif? <hex>=<dec>&<hex> =<dec>	URI pattern	[derived] — composed from the four URLs plus ?%x=%d and &%x=%d at L4 0x216b0	The request shape. A high-value detection surface — see Detection Opportunities.
Mozilla/4.0 (compatible; MSIE 7.0b; Windows NT 6.0)	User- Agent	[derived] — NRV2B-decoded L4, offset 0x21120	MSIE 7.0b is a beta token; rare in genuine traffic.
TCP port 25 outbound, blocked at the kernel	Behaviour	[derived] — carved driver disassembly, filter at offset 0x108f0; 0x19 compared against both source and destination port	Inverted indicator: outbound SMTP silently failing host-wide is the symptom.

Host artefacts

VALUE	TYPE	PROVENANCE	NOTES
Ap1mutex7	Mutex	[derived] — dword-SUB decrypt, L2 offset 0x679; CreateMutexA call site at L2 0x22f	Traced to its CreateMutexA call site. The strongest single host indicator in the sample.
purity_control_7728	Section name	[derived] — dword-SUB decrypt, L2 offset 0x64c; used at L2 0x12c	Named pagefile-backed section, 0x15400 bytes.
hh8geqpHJTkdns6	Section name	[derived] — dword-SUB decrypt, L2 offset 0x660; used at L2 0x112	Named pagefile-backed section, 0x8000 bytes.
\Device\amsint32	Device	[derived] — NRV2B-decoded L4, offset 0x20a28, UTF-16	Created by DriverEntry.
\DosDevices\amsint32	Symlink	[derived] — NRV2B-decoded L4, offset 0x20a4c, UTF-16	
\Device\IPFILTERDRIVER	Device	[derived] — NRV2B-decoded L4, offset 0x206d4, UTF-16	Impersonates the legitimate Windows IP filter. Presence <i>plus</i> device type 0x15 from a non-Microsoft driver is the discriminator; the name alone is not.
amsint32	Service/driver name	[derived] — common substring of the two decoded device names at L4 0x20a28 and 0x20a4c	
Ddk (0x206b6444)	Pool tag	[derived] — carved driver disassembly, ExAllocatePoolWithTag argument	Diagnostic; a pool tag alone is weak.
uxJLpe1m	String	[derived] — dword-SUB decrypt, L2 offset 0x670	No traced code reference. Diagnostic only.
_Rar	String	[derived] — dword-SUB decrypt at L2 offset 0x647; NRV2B-decoded L4 offset 0x22cb4	Appears in both layers. No traced reference.

Signature and structural indicators

These are the highest-value detection surface in the report, because they hold for **every host this infector touches**, not only for this one file.

VALUE	TYPE	PROVENANCE	NOTES
SECURITY directory offset ≠ actual WIN_CERT offset, sizes equal	Structural	[derived] — computed from parsed PE data directories vs. the WIN_CERT located at raw offset 0xc5c00; Finding 1c	Generic across all hosts infected by this builder. Delta equals the insert size.
Section named .reloc with Characteristics 0xE2000040 (+X +W)	Structural	[derived] — Characteristics dword read from the section table at raw offset 0x2e4; Finding 1b	A genuine .reloc is 0x42000040.
Section named .text with Characteristics 0xE0000020 (+W)	Structural	[derived] — Characteristics dword read from the section table at raw offset 0x244; Finding 1b	MSVC does not emit a writable .text.
DllCharacteristics = 0x8140 (DYNAMIC_BASE) with BASERELOC rva/size = 0	Structural	[derived] — computed from the optional-header field at raw offset 0x15e vs. data directory 5 at 0x1c8, both parsed by name; Finding 1d	ASLR asserted without relocations — no linker produces this.
Entry point inside .text preceded by int3 padding, followed by a push/ret transfer to a computed address	Structural	[derived] — disassembly of the 330-byte stub at raw 0x40b96–0x40cdf; Findings 1f, 2a	
jmp over the two bytes 0f cd	Byte pattern	[derived] — decoded at VA 0x44179f; the encoded bytes eb 02 0f cd are literal in the sample. Finding 2a	Linear-sweep desynchronizer.

VALUE	TYPE	PROVENANCE	NOTES
81 f1 92 0e 00 00 twice, adjacent	Byte pattern	[derived] — located at raw offset 0xb59c3 by filtering the decoded region for memory writes; VAs 0x4b83c3/0x4b83c9. The 12 bytes are literal in sample.bin. Finding 3b	Self-cancelling XOR pair. Distinctive , but polymorphic per-build — see the caveat below.
imul eax, eax, 0x15a4e35 (encoded 69 c0 35 4e 5a 01)	Byte pattern	[derived] — NRV2B-decoded L4 island, 123 call sites; Finding 9d	Borland LCG constant. Implementation detail, not attribution.
mov dword [imm32], 0x29a (opcode c7 05)	Byte pattern	[derived] — NRV2B-decoded L4, VA 0x6a2b29; Finding 9f	The 666 marker written to the side file.
cmp ecx, 0x10b gating a PE parse (encoded 81 f9 0b 01 00 00)	Byte pattern	[derived] — NRV2B-decoded L4, VA 0x6a2676 inside fcn.006a2603; Finding 9a	32-bit-only target check.
XOR-0xf9 blocklist band, 284 B, entries reversed and NUL-terminated in ciphertext	Structural	[derived] — NRV2B-decoded L4 0x2088c–0x209a8, then XOR 0xf9 per entry; Finding 7d	
Driver: IoCreateDevice with device type 0x15 and imports only ntoskrnl.exe	Structural	[derived] — carved driver (L4 0x1fcc4), import table parsed and DriverEntry at RVA 0x62f disassembled; Finding 7a	The signature the four user-mode methods could not see.

A caveat that limits the byte patterns above, and it is important. Finding 9d establishes that this family carries a **polymorphic code generator** that emits instruction bytes on probability thresholds from an LCG. The decryptor prologue and the entry stub are therefore expected to differ per infection. **Byte patterns drawn from those regions are specimen-scoped, not family-scoped**, and are marked as such. The *structural* indicators above — permissions, the stale certificate pointer, the empty BASERELOC directory, the device type — are the ones that generalize, because they are consequences of the infection mechanism rather than of any particular emitted byte sequence.

Excluded by decision

Two clusters that an automated extractor would emit are deliberately **not** IOCs here.

The six resolved API addresses in L2 are excluded. 0x777f2ee0, 0x777ef3c0, 0x777f0f00, 0x774a7fc0, 0x774a4ae0 and the 0x777d0000 module base are **artefacts of whatever machine was infected when this body was captured** (Finding 6c). They identify a Windows build and a load order, not this malware. Alerting on them would be alerting on a stranger's DLL layout.

The 23 blocklist entries are excluded as network IOCs. kaspersky, symantec., virustotal. and the rest are **substrings the malware blocks**, not infrastructure it contacts. They belong in a YARA rule against the encrypted band and in the Detection Opportunities section, and they would be actively harmful in a network blocklist.

Adjudicated false positives — do not alert

Every one of these was emitted by the pipeline's regex extractor and adjudicated to legitimate (Finding 12g):

- **15.4.4.21 is not an IP address.** It is a fragment of `hxxp://es5[.]github[.]io/#x15.4.4.21` — an ECMAScript specification section number.
- **All 32 URLs and 17 domains in 08_iocs.txt** are Oracle, Java, DigiCert, W3C, Adobe or Microsoft infrastructure.
- **Global_MSIExecute** is the standard Windows Installer mutex.
- **A:\:z:, W:\:g:r:, T:\:d:h:p:x:, X:\:l:p:t:x:, T:\:d:x:** are getopt option strings, not file paths.
- **The 17 file paths** are Oracle's own build paths, e.g. `c:\jenkins\workspace\8-2-build-windows-x64-cygwin-sans-NAS\jdk8u401\...`
- **GdiPlus.dll** at L4 0x21654 is a name the *infector searches for*, not an imaging capability it uses (Finding 11b).
- **.adata** is an entry in an AV file-open hook list, **not** the infector's section marker (Finding 13a).
- **SALITY at L4 0x2298c** sits inside the sample's own AV kill list — it is a string the malware *searches for*. The family determination rests on `salicy-remov` and `upload_virus` in the driver blocklist instead (Finding 7d).
- **The 11 WinINet and 4 CRYPT32 imports in L0's static import table** belong to the benign Oracle updater and are cited as malware capability nowhere in this

report (Finding 1a).

Negative Findings

Each negative below states the **scope it was searched under**. That is not boilerplate. This engagement's single largest error was a convergent negative held for most of its duration whose four methods shared one unstated assumption, and the correction is Finding 7f.

§N1 — No inbound network capability in any of the four programs. *Searched:* static import tables of L0, L3, L4 and each of the three carved embedded PEs; the runtime-resolved name block at L4 0x21600–0x218f0; ASCII and UTF-16LE string scans across all five layers for bind, listen, accept, WSAAccept, recvfrom, WSARcvFrom, getpeername, sockaddr, WSAShutdown, ws2_32, wsock32. *Result:* zero.

Positive control: the same method finds all four WinINet names at L4 0x218a0–0x218d8. *Scope limit:* user-mode surfaces plus the kernel driver's full 26-import list.

The driver's traffic handling is a filter attached to the existing stack — it receives IRPs, it does not accept connections. Its dispatch was transcribed (Finding 7c) and contains no listener. *Conclusion:* **High confidence.** No peer-to-peer capability is present in this specimen.

§N2 — No cryptographic verification routine anywhere in the sample. *Searched:* crypto imports across all four programs (zero); and — decisively — instruction-level counts over the code island's full 24,530-instruction disassembly: mul = 0, shld/shrd = 0, mul→adc chains = 0. *Positive control:* the same disassembly yields imul = 210, adc = 10, sbb = 15, and the MD5/CRC32 constant scan of Finding 8e fires. *Scope limit:* the code island and the four carved programs. **A downloaded stage is out of scope by construction.** *Conclusion:* **High confidence.** Multi-precision modular exponentiation is arithmetically absent. The recovered v4 key has no consumer on this disk.

§N3 — No further encrypted stage below L4. *Searched:* chi-square per 4 KB window across all 741,648 payload bytes. *Result:* no region with uniform-random statistics remains after the identified structures are accounted for. Structural coverage of every layer is ~100%. *Positive control:* the same measure identifies the L0 encrypted band at 0xb5c00–0xc5c00 (215–283 vs ~255) and the L2 compressed band at 0x4000–0xe000 (350–1,100). *Scope limit:* **entropy was explicitly not used** — see Finding 1e. Chi-square detects uniform-random data; it would not flag a stage encrypted with a scheme that preserves byte statistics, nor one stored as plausible-looking structured data. *Conclusion:* **High confidence** for uniform-random-statistics stages. **Moderate** as a general statement.

§N4 — No hash-based API resolution. *Searched:* 12–15 hash constructions (`ror13`, `ror13+null`, `ror7`, `rol5^`, `rol7^`, `djb2`, `djb2^`, `sdbm`, `fnv1a`, `fnv1`, `crc32`, `crc32+null`, and additive variants) over 26 Winsock and 12 CryptoAPI names, across all layers. *Result:* zero. *Positive control:* **failed in run 2** — 16 of 16 known-present APIs were not found, so that run is void (Finding 12b). Run 1's control also failed, in the opposite direction: 16 of 26 *random fake* names were "found". *Why the answer is nonetheless zero:* Finding 4b establishes the resolver directly. It is a **literal repe cmpsb string comparison** at L2 0x2f7 with a 150-byte name cap. There is no hash to find. *Conclusion:* **High confidence — but resting on the positive identification of the resolver, not on the sweep.** The sweep is reported here as a failed instrument.

§N5 — v3 and v2 public keys are absent. *Searched:* the published v3 modulus, as-published and byte-reversed, across `sample.bin`, `plain.bin`, `embedded_upx.bin`, `payload_unpacked.bin` and the carved infector DLL — ten searches. *Result:* zero hits. v2 was not located at any layer. *Positive control:* the identical method applied to the v4 bytes produces an exact hit at L4 0x13d8. *Conclusion:* **High confidence.** This is a v4-only binary.

§N6 — No self-modifying code and no additional decryption layer within L2. *Searched:* the 262 recursively-reached instructions for a memory destination with an opcode in `{xor, sub, add, rol, ror, not, neg, shl, shr, adc, sbb}`. *Result:* zero. *Scope limit:* **the recursively-reached instruction set, not a linear sweep.** A linear sweep at every byte offset produces thousands of such hits from mid-instruction decode garbage; using it would have manufactured a finding. Also excluded: the compressed region and everything past `jmp 0x785500`. *Conclusion:* **High confidence** within that scope.

§N7 — No anti-debug checks in the malware layers. *Searched:* segment-register accesses across the 262 reached L2 instructions (exactly four exist: the PEB walk at 0x046 and the three SEH-frame accesses); `antidebug_scan.py` across L0. *Result:* L2 reads none of `PEB+0x02`, `PEB+0x68`, or the heap flags. `GetTickCount` is resolved with **no call site**. The 10 confirmed hits in L0 (Finding 12a) sit in the **benign Oracle host's** code — `IsDebuggerPresent` at 0x43f948, 0x441c5c, 0x4463eb; `NtGlobalFlag` at 0x454247; `ProcessParameters` at 0x45e947 — and are **not adjudicated as malicious**; MSVC and MFC runtimes read those fields benignly. *Scope limit:* keyed on PEB offsets and import names, with **no dataflow**; the scanner's own three disclosed blind spots apply (direct syscall stubs, runtime-resolved imports, non-dataflow); L4 was not swept with this tool. *Conclusion:* **Moderate confidence** that the malware layers perform no anti-debug checks. The confidence is not higher because L4 is

unswept and the scan is not dataflow-based. *Note on counts:* the raw output is **1,711 hits = 1,701 candidate + 10 confirmed**. An earlier working note in this engagement recorded "1,711 hits, 8 confirmed"; **the correct split is 10 confirmed**, reconciled against the tool's own output before publication. The two extra are the paired PEB self-references at 0x454241 and 0x45e93b, which the earlier count folded into their dependent reads.

§N8 — Worm spread is UNRESOLVED, not withdrawn, and not confirmed.

Searched: string evidence exists — autorun.inf at L4 0x21740, [AutoRun], shell\open\command, WNetOpenEnumA/WNetEnumResourceA/WNetCloseEnum, GetLogicalDrives/GetDriveTypeA. **No code reference to any of it was traced.** *Why this is not a negative:* cross-reference extraction in the code island is unreliable in exactly this situation — Finding 13b establishes that **all 110** r2 cross-references landing in the payload's string block are decode artefacts, and the island addresses its data through unrelocated placeholder immediates. **A zero-xref result here can neither confirm nor refute.** *Contrast with a genuine withdrawal:* Finding 13a withdrew the .adata gloss because a *contrary reading* was found — the string's neighbours identify it as an AV hook-list entry. **No contrary reading exists for the autorun cluster.** *Conclusion: Unresolved.* Recorded as Low confidence in the Capabilities table. Resolving it requires reading the six unread callers of fcn.006a2802 and the code around the GetLogicalDrives reference.

§N9 — No credential access, collection, or keylogging capability. *Searched:* import tables and runtime name blocks across all four programs for SetWindowsHookEx, GetAsyncKeyState, GetKeyState, GetClipboardData, CredEnumerate, CryptUnprotectData, LsaOpenPolicy, SamConnect, and browser credential-store paths. *Result:* zero. *Positive control:* the same method over the same name blocks finds OpenProcessToken, AdjustTokenPrivileges and LookupPrivilegeValueA, all present. *Scope limit:* name-based, over the plaintext import-name table at L4 0xb4009–0xb467a and the runtime block at 0x21600–0x218f0. **Not traced to call sites,** and a downloaded stage is out of scope. *Conclusion: Moderate confidence.* This specimen is an infector and a delivery mechanism, not a stealer.

§N10 — The original Oracle code destroyed by the entry-point stub cannot be recovered from this file. *Searched:* the full 330-byte overwritten range 0x40b96–0x40cdf, and all e8 re132 call sites in the image targeting it. *Result:* five original call sites still point into the range (Finding 1f); the code they called is gone. Two methods — a raw-file relative-call scan and r2 axt at aaa depth — agree exactly on the caller set. *Conclusion: High confidence.* Reconstruction requires a donor copy

of Oracle jusched.exe 2.8.401.10. **This is the finding that makes remediation "replace", not "disinfect".**

Assessment Limitations

This assessment is static-only. No stage of this sample was executed at any point, by design. Everything below follows from that, or from an artefact that was not on this disk.

Methodological omissions, declared

- **No dynamic execution.** No sandbox detonation, no debugger, no instrumented run. Behavioural claims in this report are derived from disassembly and from re-implemented decoders running on a private register and memory model that never executes sample code. The `### Behavioral and Memory Analysis` heading is retained above with its "Not performed" body precisely so this boundary is visible rather than inferable.
- **No memory forensics.** No process dump, no injected-region recovery, no live handle or object inspection.
- **No network observation.** No traffic capture, no C2 interaction, no DNS resolution, no WHOIS, no submission to any online service. No recovered URL or IP was contacted at any point in the engagement.
- **No detonation-derived host artefacts.** Registry writes, dropped files, service installation and mutex creation are reported from **code and strings**, and are marked in the Capabilities table with the confidence that warrants — several at Moderate or Low precisely because only the string or the import, not the call site, was established.
- **Detection rules are declared and omitted.** See below.

What is missing because the artefact is missing

- **The downloaded payload was never obtained.** Every URL is defanged and uncontacted. The content of `home.gif` is unknown, and everything downstream of the download is unassessed.
- **The signature-verification stage is not on this disk** (Finding 10b, §N2). The recovered v4 key has no consumer. Any protocol-level claim about signed tasking is `[intel]` and is not asserted.
- **The infector DLL is a skeleton.** 1,285 of 121,856 declared bytes are present (1.1%). It is a header-and-imports shell; **there is no body to analyse.** Its declared section sizes state an intended memory layout, not the existence of bytes.

- **The embedded EXE stub is 35% present** — 179 of 6,280 bytes.
- **Acquiring the missing stage is a sourcing decision, not an analysis step.** It would push sample identifiers off this machine. It is **flagged and was never initiated**.

Unresolved within the available bytes

These are answerable from this sample; they were not reached.

1. **The six unread callers of fcn.006a2802** — 0x6a337b, 0x6aa38b, 0x6aaa01, 0x6aaf4f, 0x6ac297, 0x6ac93d. The mmap → patch → SetFileTime sequence is **not in fcn.006a2802** (Finding 9g, a withdrawal) and is unstepped. These callers are where it would be.
2. **Worm spread** (§N8) — unresolved, blocked on the same callers.
3. **The infection marker** is unidentified. The .adata gloss was withdrawn (Finding 13a) and nothing replaced it.
4. **The IAT slot → API name mapping in the code island is unverified.** An index-based mapping was tested and **failed its control**, and is used for nothing in this report. Every thunk in Finding 9f is cited as a distinct call site, never as a named API.
5. **hxxp://kuku1[.]infoier** — stored bytes recorded, reading not asserted (Finding 6d).
6. **The 128-byte blob at L4 0x11bc** — [unestablished]. Closed by exclusion as a modulus (even, divisible by three); the better-scoped hypothesis (a signature over the adjacent URL pack) requires the published v3 key, which §N5 shows is absent.
7. **The keystream is not recovered in closed form** (Finding 3d). The decrypt is reproducible by re-implementation; the key schedule is not solved.
8. **The ebx write range in the decryptor is data-dependent** and not statically bounded (Finding 3d). The 0x10000 figure bounds the *ciphertext*, not the loop's stores.
9. **Three unreconciled counts of the L4 import-name table** — 105, 84 and 100, measured under different inclusion criteria (Finding 9e). Recorded rather than silently resolved. No finding depends on which is correct.
10. **L4 was not swept with antidebug_scan.py** (§N7).

Instrument limitations that bound specific claims

- **Code-island coverage is 85.4% of range / 99.5% of nonzero bytes**, corroborated by two methods agreeing within 6 bytes. The residual error direction is **undercount, never overcount**. The region has **no PE header and therefore no .pdata/.xdata**, so `seh_roots.py` contributes no root set — the indirect-call floor bounding any "no path exists" claim here is **wider than usual, not narrower**.
- **The L2 residual indirect count is 1** (`jmp 0x785500` at `0x258`). That single unresolved exit is the scope limit on every L2 reachability claim.
- **Cross-reference extraction in the code island is unreliable** — 110 of 110 xrefs into the string block are decode artefacts (Finding 13b). Any zero-xref result in that region is inconclusive by itself, which is why Finding 13a required three addressing models and a firing 3,420-xref control before withdrawing anything.
- **Three silent tool failures were caught during this engagement**, each producing a plausible number rather than an error (Finding 8d). All measurements that depended on them were re-run. There is no way to prove a fourth was not missed.

Declared-but-omitted sections

*None. The Detection Rules section was declared-but-omitted for most of this engagement and is now **emitted**: the rules are in `detections/` and `./registry/check_detections.sh` reports PASS.*

Detection Rules — shipped and validated. Four YARA rule files (8 rules) and three Sigma rules are in `detections/`. Every rule compiles, every YARA rule fires on at least one supplied artefact, and none fires on the benign control. False-positive measurement is in `detections/VALIDATION.md` — **171 benign files scanned, 0 hits**, of which 163 are 32-bit PE32 binaries assembled specifically because the structural rules are gated on `MACHINE_I386` and a 64-bit corpus cannot exercise them at all. That document also records per-clause reachability, so the zeros are measurements rather than artefacts of an inert corpus, and it names the clauses that do **not** discriminate on their own — writable `.text` occurs on 1 of 163 benign files and an empty `BASERELOC` directory on 9, so neither is an alerting condition by itself.

The rules validate against unpacked layers, not only the on-disk file. Passing `check_detections.sh` only `sample.bin` reports FAIL for the three layer-scoped rules, which is the checker being correct and the invocation being under-specified;

VALIDATION.md records the full invocation. Two statements in Detection Opportunities below were re-checked against the sample's own headers while writing the rules — one was wrong and is corrected in place (Correction 28), the other was confirmed and an initial contrary reading of it proved to be a defect in the checking script rather than in the report.

Recommendations

Ordered by priority. Each states the finding it rests on.

1. Replace every infected binary. Do not attempt disinfection. (*Finding 1f, §N10*)

This is the load-bearing recommendation and it overrides any vendor "clean" or "repair" option.

The infector **overwrote 330 bytes of the host's original .text** to place its entry stub. Five original call sites in the host still point into that destroyed range. Removing the 69,632-byte insert restores the file's *size* and *statistics*; it does not restore the code. The result is a binary that loads and then jumps into whatever now occupies those addresses.

- Restore affected executables from known-good media or reinstall the products that own them.
- For this specific host: reinstall Oracle Java rather than repairing jusched.exe.
- If a repaired file must be validated, compare its Authenticode digest against 3cdf6621f8f30c840c185b90ae2c5c56dd496137203d00e2de82a626aa2aa392 — the pristine Oracle digest recovered from inside the signature blob. **A file that does not match that digest is not repaired.**

2. Hunt for the kernel driver, not for the file infection. (*Findings 7a–7f*)

The driver is the component with the longest dwell and the widest blast radius, and it is the component every user-mode method in this engagement failed to find.

- Enumerate `\Device\` for `amsint32` and for **IPFILTERDRIVER created by a driver that is not Microsoft's**. The name alone is not a discriminator — the legitimate Windows filter uses it. **IoCreateDevice device type 0x15 from a non-Microsoft image is.**
- Enumerate loaded kernel modules for images importing `ntoskrnl.exe` **exclusively**, with a Native subsystem and a TimeDateStamp of 2005-12-25.
- Check for `\DosDevices\amsint32`.
- **Do not rely on user-mode network telemetry to find this.** It borrows the OS

TCP/IP stack and has no network imports.

3. Treat "outbound SMTP silently fails" as an indicator, not as a mail problem. (*Finding 7b*)

The driver blocks TCP port 25 in **both directions** at the kernel. On an infected host, mail simply stops working with no error a user-mode tool can attribute. **A helpdesk ticket about outbound mail failing on one machine is a rootkit indicator.**

Likewise: an infected host **cannot reach 23 security-vendor domains** — kaspersky, symantec., virustotal., sophos., drweb., and the rest — because the driver blocks them by substring match against the request payload. **A machine that cannot reach AV vendor sites while reaching everything else is infected.** This is a far cheaper detection than any signature.

4. Search for hosts, not just for this sample. (*Findings 1b–1d*)

The structural indicators generalize across every file this builder infects, while the byte patterns do not — Finding 9d establishes a polymorphic generator, so the stub and decryptor differ per infection.

Scan the estate for PE files with **any** of:

- a SECURITY directory offset that does not locate a valid WIN_CERT, where the size field nonetheless matches;
- a section named .reloc with Characteristics 0xE2000040 (executable and writable);
- a section named .text with the write bit set (0xE0000020);
- DllCharacteristics asserting DYNAMIC_BASE while the BASERELOC directory is zero;
- an entry point immediately following an int3, in .text, that terminates in push/ret.

Any one of these on a signed binary is high-signal. They cost a header parse, not a scan.

5. Verify signatures properly, and fix

the tooling that does not. (*Finding 1d*)

This sample is a working demonstration that "has a certificate" and "is validly signed" are different questions, and that some tools answer the wrong one. **A verifier must fail closed when the SECURITY directory does not locate a well-formed WIN_CERT** — reporting "unsigned" for a file carrying a physically present, structurally valid Oracle certificate is a materially misleading answer.

Audit whatever performs signature checks in the estate against exactly this case.

6. Assume credential compromise on any host that ran this. (*Capabilities table; §N9 is scoped*)

§N9 finds no stealer capability **in this specimen**, and that negative is name-scoped and does not extend past the download. The sample has SeDebugPrivilege acquisition, kernel-mode process termination, and a download-and-execute channel whose payload was never obtained. **Treat any host that executed this as fully compromised** and rotate credentials that were used on it.

7. Restore AV and firewall configuration by policy, not by inspection. (*Capabilities table*)

The payload carries registry paths for AntiVirusOverride, AntiVirusDisableNotify, FirewallDisableNotify, UpdatesDisableNotify, UacDisableNotify, EnableLUA, DisableTaskMgr, DisableRegistryTools, plus Windows Firewall AuthorizedApplications entries. These are Moderate-confidence, string-and-path scoped findings. **Re-apply policy rather than checking whether each value looks right** — the cost of re-application is negligible and the cost of missing one is not.

8. Next analysis steps, if this specimen is pursued further

In priority order, all answerable from bytes already on disk:

1. Read the six callers of fcn.006a2802 — this resolves both the infecting write and worm spread (§N8, Limitation 1).
 2. Sweep L4 with antidebug_scan.py to close §N7 at full scope.
 3. Write and **validate** the detection rules sketched below, via ./registry/check_detections.sh.
 4. Reconcile the three L4 import-table counts (Limitation 9).
-

5. Solve the keystream schedule, if a closed form is wanted for tooling rather than for the decrypt itself.

Not recommended: attempting to acquire the downloaded stage. That is a sourcing decision with off-machine consequences and belongs to the engagement owner, not to the analysis.

Detection Opportunities

What a rule should key on, with the offsets and constants to key on it with. **These are inputs to rule-writing, not validated rules** — see the omission note under Assessment Limitations.

Highest value: structural, generic across the family

These hold for every host this builder infects and survive the polymorphic engine, because they are consequences of the infection *mechanism*.

SURFACE	TEST	WHY IT GENERALIZES
Stale certificate pointer	SECURITY directory offset does not locate a WIN_CERT whose wRevision is 0x0200 and wCertType is 0x0002, while its size field matches the directory	The infector appends before the certificate and never updates the pointer. Mechanical, not stylistic.
Writable/executable .reloc	Section named .reloc with Characteristics 0xE2000040	It needs the section executable to run from it and writable to decrypt in place.
Writable .text	Section named .text with the IMAGE_SCN_MEM_WRITE bit	Required to overwrite the entry-point region.
ASLR without relocations	DllCharacteristics & 0x0040 set while BASERELOC rva and size are 0	The loader must be prevented from processing relocations, but the data must remain for the infector to read (Finding 9c).
Certificate arithmetic	SECURITY.VA + size != filesize on a PE that still declares a certificate — the pointer is short of EOF by the insert size (0xb7458 vs 0xc8458, a gap of 0x11000)	A precise, cheap, near-zero-false-positive test. Note the equality cert_offset + cert_size == filesize is the clean condition, not the malicious one. (Corrected — see Correction 28.)

Combine two or more of these. Each alone has a plausible benign explanation somewhere in the world; together they do not.

High value: network

SURFACE	TEST
URI shape	GET /home.gif?<hex>=<dec>&<hex>=<dec> — a .gif request carrying a two-parameter hexadecimal-named query string. Key on the shape, not on the four hostnames , which rotate.
User-Agent	Mozilla/4.0 (compatible; MSIE 7.0b; Windows NT 6.0) — the 7.0b beta token is rare in genuine traffic.
Response content-type mismatch	A response to a .gif request whose body does not begin with GIF87a or GIF89a. Finding 11b establishes there is no image handling anywhere in the payload.
Inverted: SMTP	Outbound TCP/25 failing host-wide with no firewall rule to explain it.
Inverted: vendor domains	A host unable to reach AV-vendor domains while reaching everything else. Detect the absence.

High value: host

SURFACE	TEST
Mutex	Ap1mutex7 — the one mutex traced to its CreateMutexA call site (L2 0x22f).
Named sections	purity_control_7728 (0x15400 bytes) and hh8geqpHJTkdns6 (0x8000 bytes), both pagefile-backed. Two named sections of exactly those sizes is a strong pair.
Kernel objects	\Device\amsint32, \DosDevices\amsint32; \Device\IPFILTERDRIVER from a non-Microsoft image.
Driver identity	Native subsystem, imports ntoskrnl.exe only, 26 imports, IoCreateDevice device type 0x15, TimeDateStamp 2005-12-25, pool tag Ddk .
Side file	A file written with the dword 0x0000029a at offset 0, followed by an array of dwords (Finding 9f, stage 7).

Moderate value: byte patterns — specimen-scoped, and say so in the rule

Finding 9d means these will differ per infection. Use them for *this* specimen and for clustering, never as the sole family test.

PATTERN	LOCATION	NOTE
81 f1 92 0e 00 00 81 f1 92 0e 00 00	L0 0xb59c3	The self-cancelling XOR pair. 12 bytes, distinctive.
eb 02 over 0f cd	L0 0x44179f	Linear-sweep desynchronizer.
69 c0 35 4e 5a 01 (imul eax, eax, 0x15a4e35)	L4 island, 123 sites	Borland LCG. Common constant — implementation detail, not attribution.
81 f9 0b 01 00 00 (cmp ecx, 0x10b) adjacent to MZ/PE checks	L4 VA 0x6a2676	The 32-bit-only PE gate.
c7 05 opcode search inside a relocation walk	L4 0x6a269a	The marker constant 0x5c7 compared against word [candidate-2]. Highly distinctive as a <i>combination</i> .

The encrypted blocklist — the single best static rule surface in the sample

The 284-byte band at L4 0x2088c–0x209a8 holds 23 vendor strings under XOR 0xf9, NUL-terminated in the ciphertext.

A YARA rule keying on **three or more** of the XOR-0xf9-encoded forms of kaspersky, symantec., virustotal., sality-remov and upload_virus, **within a 300-byte window**, would be highly specific. Encoding kaspersky under 0xf9 gives 92 98 8a 89 9c 8b 8a 92 80; the others follow the same transform. (*Corrected — see Correction 27.*)

Why this is the best surface: it sits in the payload, so it survives the polymorphic engine entirely; it is semantically load-bearing, so the author cannot drop it without losing the capability; and the null model is established — 400 random 12-byte patterns across all 255 single-byte keys produced **zero** hits (Finding 7d).

Validation requirements before any of this ships

1. Compile every rule. **A rule that does not load is not a rule** — that failure has occurred in this repository before.
2. Match against the sample and confirm the expected hit.
3. Run against a benign corpus. **The structural rules above are the ones most likely to false-positive**, particularly against packers and installers that legitimately produce writable .text.

4. Run against a benign **signed** corpus specifically, for the certificate-arithmetic rules.
5. Record the false-positive rate in the rule itself.

`./registry/check_detections.sh <report-dir> <sample>` performs 1-3.

About this Report

Produced by: an AI-automated static malware analysis pipeline (aima), run under a human-directed engagement. Four analysis agents worked the sample in parallel across the engagement; their findings were merged by **union, never consensus**, with the grounding gate as arbiter.

Method: static only. The sample was never executed, in whole or in part. Every decoder in the chain — the dword-SUB body cipher, NRV2B decompression, the E8/E9 unfilter, the XOR-0xf9 blocklist — was **re-implemented in Python from the sample's own instructions**, and run over a private register and memory model. That choice is not stylistic: it means a wrong model produces a decode failure rather than silent execution of sample code.

Evidence discipline. Tool output is a hypothesis. capa, r2inspect, Ghidra pseudocode and multi-scanner labels generated candidates for this report and appear in none of its findings without independent disassembly verification. Every negative claim states the scope it was searched under and, where one was run, its positive control. Where a control failed, the negative is reported **as a failed instrument** rather than as a result (§N4 is the clearest case).

Confidence vocabulary. ICD-203: *almost certainly / highly likely* (High), *likely / probably* (Moderate), *roughly even chance* (Low), *unlikely / very unlikely*. Stated per capability row and per negative finding rather than once for the report.

Provenance tags. [static] — read from the sample's own bytes. [derived] — produced by a re-implemented decoder or by stated arithmetic over [static] values, with the mechanism named. [intel] — from external reporting, quarantined in one section and never migrated. gate/check_provenance.py enforces the separation.

Reproducibility. opus/r2_evidence/regen_plain_body.sh and opus/pass2_evidence/regen.sh both pin the sample hash and refuse to run on mismatch. Appendix A lists what each artefact is and how to regenerate it.

Handling. This report contains live sample identifiers — hashes, C2 URLs, device and service names. It lives in the engagement's gitignored analysis directory and is not committed to any repository. No sample, hash, domain or IP from this engagement was submitted to any online service. No recovered URL, domain or address was contacted at any point.

Report Changelog

VERSION	DATE	CHANGE
1.0	2026-09-09	Initial structural analysis of the infected host. Insert located and bounded; certificate anomaly identified.
1.1	2026-09-09	Entry-point transfer derived. Two derivations retracted before the third was confirmed at the bytes (Finding 2b).
1.2	2026-09-09	Body decryptor located by filtering for memory writes; cipher identified as 32-bit SUB. Prior byte-scoped keystream negative re-scoped (Finding 3d).
1.3	2026-09-10	L2 fully mapped: 6 functions, 262 instructions, residual indirect count 1. API resolver identified as literal repe cmpsb, voiding the hash-sweep premise.
1.4	2026-09-10	UPX/NRV2B re-implemented from the stub's instructions; L4 recovered at 741,648 bytes, two-sided bound.
1.5	2026-09-10	plain.bin identified as a captured memory image , not a pure static decrypt. Six resolved API addresses reclassified as environment artefacts and excluded from IOCs (Finding 6c).
1.6	2026-09-10	Three embedded PEs carved. Central negative reversed: the network component is a kernel driver (Finding 7f). Blocklist decoded; family established from the sample's own bytes.
1.7	2026-09-10	Code island measured — 90,516 bytes belonging to no embedded PE. Load base recovered by pointer distribution. Three silent tool failures caught (Finding 8d).
1.8	2026-09-10	Infection routine traced. Polymorphic engine observed as code. .adata gloss withdrawn (13a); fcn.006a2802 shown not to patch the victim (9g).
1.9	2026-09-10	Host-scoped 69.6% functional-coverage figure retired as answering a question the engagement does not ask. Per-program figures substituted.
2.0	2026-09-10	This document. Full report assembled to template; five figures re-themed for light backgrounds; antidebug confirmed-hit count reconciled 8 → 10 against the tool's own output.

Glossary

Included because this report is intended to be readable by someone who is not a reverse engineer. Terms are defined as they are used here.

Authenticode — Microsoft's PE code-signing scheme. The signature covers the file *except* the certificate itself, the SECURITY data directory entry, and the CheckSum field. Those three exclusions are why Finding 1d distinguishes four digest-covered modifications from two that are excluded by specification.

Basic block — a straight-line instruction sequence with one entry and one exit. Reading three instructions is not reading a basic block; Finding 3b is a case where that distinction decided whether a cipher was invertible.

Chi-square (χ^2) — a statistical test of how far an observed byte distribution is from uniform. For 256 symbols the expectation under uniformity is ~255 with sd ~23. Unlike entropy it discriminates cleanly at small window sizes, which is why it, and not entropy, bounded the encrypted region in Finding 1e.

Convergent negative — several independent methods all finding nothing. Corroborating **only** if the methods do not share an assumption. Finding 7f is this report's cautionary case: four methods, non-overlapping with respect to each other, all searching user-mode APIs.

Coverage (of a code region) — the count of **distinct bytes** attributed to some identified function, expressed as an interval union. Never `sum(function sizes) / range`, which double-counts overlaps, and never linear-sweep decode success, which measures whether bytes decode rather than whether they are code.

Defanging — rewriting an indicator so it cannot be accidentally clicked or resolved: `http://` → `hxxp://`, `example.com` → `example[.]com`. Every network value in this report is defanged.

Delta-offset / GetPC — how position-independent x86 code learns its own address, since x86-32 has no PC-relative addressing. A `call` to the next instruction pushes the return address; a `pop` retrieves it. Findings 2a and 4a are both instances.

Entry point (EP) — where the loader transfers control. `AddressOfEntryPoint` in the optional header. Redirecting it is the classic infection technique, and Finding 1f shows the cost: 330 bytes of the host's original code destroyed.

Entropy (Shannon) — average bits of information per byte, capped at 8.0. Over a

256-byte window the mathematical ceiling is nearer 7.17, which is why a threshold of 7.4 (Finding 10a) passed *every possible input*. Entropy is not a discriminator at small window sizes.

FILE_DEVICE_NETWORK (0x15) — the device-type constant passed to IoCreateDevice. **The identifier that found this sample's network component** after four user-mode API searches could not.

HIGHLOW relocation — relocation type 3, the ordinary 32-bit absolute fixup. The infector's relocation walker filters to exactly this type (Finding 9b).

IAT / import table — the table naming which DLL functions a program calls. Malware often avoids it by resolving names at runtime, which is why an import-scoped negative is a weak negative — Finding 8e and \$N4 both turn on this.

ICD-203 — the US intelligence-community standard for probability language. Used here so that "likely" and "almost certainly" mean fixed things.

Indirect call / residual indirect count — a call through a register or memory location, whose target static analysis may not resolve. The count of unresolved ones is **the scope limit on any "no path exists" claim**. For L2 that number is 1.

In-place decryption — decrypting a buffer over itself rather than into a new one. The signature is a load and a store through the same pointer, as in Finding 3a.

IRP / MajorFunction — the Windows kernel's I/O Request Packet and the driver's dispatch table indexed by operation. This driver keys on `MajorFunction == 4` (`IRP_MJ_WRITE`) and a code at `[irp+0x3c]`, which is why Finding 7c withdraws the earlier "IOCTL dispatcher" framing.

LCG (linear congruential generator) — $x = a \cdot x + c$, a cheap pseudo-random generator. `0x15A4E35 = 22,695,477` is the Borland multiplier. **A common library constant, so a match is an implementation detail, not attribution.**

Linear sweep vs. recursive descent — two disassembly strategies. Linear sweep decodes from a start address onward and can be desynchronized by a single misaligned byte (Finding 2a exploits exactly this). Recursive descent follows control flow and misses code it cannot reach. This report's coverage figures use recursive descent plus a prologue sweep, and states the direction of the residual error.

MBC (Malware Behavior Catalog) — a behaviour taxonomy that, unlike ATT&CK,

has Anti-Static Analysis and Anti-Behavioral Analysis objectives. Primary axis of the Capabilities table for that reason.

Multiple comparisons — testing many hypotheses and reporting the one that passed, without correcting for how many were tried. One survivor in ~38 alignment trials is what chance produces (Finding 10c); one section landing correctly out of a 57-page mapping that fails 29 times is not corroboration (Finding 5e).

NRV2B — the LZ-family compression algorithm UPX uses by default. Re-implemented here from the sample's own stub rather than from a published listing, which is how the two flag-provenance defects in Finding 5c were found.

Null model — what the method produces on data known to contain nothing. The XOR-0xf9 blocklist finding rests on one: 400 random patterns × 255 keys → 0 hits.

Parasitic infector — malware that modifies existing executables rather than existing as a standalone file. Its defining consequence, and this report's most important practical finding, is that infected files are **damaged**, not merely contaminated.

PEB (Process Environment Block) — a per-process structure at fs:[0x30] (x86). BeingDebugged at +0x02 and NtGlobalFlag at +0x68 are read by debugger checks — **and by ordinary MSVC runtimes**, which is why Finding 12a does not adjudicate the host's 10 hits as malicious.

Polymorphic engine — code that generates functionally equivalent but byte-different code per instance. Its practical consequence for defenders is stated explicitly in the IOC section: **byte patterns from a polymorphically generated region are specimen-scoped, not family-scoped.**

Positive control — running a method on input known to contain what it looks for, to prove it can fire. **A negative from a method with no firing control is not a result.** This report reports two such runs as void.

repe cmpsb — the x86 string-comparison instruction. Its presence in the resolver (Finding 4b) is what makes the entire hash-sweep negative moot.

SEH (Structured Exception Handling) — Windows exception dispatch. A handler reached only through dispatch **has no call site at all**, so it never enters an indirect-call denominator — the reason seh_roots.py exists, and a stated widening of the scope limit in the code island, which has no .pdata.

Stale pointer (certificate) — the specific anomaly in Finding 1c: content moved, the pointer to it did not. Size matches, offset is short by exactly the insert.

TDI (Transport Driver Interface) — the older Windows kernel interface between transport drivers and clients. This driver receives write IRPs through it rather than through IOCTLs.

Timestomping — restoring a file's original MAC times after modifying it, so a last-modified sweep finds nothing. The GetFileTime/SetFileTime **pair** is the tell (Finding 9e).

UPX — a widely used executable packer. Legitimate as well as malicious; its presence is not itself an indicator.

WIN_CERT — the structure holding a PE's Authenticode certificate. `wRevision = 0x0200` and `wCertType = 0x0002` (PKCS#7) are the valid values this report checks for.

Withdrawal vs. narrowing — a **narrowed** claim was right in a smaller scope than stated (the byte-scoped keystream negative, Finding 3d). A **withdrawn** claim was wrong (that `fcn.006a2802` patches the victim, Finding 9g). This report distinguishes them, because they carry different lessons.

Appendix A: Evidence and Reproduction

All paths are relative to the engagement's analysis directory. Every artefact below exists on disk.

Hash-pinned regeneration

Both scripts pin the sample's SHA-256 and **refuse to run on mismatch**. Neither executes any sample code: both drive Python re-implementations over capstone-decoded instructions on a private register and memory model.

SCRIPT	PRODUCES	PIN
opus/r2_evidence/ regen_plain_body.sh	plain.bin (L2, 61,160 B)	EXPECT=07f7f19d80f9b4758f7ad785ca48a6c90315 1cbb52f0ec14394938bf430414d6
opus/pass2_evidence/ regen.sh	full-chain re- derivation	WANT=f073b23d...c70d59

Disassembly evidence — opus/r2_evidence/

FILE	CONTENTS
01_data_directories.txt	All sixteen PE data directories dumped by name. The artefact that prevents the hand-indexing error of Correction 2.
02_section_chars.txt	Section table with Characteristics decoded — the basis for Finding 1b.
03_reloc_walk.txt	143 relocation blocks, 12,410 entries, content ending at 0xb4b6c — Finding 1e.
04_entrypoint.txt	The 330-byte entry stub, disassembled — Finding 2a.
05_transfer_4418ab.txt	The push ecx / ret transfer, with the register trace.
06_resources.txt	Version resources parsed with pefile — Finding 1a.
07_authenticode.txt	PKCS#7 structure, signing time, chain, and the pristine digest.
08_slack_gaps.txt	Inter-section slack and unattributed regions — the structural-coverage basis.
api_table.txt + gen_api_table.py	The L2 name-to-slot mapping and the k-shift alignment test of Finding 4d.
decryptor_0x4b7630.asm	All 1,122 instructions of the decryptor region.
plain_body.asm + gen_body_asm.py	L2 disassembly, 262 reached instructions across 6 entry points.
cg/	Call-graph extraction output, with the indirect count.

Decoder re-implementations — opus/reimpl/

FILE	PURPOSE
keystream.py	The dword-SUB body decryptor, re-implemented from the sample's own instructions.
keystream_bytemem.py	Byte-granular memory model. Negative control for Finding 6b — produces byte-identical output.
nrv2b.py, nrv2b_stub.py	NRV2B decompression, derived from the stub's instruction stream (Finding 5c).
plain.bin	L2, 61,160 B.
embedded_upx.bin	L3, 46,080 B.
payload_unpacked.bin	L4, 741,648 B.
v4_modulus.bin, netsurface/ v4_pubkey.bin	The 2048-bit modulus (256 B) and the key blob with its 00 01 00 01 prefix (264 B, bd912e...bd7bd695).
carved_embedded_* (×3)	The stub EXE, the infector DLL skeleton, and the kernel driver.
netsurface/ wininet_names.bin	88 B, 471cfe...35a6c6e4 — the complete user-mode network surface (Finding 11a).
netsurface/url_pack.bin	146 B, 7fccb8...2b4fdb3 — the L4 URL pack.
netsurface/ fmt_and_probe.bin	108 B, 88b441...916cd90 — format strings and the connectivity probe.
NOTES.md	Per-artefact derivation notes.

Interpreter evidence — opus/pass2_evidence/

FILE	PURPOSE
nrv2b_stub_model.py	Symbolic model of the UPX stub, from which every decompression parameter was read.
stage_interpreter.py	The instruction interpreter. Full decryptor run: 187,761,768 steps, terminated normally at the sample's own ret, 368.7 s.
store_probe.py	Memory-write instrumentation — the basis for Finding 12e's 40 / 4,000 / 40,000-event progression.
upx_control.m	The two-sided completeness bound of Finding 5a.

Figures — opus/figures/

fig1_host_file_layout.png · fig2_decrypt_chain.png · fig3_payload_map.png · fig4_four_programs.png · fig5_infection_routine.png, each with its build_fig*.py generator and the shared _style.svg. All five are regenerated from the numbers in this report, not drawn by hand.

Appendix B: Methodology

The pipeline

`analyze.sh` steps 0–11 produce the baseline: identification and hashing, PE structure, sections and entropy, imports and exports, resources, strings, YARA/capa, regex IOC extraction, r2inspect JSON, Ghidra headless decompilation, and language/toolchain detection. **All of it is hypothesis generation.** Finding 12g records what that baseline said about this sample on its own: a clean, signed Oracle binary with not one malicious indicator.

Everything of substance in this report came from what followed.

The layer-by-layer method

Each transition was handled the same way, and the discipline is the point:

1. **Locate the transition mechanism in the sample's own instructions** — the entry stub, the single memory-writing instruction in the decryptor region, the UPX stub's parameter loads.
2. **Re-implement it in Python.** Never execute it. A wrong model then yields a decode failure instead of running sample code.
3. **Bound the output on both sides where possible.** The UPX carve is the model case: the PE's own section table gives one bound, the stub's declared `p_filesize` gives another, and the bit stream terminating three bytes before the stub entry gives a third.
4. **Verify with a positive control before believing any negative.**
5. **Measure coverage as distinct bytes**, and state the residual indirect count.

Dual-agent, union merge

Where two agents were available they worked independently and were **never seeded with each other's findings**. Results merged by **union**, with the grounding gate as arbiter. Finding 6a is what that buys: two independently written decryptors whose outputs a naive comparison called a total mismatch, resolved to a constant `0x10e6` displacement and then **0 mismatches over 56,828 bytes** — a corroboration no single implementation could have produced.

It also buys corrections. The entry-transfer retraction of Finding 2b came from an adversarial second pass, not from re-reading the first.

Scoping every negative

Every negative in this report states the encoding, operand width, call-edge type or decoder scheme it was searched under, plus its positive control. Three of this engagement's largest errors were negatives that were true in a narrower scope than they were stated in:

- The keystream negative was measured at **byte** granularity against a **dword** cipher (Finding 3d).
- The "no static reference reaches the blob" negative was scoped to **static immediates** against **computed** control flow (Finding 2b).
- The network-component negative was scoped to **user-mode APIs** against a **kernel driver** (Finding 7f).

None was wrong. All three were narrower than the sentence that carried them.

Instrument validation

Positive controls were run **before** believing negatives, and two runs are reported as void on that basis (§N4). Three silent tool failures were caught (Finding 8d), each producing a plausible number rather than an error. Two working measurements were caught by **internal contradiction** rather than by external comparison — a covered-bytes figure equal to the largest single function, and a coverage figure exceeding 100% of nonzero bytes.

The general rule the engagement kept re-learning: **audit a classifier by its output counts, not its verdict**. A gate that extracts zero values passes.

What was deliberately not done

No execution. No network contact. No submission of any artefact to any online service. No acquisition of the missing downloaded stage — that is a sourcing decision with off-machine consequences, flagged and never initiated.

Appendix B2: Tools and What Each Is Trusted For

TOOL	TRUSTED FOR	NOT TRUSTED FOR	FAILURE SEEN HERE
pefile	Enumerating data directories, sections, imports, resources by name	Anything hand-indexed	Has the same .pdata scope-table defect as seh_roots.py did — the cross-check found a bug in the checker.
radare2	Disassembly bytes; axt cross-references at aaa depth	Cross-references landing in string blobs; any config value not verified against its own enumeration	anal.in=raw accepted as a silent no-op; -m 0x69b000 collapsed 178 functions → 1; the documented range-bounding form suppressed the prologue sweep entirely (0 vs 177).
capstone	Instruction decode, mode-pinned	Nothing beyond decode	Correctly in CS_MODE_32; the <i>detail string</i> printed x64 register names, which is a display defect in the consumer, not in capstone.
capa	Candidate capabilities	Any reported finding	Contributed no row to the Capabilities table.
Ghidra (headless)	Pseudocode as a reading aid	Any claim in this report	—
signify	Whether a signature verifies	Whether a certificate is <i>present</i>	Correctly reports no signature; that answer alone would hide the tampering (Finding 1d).
antidebug_scan.py	The confirmed tier	The candidate tier	1,701 of 1,711 hits are noise; 57% of sampled PEB candidates are memory <i>writes</i> , which cannot be probes (Finding 12a).
build_callgraph.py	Direct call edges + the unresolved count	Reachability without citing that count	The indirect count is a floor — thunks and SEH entries are not call sites at all.

TOOL	TRUSTED FOR	NOT TRUSTED FOR	FAILURE SEEN HERE
seh_roots.py	Handler roots on PE64 with .pdata	The code island (no PE header, so no .pdata at all)	Contributes no roots here, which widens the scope limit rather than narrowing it.
cover_check.py / gate/	Grounding IOC values against the sample	Passing as evidence of quality	A gate extracting zero values also passes. Read the counts.
VirusTotal (consulted, nothing submitted)	An independent implementation's structural verdict	Family attribution, capability claims	Reports "not signed" — correct, and for the reason in Finding 1c.
Published vendor reporting	Family-level context, tagged [intel]	Anything applied to this build's bytes	An RC4 step correctly reported for the family, applied to the wrong stage here, produced a plausible wrong decode (Correction 21).

Every tool in this table produced at least one confident, plausible, wrong answer during this engagement. That is the reason for the middle column.

Appendix C: Corrections Made During Audit

Every correction that changed a claim. **Kept as a log:** each entry states what was believed, what is true, and how the error was caught. Entries are not overwritten when later work supersedes them — they are annotated in place, because a corrections log that quietly rewrites itself is not a log.

#	WAS BELIEVED	IS TRUE	HOW IT WAS CAUGHT
1	13 imported DLLs	12	Recount against pefile's enumeration.
2	The DEBUG directory's contents indicated tampering	It is the DEBUG directory, index 6 at 0x85490, read as SECURITY by a hand-indexed offset	Enumerating all sixteen directories by name. Produced the standing rule.
3	axt returns no callers into the overwritten range, so the raw scan is wrong	The raw scan is right; axt needed aaa depth, not aa	Two methods re-run at matching depth; they then agree exactly. Third instance of a cross-check being wrong where the tool was right.
4	or ecx,[ebx] destroys information, so the cipher is not invertible	ecx == 0 at the load — two identical adjacent XORs cancel — so it is mov	Reading the basic block , not three instructions. Byte pattern confirmed to occur exactly twice.
5	90 rounds over 16,384 dwords	2 dwords per outer pass × 32 rounds; 7,644 passes, 489,216 stores	Tracing ebx and esi at the store across two complete passes instead of reading the loop bound.
6	The interpreter cannot reach termination	It terminates normally at the sample's own ret — 187,761,768 steps, 368.7 s	Re-reading the background output after it finished. An unfinished run and a failed one look identical if you check too early.
7	L2 prologue is call/pop/sub 0x401000, no mov eax,ebp	There is a mov eax,ebp at offset 6, and the constant is 0x401005	Byte-level re-read.

#	WAS BELIEVED	IS TRUE	HOW IT WAS CAUGHT
8	The driver attacks the legitimate IPFilter stack	It attaches itself to TCP/IP; the device name is camouflage	The import set is a filter's, not a tamper tool's.
9	The driver dispatches on IOCTLs	It keys on IRP_MJ_WRITE and a code at [irp+0x3c]	Zero 0x22xxx constants anywhere in the driver.
10	The XOR-0xf9 band is either an obfuscated table, or 0xf9 is not a key at all	0xf9 is the key; the truncation seen one layer up is NRV2B literal-run structure	Hexdumping the raw bytes instead of reasoning about the encoding. Entries are NUL-terminated in the ciphertext . Note the crib sality-remov was true by construction and could not confirm the hypothesis it was used to test.
11	The 90 KB region carries no crypto	MD5 constants at +0x12858, CRC32 0xEDB88320 at +0x0229c, MD5Init at +0x1283c on a live call chain	The original claim was inferred from absent Crypt* imports and never measured . A constant scan contradicted it.
12	fcn.006a2802 performs the infecting write (mmap → patch → SetFileTime)	It performs mmap → read → write elsewhere . No store into the mapped view exists in the routine.	Enumerating every memory store in 0x6a2802–0x6a2bca. The original inference was string-and-import shaped.
13	[0x54010000] is SetFileTime	It takes one argument — a path buffer — after the handle is closed	Argument-shape check against MSDN arity.
14	The modulus scan found nothing	The entropy threshold 7.4 was above the mathematical ceiling for a 256-byte window, so the test passed every possible input, including a planted blob	Running the positive control before believing the negative. A second version then silently dropped the planted site through faulty de-duplication.

#	WAS BELIEVED	IS TRUE	HOW IT WAS CAUGHT
15	The 128-byte blob opens with a length dword of 226	Misalignment. 0xe2 is the last byte of the preceding record; there is no length field	Re-reading at the natural record boundary.
16	Static imports are KERNEL32, USER32, GdiPlus, NTDLL, ADVAPI32, WININET, ipfltdrv.sys	That list is a union of two different mechanisms presented as one. Static: KERNEL32, USER32, ADVAPI32, SHELL32 — no WININET. WinINet is runtime-resolved.	Separating the static import table from the runtime name block.
17	IPFIL + RDRI at plain.bin 0x557c indicates a partially corrupt decrypt	Pareidolia. Short letter runs in high-entropy data; TE and VE absent, spacing wrong	Withdrawn after a corruption theory and one full re-run were built on it. Not a blind search , so the multiple-comparisons correction is large.
18	82% of printable dwords in 0x4c00-0x5700 are corrupt	The statistic is real and meaningless — that band is the UPX-compressed body, where a correct decrypt is indistinguishable from noise	Chi-square. Printability has no discriminating power on compressed data. A dependent claim about a wrong keystream subset is withdrawn with it.
19	The decryptor loop covers <1% of the ciphertext, or decrypts nothing at all	It covers all of it; scales linearly and closes with the static bound	Under-sampling. 40 events → "inert", 4,000 → "<1%" (this reached a draft as a retraction), 40,000 → correct. Compounded by an interpreter started at stage 2 with no stage-1 stack, so every store landed outside the filter's range and it reported "0 writes".
20	.adata is the section name the infector writes into its hosts	It is one entry in a list bracketed by AV-product tokens and file-open API names — a hook/exclusion list. The infection marker is now unidentified.	Dumping the string's neighbours. Zero xrefs under three addressing models against a firing 3,420-xref control.

#	WAS BELIEVED	IS TRUE	HOW IT WAS CAUGHT
21	The family's published RC4 step applies to this sample's body	It belongs to a different stage of the family's chain. This body's cipher is 32-bit SUB with no RC4 key schedule	Tracing the single memory-writing instruction in the region. Published reporting describes a family across builds; a sample is one build.
22	Functional coverage is 69.6%	Retired, not corrected. It was measured on the benign host — 2,691 functions of C++/ATL — and no coverage figure had ever been produced for any malware body	It answered a question the engagement does not ask. Same defect class as the entropy threshold: a measurement that cannot distinguish the two outcomes it decides between. Per-program figures substituted.
23	Island coverage is 10.8%	85.4% of range, 99.5% of nonzero, zero overlap	A JSON key mismatch (addr vs offset, defaulting to 0) stacked all 178 functions at address 0. Caught by internal contradiction: covered bytes equalled the largest single function.
24	The payload embeds three analysable programs	Four programs, one of which — the infector DLL — is a skeleton : 1,285 of 121,856 declared bytes present (1.1%)	Content-density measurement. A section header states an intended layout; it does not promise the bytes exist.
25	The antidebug scan confirmed 8 hits	10. The two additional are the paired PEB self-references at 0x454241 and 0x45e93b	Reconciled against the tool's own output before publication. The earlier count folded each self-reference into its dependent read. Same defect class as counting a loop's iterations as one check.

#	WAS BELIEVED	IS TRUE	HOW IT WAS CAUGHT
26	The PE validator's compares are 16-bit: <code>cmp dx, 0x5a4d</code> and <code>cmp cx, 0x10b</code> , the latter encoded 66 81 f9 0b 01 at VA 0x6a2603	All four compares are 32-bit: <code>cmp edx, 0x5a4d</code> (81 fa 4d 5a 00 00) and <code>cmp ecx, 0x10b</code> (81 f9 0b 01 00 00), at VA 0x6a2676 — 0x6a2603 is the function entry, not the compare). A fourth compare, a second PE check at 0x6a2646, was missing from the listing entirely	The grounding gate . <code>cover_check.py</code> reported the byte string occurring nowhere in the sample; the encoding had been written from the mnemonic rather than read off the bytes, and the 66 operand-size prefix was assumed. Counting confirmed 0 occurrences of both 66 81 f9 0b 01 and 66 81 3f 0b 01, while the neighbouring cited patterns verified (69 c0 35 4e 5a 01 ×1, c7 05 ×15); capstone at island offset 0x28507 then gave the true instruction. Re-disassembling the whole function then showed the same error in the MZ compare and a dropped instruction — the transcription had been narrowed to fit the constants, not read off the encoding. The semantic reading — PE32 magic, 32-bit targets only — is unaffected; the widths, one address and the instruction count were wrong. A value that was never verified is not made true by the conclusion it supports being right, and fixing only the byte the gate flagged would have left the identical defect one line above it.
27	Encoding kaspersky under XOR 0xf9 gives 92 98 8a 89 8a 8b 8e 80 a0 (Detection Opportunities' worked example)	92 98 8a 89 9c 8b 8a 92 80. The printed value is correct for the first four bytes and wrong from byte 5 onward	Re-deriving all five blocklist strings from the artefact (<code>opus/reimpl/payload_unpacked.bin</code>) while writing the rule , rather than copying the report's prose into it. The band at 0x2088c decodes to all 23 vendor entries and each chosen string was confirmed <code>present=True</code> . A worked example is a value like any other and needs the same grounding as a table cell — this one sat in a section whose entire purpose is to be copied into a rule, so the defect would have shipped as a rule matching nothing.

#	WAS BELIEVED	IS TRUE	HOW IT WAS CAUGHT
28	Certificate arithmetic to key on is cert_offset + cert_size == filesize	That is the clean condition. This sample reads 0xb4c00 + 0x2858 = 0xb7458 against a filesize of 0xc8458 — short by exactly 0x11000 , the insert size. The malicious tell is the pointer going stale , i.e. the inequality	Reading the data directory and the filesize directly while writing the structural rule. The row's own second clause (SECURITY.VA + size != filesize) already stated the correct test, so the row contradicted itself and the equality half would have matched ordinary signed files while missing this one. A row that states both a condition and its negation is not a detail to reconcile later; one half of it is wrong.

Two entries are annotated rather than rewritten. Correction 15 records the length-field misalignment; the blob's identity was **later** closed by exclusion (Finding 10c) rather than resolved, and the entry above is left as written because it describes what was believed at the time. Correction 5e in the Technical Analysis has the same shape: the .CRT completeness argument was **disproved as reasoning while its conclusion survived on a different basis**, and both halves are stated.

Appendix D: Analysis Scripts

Every decoder in this chain was re-implemented rather than executed. This appendix lists what was written, what it does, and — where one exists — what proves it correct.

`keystream.py` — the body decryptor

Re-implements the 32-bit subtractive stream cipher of Finding 3a from the sample's own instructions, driving a capstone-decoded instruction interpreter over a private register and memory model. **No sample code executes.**

Correctness evidence: an independently written second implementation produces output that, after the 0x10e6 displacement of Finding 6a is accounted for, matches over **56,828 bytes with 0 mismatches**. Output hash pinned in `regen_plain_body.sh`.

`keystream_bytemem.py` — the negative control

Identical, with a byte-granular memory model instead of a dword-keyed one. Written to test the hypothesis that the dword keying caused the two implementations' disagreement.

Result: **byte-identical output**, and instrumentation over 3 million steps shows 14,701 exact hits, 7,881 clean image reads, **0 overlap misses**. The hypothesis was refuted before it was believed. Retained because a negative control that was never run proves nothing.

`nr2b.py` / `nr2b_stub.py` — UPX decompression

NRV2B, LE32 shift-register bit order, MSB-first, sentinel refill, plus the E8/E9 unfilter. **Every parameter is read out of the sample's own stub**, not from a published listing — which is how the two defects in Finding 5c were found, both invisible unless branch *targets* are read rather than mnemonics.

Correctness evidence, three-sided: output length 0xb5110 matches the stub's declared `p_filesize` **exactly**; the unfilter performs **1,167** fixups matching the stub's hardcoded `ecx = 0x48f`, a count reached by scanning decompressed bytes and therefore not circular; and with `base = 0`, **1,167 of 1,167** rewritten targets land in-image, against 1,133 at 0x1000 and 0 at the VA base.

nrv2b_stub_model.py — symbolic stub model

Traces the stub's register state symbolically to recover the compressed-stream start, the method, the level and p_filesize. Necessary because the p_info/b_info header fields **do not parse at any alignment** (Finding 5b) and were not used.

stage_interp.py — the instruction interpreter

Interprets capstone-decoded instructions over a private register and memory model. Used for the full decryptor run: **187,761,768 steps, terminated normally at the sample's own ret at 0x4b845c**, no halt, no step cap, ~497,694 steps/sec, 368.7 s.

Known model limitation, stated because it mattered: imul leaves ZF, SF and PF **undefined** on x86. Two interpreter models disagreed on ZF at 0x4b83de (64/3,000 vs 0/7,527) and **neither is incorrect**. The divergent path leaves edi different, and edi is dead at the decrypt, which reads esi — fixed before the branch. Confirmed empirically: esi == edx in **7,527 of 7,527** (Finding 12f).

store_probe.py — memory-write instrumentation

Records every store with its address and value. The instrument behind Finding 12e, and behind the discipline that finding produced.

The lesson it taught, recorded because it produced this engagement's largest single error: **a short capture of a nested loop samples the inner level and mimics a stuck outer one**. 40 events said "decrypts nothing"; 4,000 said "<1% of ciphertext" and reached a draft as a retraction; 40,000 gave the correct answer, which scales linearly and closes with the static bound. A related defect — starting the interpreter at stage 2 without a stage-1 stack — put every store outside the filter's address range and made the loop look inert.

Blocklist decoder (in regen.sh)

Splits the 284-byte band at L4 0x2088c on \x00 **in the ciphertext**, then XORs each entry with 0xf9, then reverses the order.

Two failed approaches are recorded because both produced a single merged entry that looked like a decode failure rather than a method failure: XORing the whole blob then splitting on \x00; and splitting the ciphertext on \xf9. **The fix was hexdumping the raw bytes rather than reasoning about the encoding.**

Null model: 400 random 12-byte patterns × 255 single-byte keys → **0 hits**.

gen_api_table.py — L2 name-to-slot mapping

Pairs each NAME\0 with the 4-byte slot immediately following it, then **tests the alignment rather than assuming it** by shifting the mapping by *k* and scoring pushed-argument counts against MSDN arity: *k* = −1 gives 1/12, ***k* = 0 gives 13/14**, *k* = +1 gives 0/16. This is what settled the one ambiguous entry (CloseHandle, not GetLastError) in Finding 4d.

gen_body_asm.py — L2 disassembly

Recursive descent from the six entry points, emitting the 262 reached instructions and counting indirect transfers. **The reached set, not a linear sweep, is the denominator for §N6** — a linear sweep at every byte offset produces thousands of spurious self-modification hits from mid-instruction decode garbage.

Coverage measurement (island)

Computes coverage as an **interval union of distinct bytes**, never `sum(size)/range`, across four r2 configurations, and reports overlap as a defect signal. The `hasnext=1` configuration exceeding 100% of nonzero bytes with 15 bytes of overlap is exactly the artefact that discipline is designed to expose.

End of report.